

분산처리

A Modern Multi-Core Processor



Computer & Ai

Department of Computer Engineering

A program is a list of processor instructions!

```
int main(int argc, char** argv) {  
  
    int x = 1;  
  
    for (int i=0; i<10; i++) {  
        x = x + x;  
    }  
  
    printf("%d\n", x);  
  
    return 0;  
}
```



Compile
code



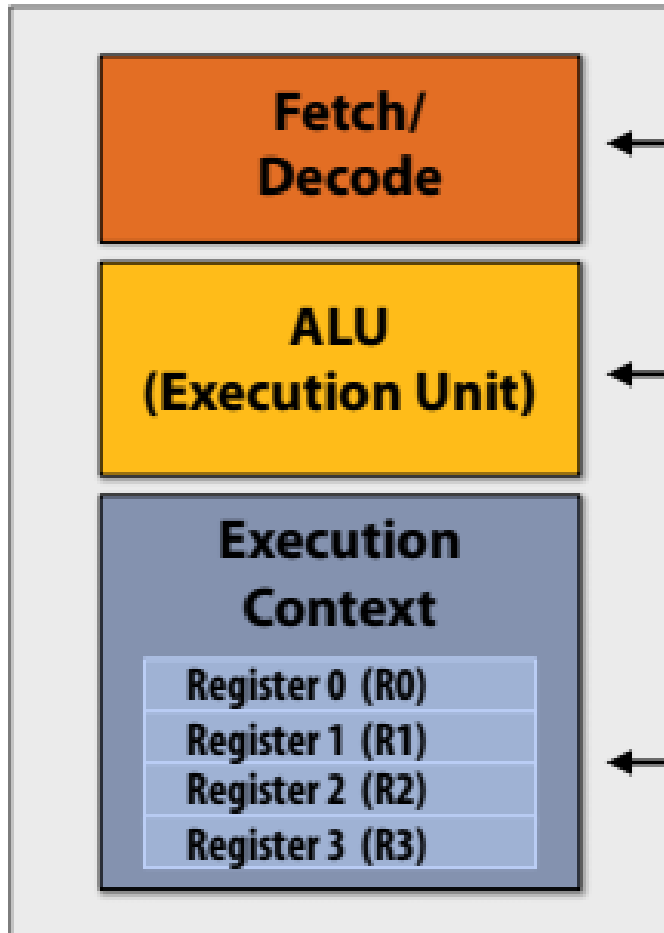
```
_main:  
100000f10:    pushq    %rbp  
100000f11:    movq     %rsp, %rbp  
100000f14:    subq     $32, %rsp  
100000f18:    movl     $0, -4(%rbp)  
100000f1f:    movl     %edi, -8(%rbp)  
100000f22:    movq     %rsi, -16(%rbp)  
100000f26:    movl     $1, -20(%rbp)  
100000f2d:    movl     $0, -24(%rbp)  
100000f34:    cmpl     $10, -24(%rbp)  
100000f38:    jge      23 <_main+0x45>  
100000f3e:    movl     -20(%rbp), %eax  
100000f41:    addl     -20(%rbp), %eax  
100000f44:    movl     %eax, -20(%rbp)  
100000f47:    movl     -24(%rbp), %eax  
100000f4a:    addl     $1, %eax  
100000f4d:    movl     %eax, -24(%rbp)  
100000f50:    jmp      -33 <_main+0x24>  
100000f55:    leaq     58(%rip), %rdi  
100000f5c:    movl     -20(%rbp), %esi  
100000f5f:    movb     $0, %al  
100000f61:    callq    14  
100000f66:    xorl     %esi, %esi  
100000f68:    movl     %eax, -28(%rbp)  
100000f6b:    movl     %esi, %eax  
100000f6d:    addq     $32, %rsp  
100000f71:    popq     %rbp  
100000f72:    rets
```

프로세서의 역할은 무엇인가요?



프로세서는 명령어를 실행

Very Simple Processor



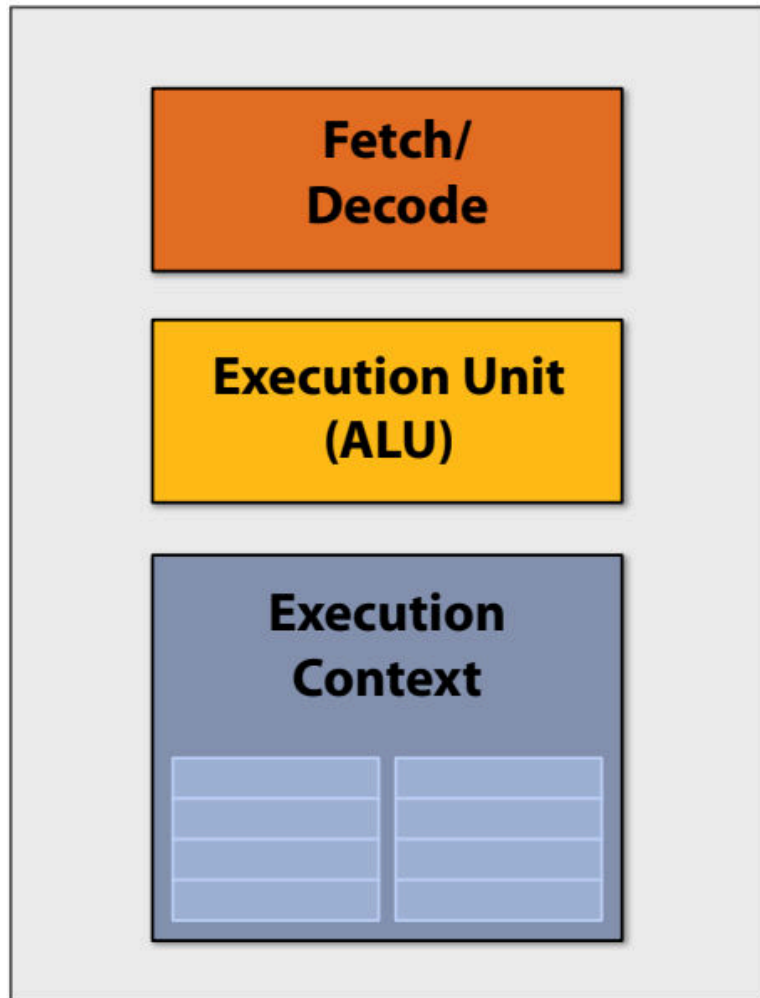
- 다음에 실행할 명령어를 결정.

- 실행 단위: 명령어로 기술된 작업을 수행하며, 프로세서 레지스터 또는 컴퓨터 메모리의 값을 수정할 수 있음

- 레지스터: 프로그램 상태 유지: 연산에 입력 및 출력으로 사용되는 변수의 값을 저장.

프로그램 실행

아주 간단한 프로세서: 클럭당 하나의 명령어 실행



```
ld    r0, addr[r1]
```

```
mul   r1, r0, r0
```

```
mul   r1, r1, r0
```

```
...
```

```
...
```

```
...
```

```
...
```

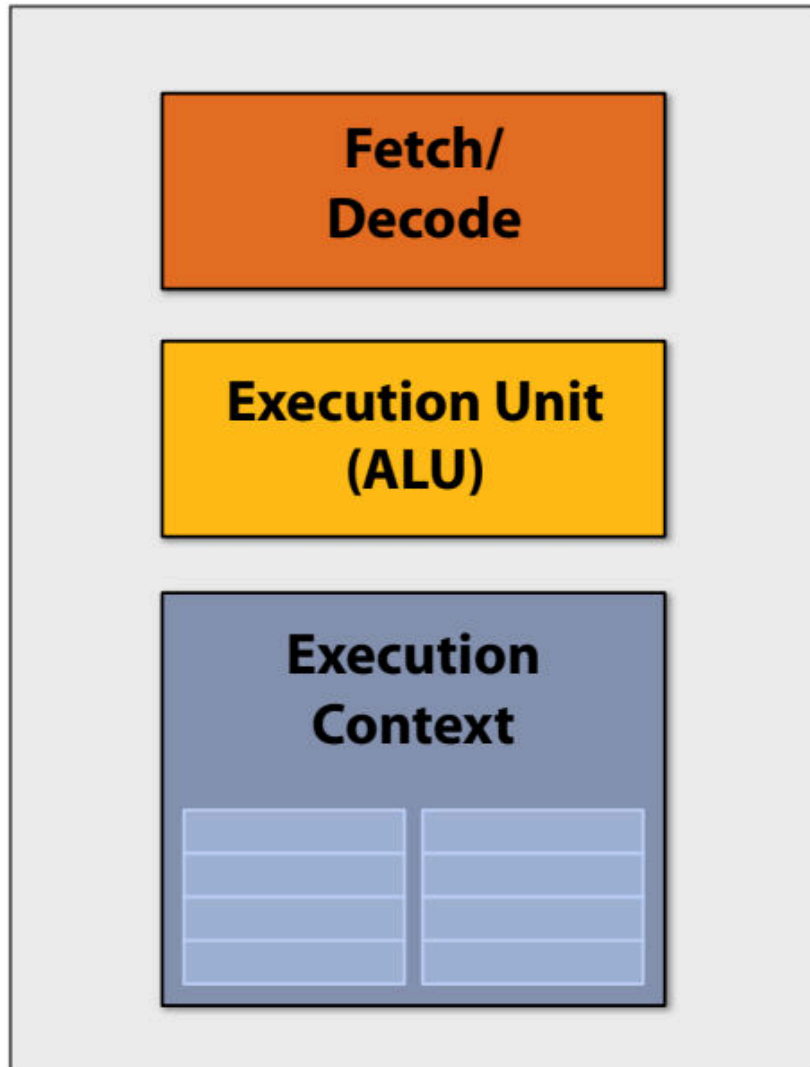
```
...
```

```
...
```

```
st    addr[r2], r0
```

프로그램 실행

아주 간단한 프로세서: 클럭당 하나의 명령어 실행



```
ld  r0, addr[r1]
```

```
mul r1, r0, r0
```

```
mul r1, r1, r0
```

```
...
```

```
...
```

```
...
```

```
...
```

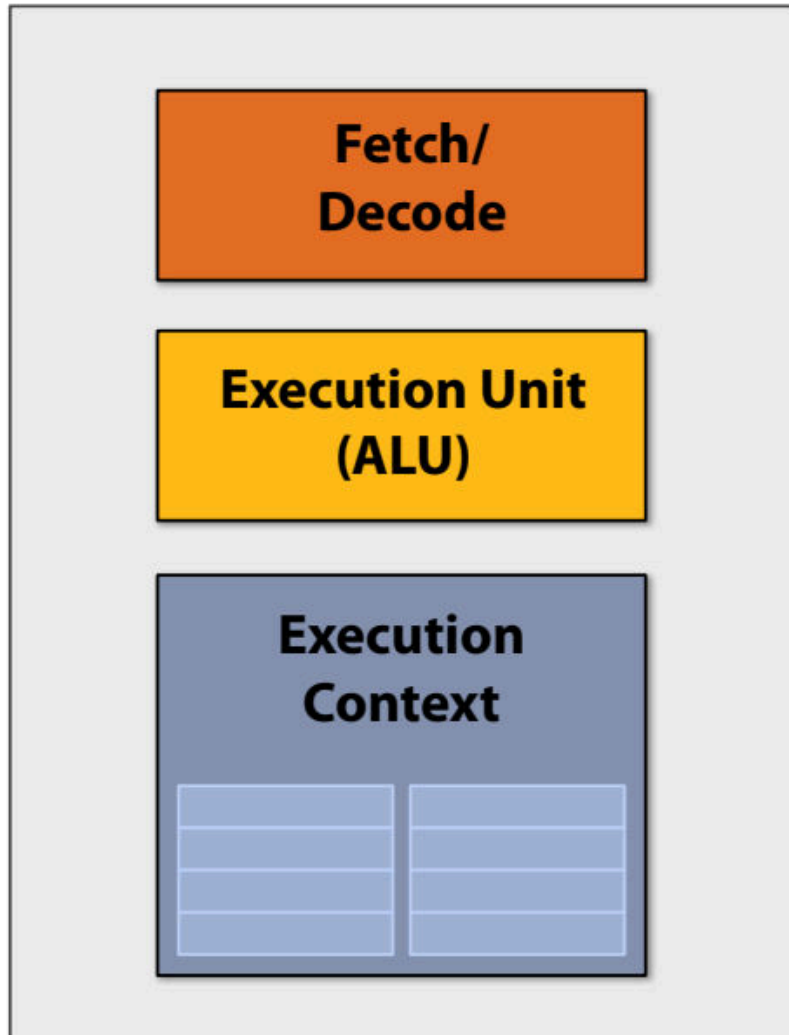
```
...
```

```
...
```

```
st  addr[r2], r0
```

프로그램 실행

아주 간단한 프로세서: 클럭당 하나의 명령어 실행



```
ld    r0, addr[r1]
```

```
mul   r1, r0, r0
```

```
mul   r1, r1, r0
```

```
...
```

```
...
```

```
...
```

```
...
```

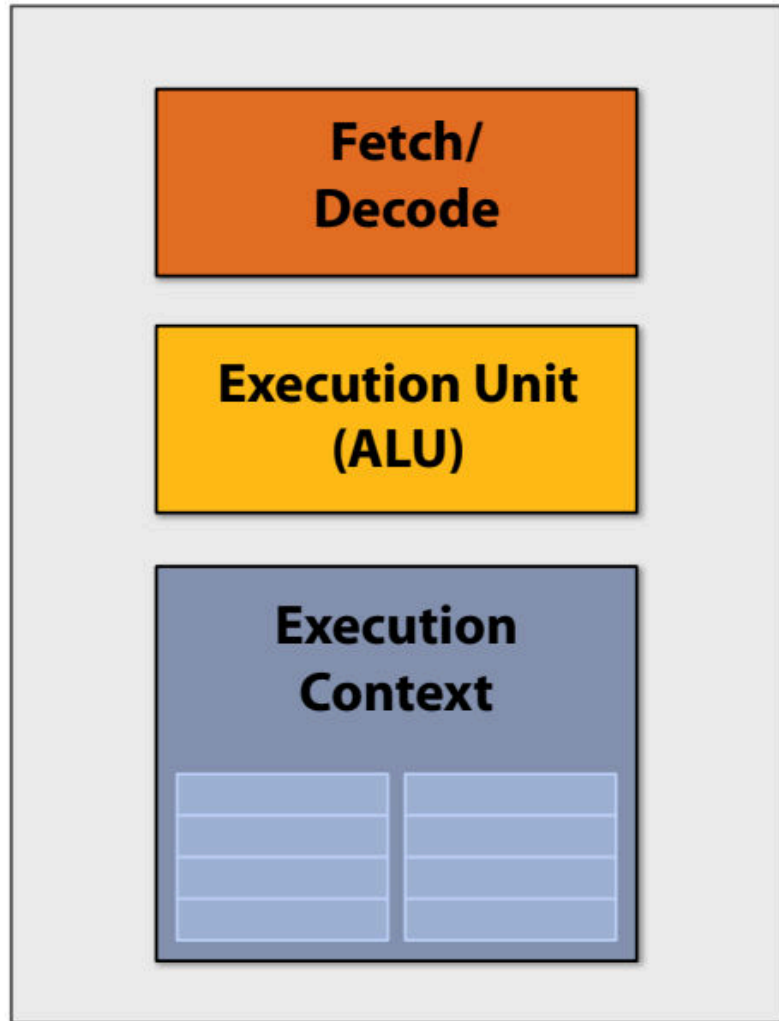
```
...
```

```
...
```

```
st    addr[r2], r0
```

프로그램 실행

아주 간단한 프로세서: 클럭당 하나의 명령어 실행



```
ld    r0, addr[r1]
mul   r1, r0, r0
mul   r1, r1, r0
...
...
...
...
...
...
st    addr[r2], r0
```

명령어 수준의 병렬성을 갖춘 프로그램

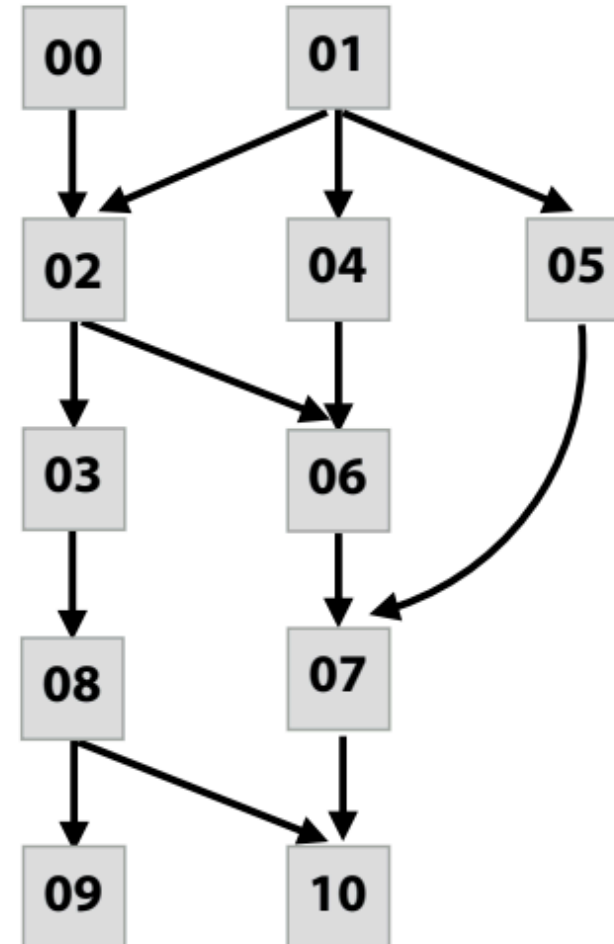
Program (sequence of instructions)

PC	Instruction	
00	a = 2	
01	b = 4	
02	tmp2 = a + b	// 6
03	tmp3 = tmp2 + a	// 8
04	tmp4 = b + b	// 8
05	tmp5 = b * b	// 16
06	tmp6 = tmp2 + tmp4	// 14
07	tmp7 = tmp5 + tmp6	// 30
08	if (tmp3 > 7)	
09	print tmp3	
	else	
10	print tmp7	

Computed value

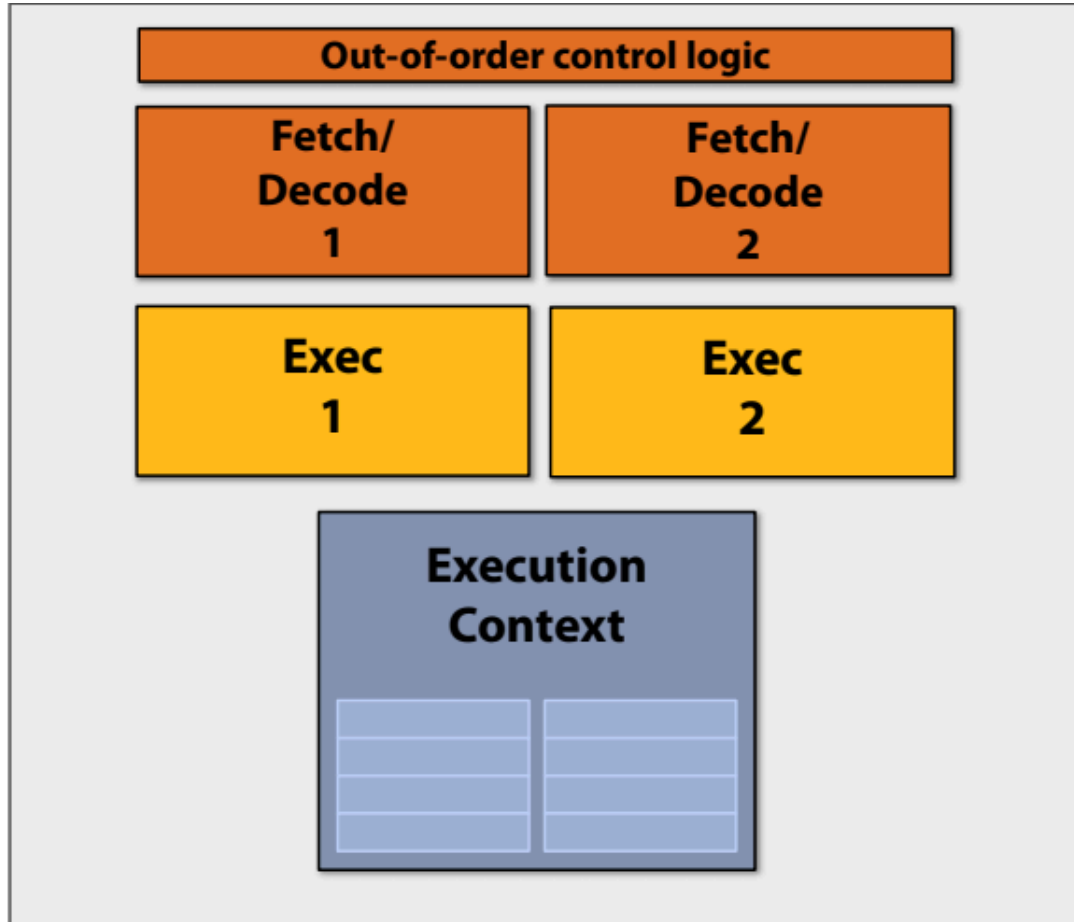


Instruction dependency graph



Superscalar processor

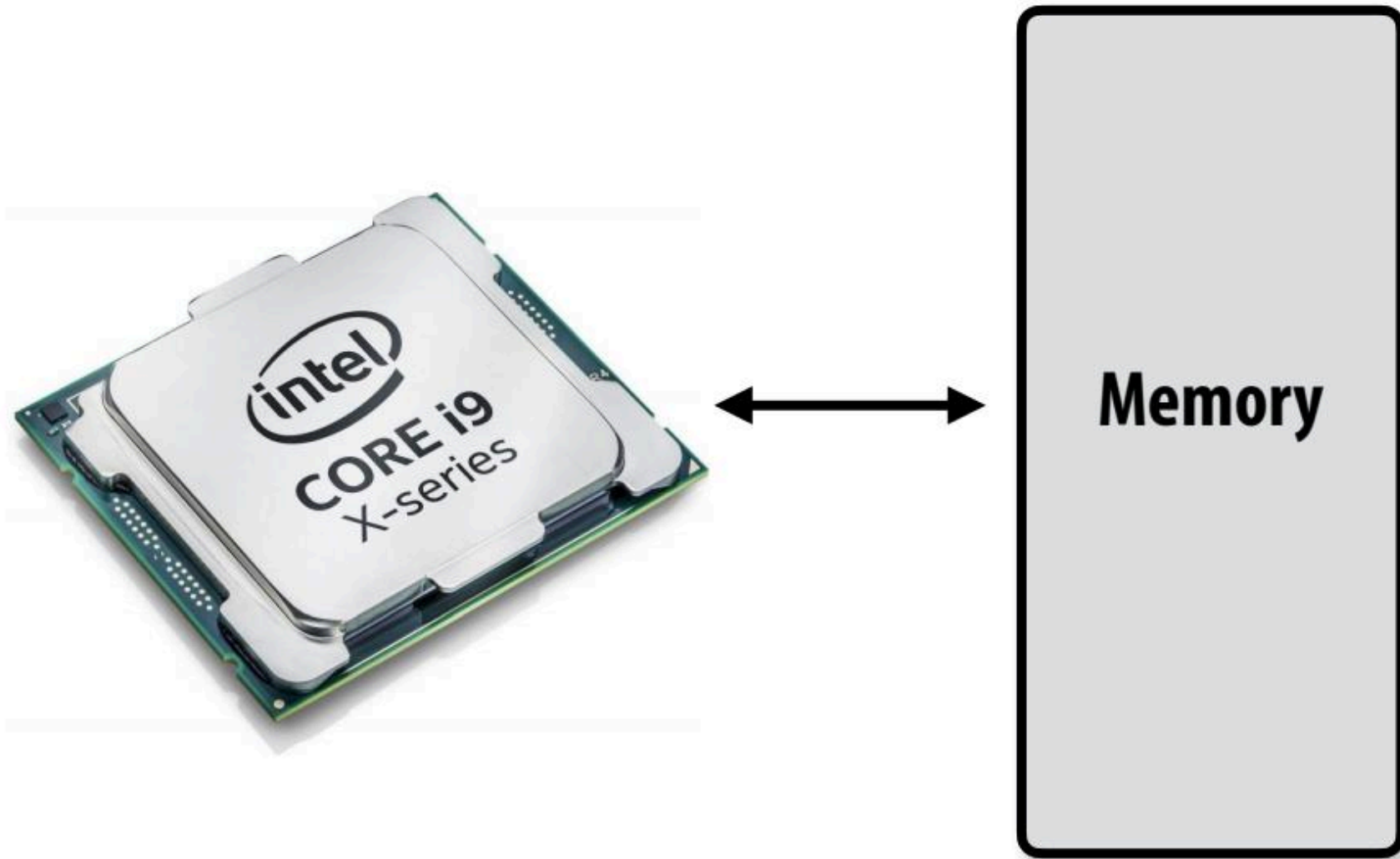
이 프로세서는 클럭당 최대 2개의 명령어를 디코딩하고 실행가능



슈퍼스칼라 실행: 프로세서가 명령어 시퀀스에서 독립적인 명령어를 자동으로 찾아서 시퀀스에서 독립적인 명령어를 생성하고 여러 실행 단위에서 병렬로 실행할 수 있습니다.

슈퍼스칼라 프로세서가 "프로그램 순서를 존중"한다는 것은 무엇을 의미할까요?

What is memory?



- 소프트웨어 엔지니어의 관점에서 바라본 컴퓨터 아키텍처에 대해서 살펴보자
- 최신 병렬 프로세서가 높은 처리량을 달성하는 방법에 대한 핵심 개념
 - 두 가지는 병렬 실행(멀티코어, SIMD 병렬 실행)에 관한 것.
 - 하나는 메모리 지연 시간(멀티 스레딩)의 문제를 다룸.
- 이러한 기본 사항을 이해하면 다음과 같은 도움이 됨
 - 병렬 프로그램의 성능 이해 및 최적화
 - 어떤 워크로드가 빠른 병렬 머신에서 이득을 볼 수 있는지에 대한 직관력 향상

Today's example program

- N개의 Floating-Point 배열의 모든 Sin 값을 구하는 예제를 사용하려고 한다.
- gcc로 컴파일 되는 아래 프로그램은 단순히 하나의 Processor에서 하나의 Thread로 실행될 것이다.
(현재의 코드는 아무런 Parallelism이 표현되어 있지 않다.)

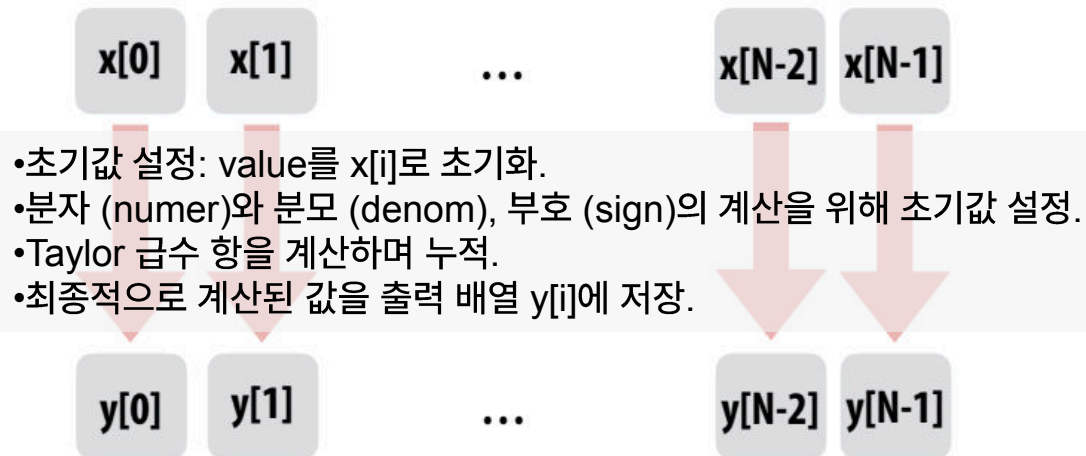
```
void sinx(int N, int terms, float* x, float* y)
{
    외부 루프: 배열의 각 요소 x[i]를 순회하며 작업을 수행
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;
        내부 루프: Taylor 급수의 각 항(term)을 계산하고 누적하여 근사값을 구함
        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

Compute $\sin(x)$ using Taylor expansion:

$$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \dots$$

for each element of an array of N floating-point numbers



- 입력 배열 x 의 각 요소에 대해 사인 값을 계산하고, 그 결과를 출력 배열 y 에 저장
- Taylor 급수를 활용하여 사인 값을 근사 계산
- 배열의 각 요소에 대한 계산은 독립적이므로, 병렬 처리를 통해 성능을 크게 향상시킬 수 있음

Compile program

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

compiler

Compiled instruction stream
(scalar instructions)

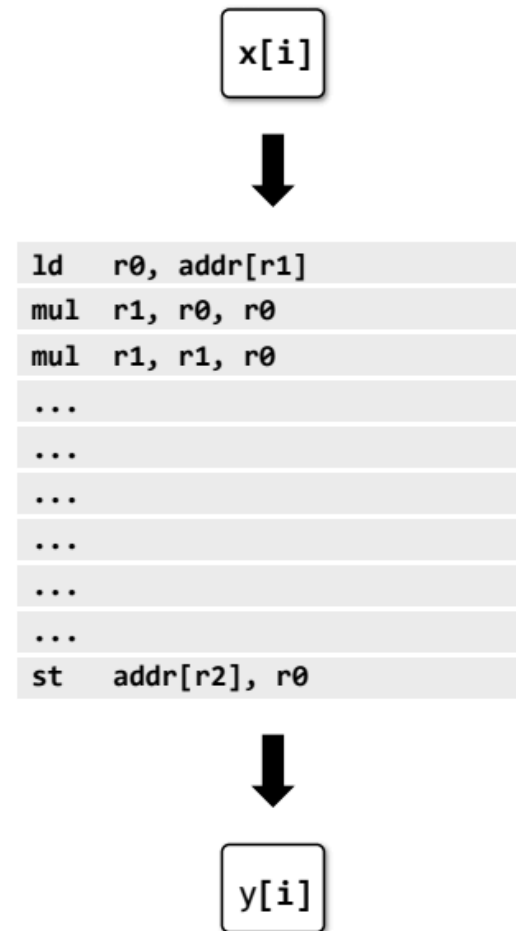
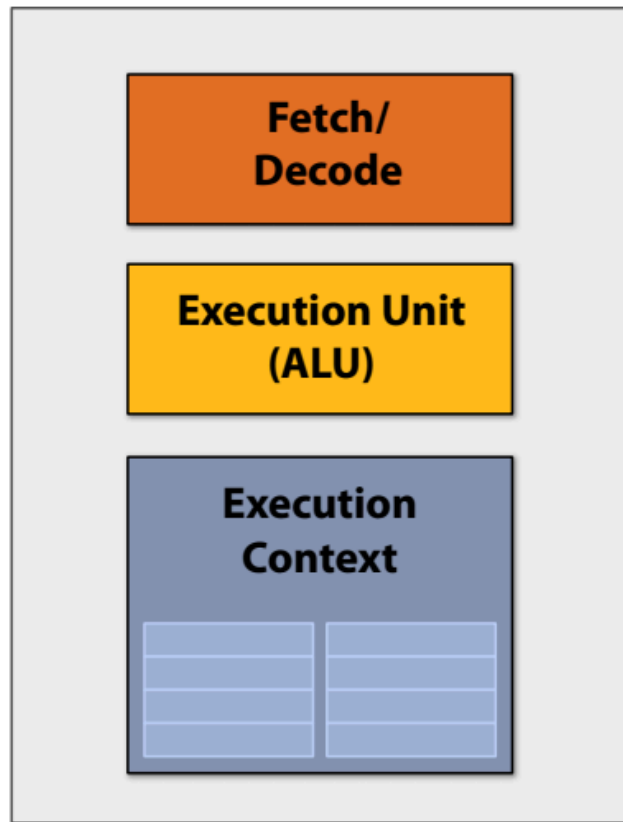
x[i]

```
ld  r0, addr[r1]
mul r1, r0, r0
mul r1, r1, r0
...
...
...
...
...
st  addr[r2], r0
```

y[i]

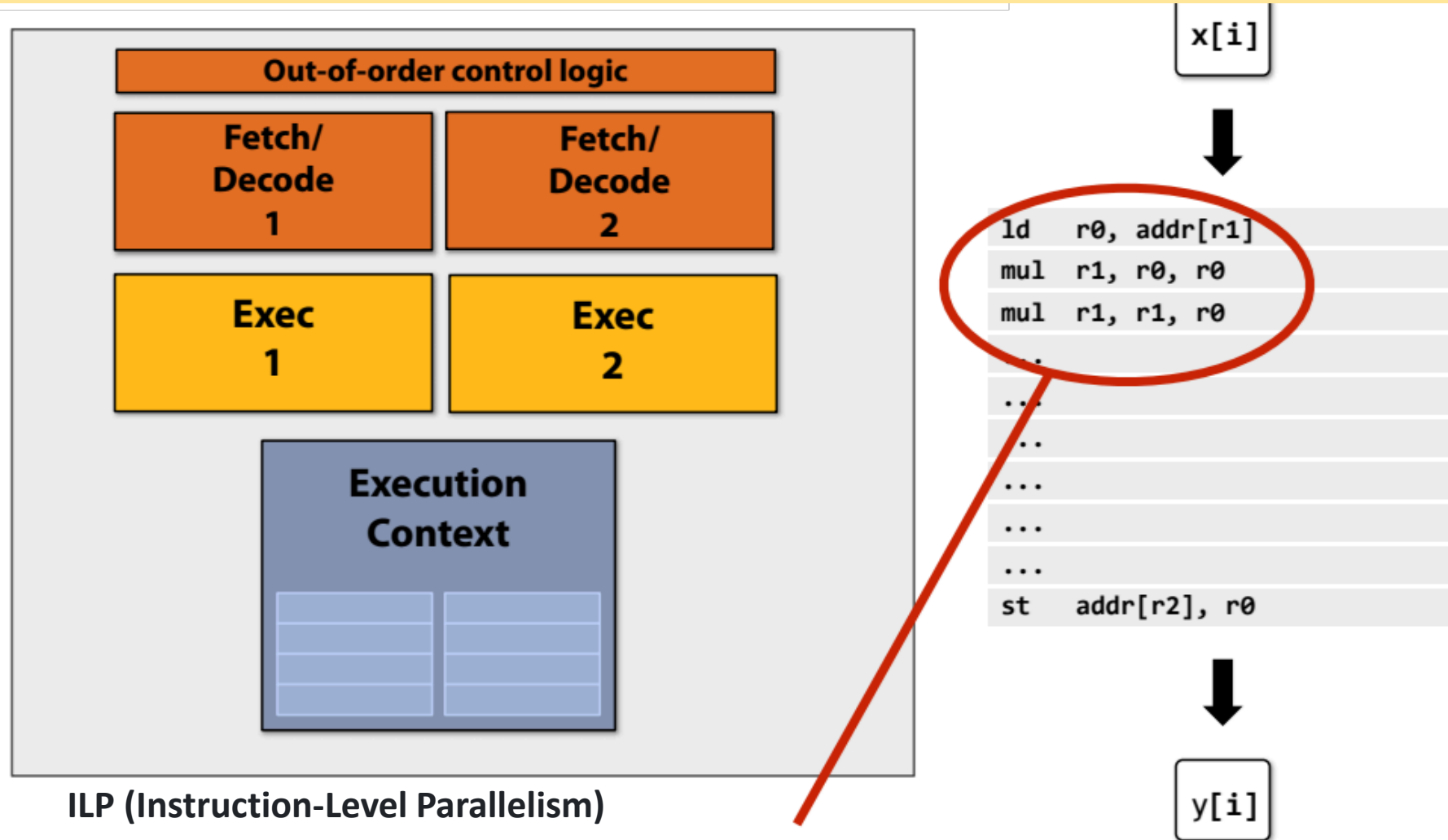
Execute program

아주 간단한 프로세서: 클럭당 하나의 명령어 실행
일반적으로 간단한 Processor는 매 Clock마다 하나의 Instruction을 실행한다.



Superscalar processor

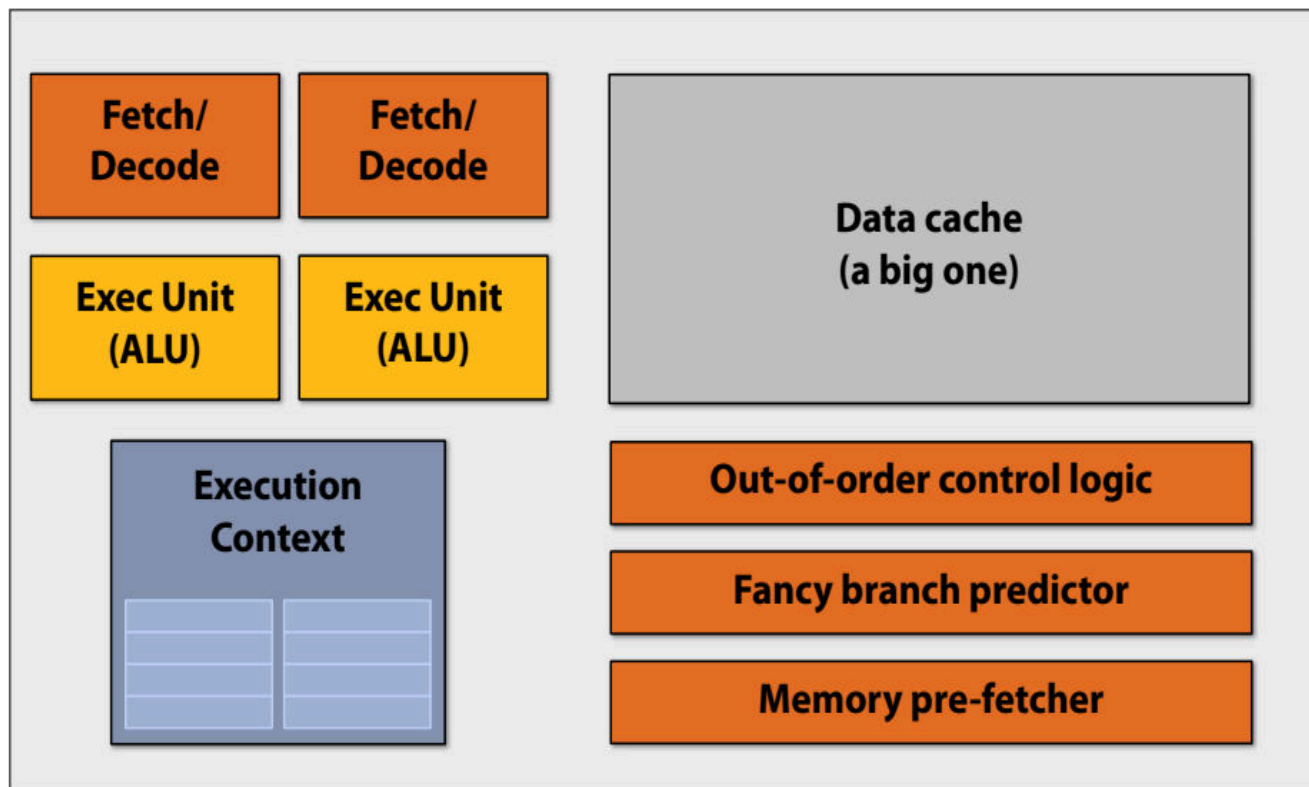
SuperScalar Processor는 Instruction-Level Parallelism(ILP)을 통해 매 Clock마다 두 개의 Instruction을 해독하고 실행이 가능하다. (명령어 스트림에 독립적인 명령어가 존재하는 경우).



Note: No ILP exists in this region of the program

- 초기의 Multi-Core에서는 하나의 Instruction Stream을 빠르게 수행하도록 돕는 연산을 수행하기 위해 많은 Chip Transistor를 사용하곤 했다. Transistor가 많을수록 Larger Cache, Smarter Out-Of-Order Logic, Smarter Branch Predictor 등 특징을 가졌다.

단일 명령어 스트림을 빠르게 실행하는 데 도움이 되는 연산을 수행하는 데 사용되는 대부분의 칩 트랜지스터



더 많은 트랜지스터 = 더 큰 캐시, 더 스마트한 비순차적 로직, 더 스마트한 분기 예측기 등.

1. 멀티코어 프로세서의 탄생 배경

- 초기 프로세서 설계는 단일 코어의 성능을 최대화하는 데 초점. 이는 클럭 속도 증가, 복잡한 명령어 재정렬(out-of-order execution), 브랜치 예측(branch prediction) 등의 기술을 활용.
- 그러나 클럭 속도 증가로 인한 전력 소비와 발열 문제로 인해 한계에 도달하게 되었음.

2. 멀티코어 설계의 원리

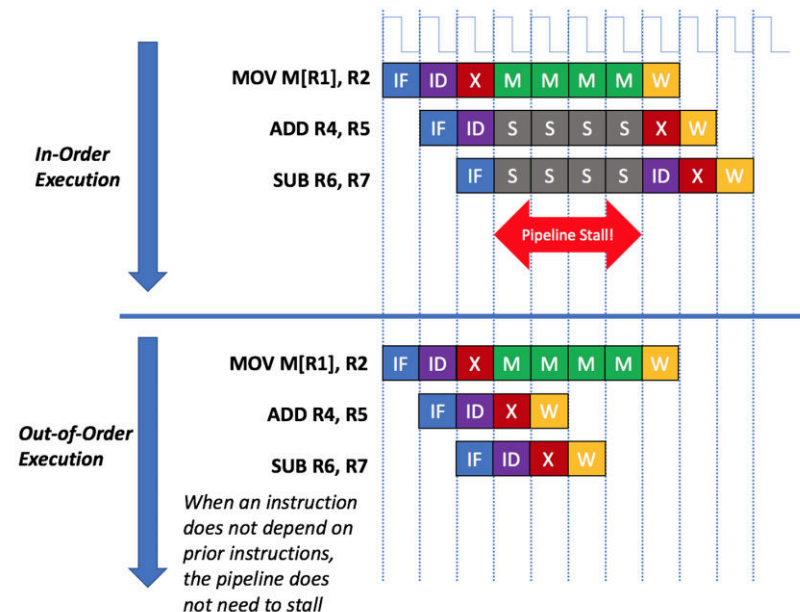
- 증가하는 트랜지스터 수를 활용하여 단일 코어의 복잡성을 높이는 대신, 여러 개의 코어를 동일한 칩에 추가함으로써 병렬 처리를 가능하게 했음.
- 각 코어가 독립적으로 명령어를 실행할 수 있도록 설계되었음.

아웃오브오더(Out-Of-Order) 처리란

- 프로그램에서 실행 결과가 변하지 않도록 고려하여 데이터 등의 의존 관계에서 처리 가능한 명령어에 대해 명령어 실행 및 완료 순서를 변경하여 병렬도를 높이고 성능을 향상시키는 기법.
 - 아웃오브오더 처리(out of order): 명령어를 동적으로 바꾸어 의존관계가 없는 순서대로 순차적으로 실행하는 방식.
 - 인오더 처리(in order): 명령어를 읽은 순서대로 실행하는 방식.

● 순서를 바꾸는 타이밍에 따라 두 가지가 있음.

1. 아웃오브오더 실행(out of order execution)
 - 명령어를 EX 스테이지에 넣는 순서를 바꾼다.
2. 아웃오브오더 완료(out of order completion)
 - 실행 결과를 레지스터에 저장하는 순서를 바꾼다.

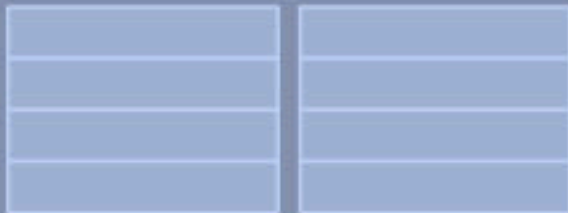


Multi-core era processor

**Fetch/
Decode**

**Exec Unit
(ALU)**

**Execution
Context**



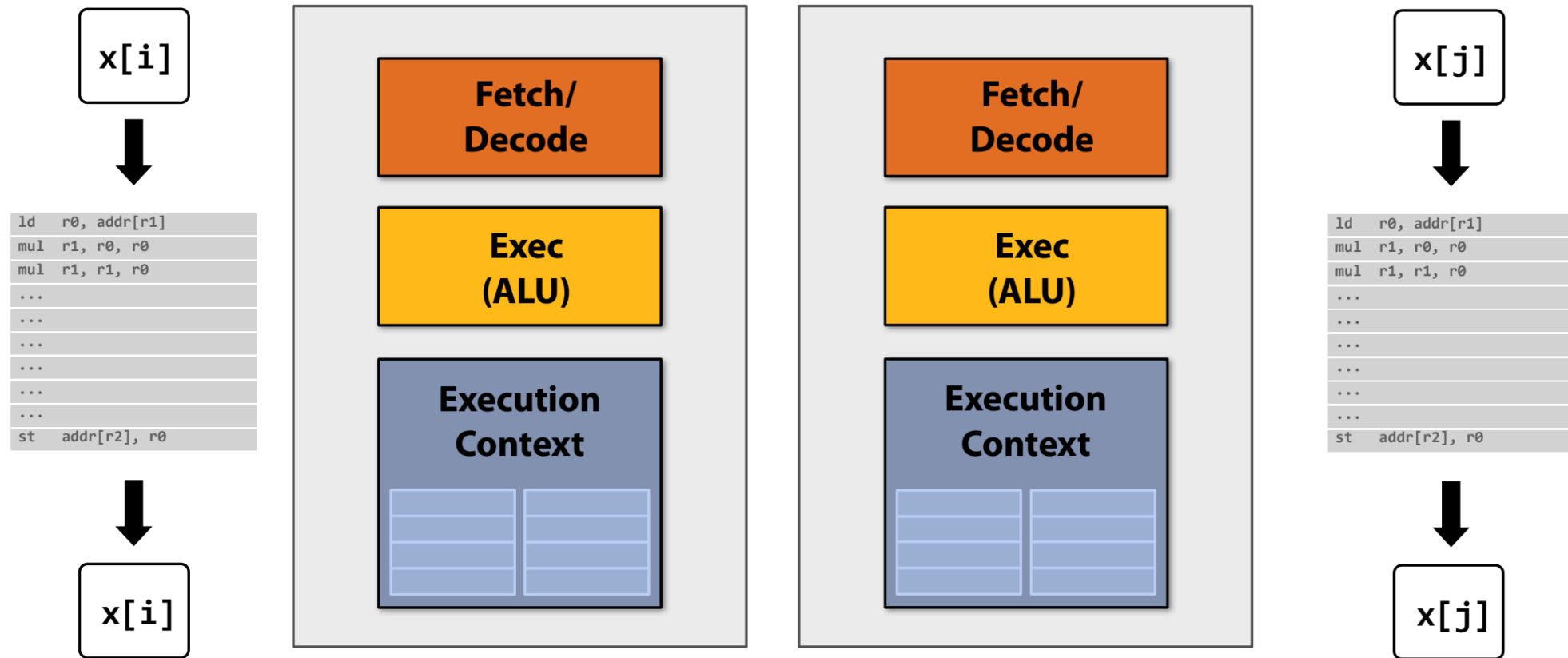
Idea #1:

트랜지스터를 사용하여 단일 명령어 스트림을 가속하는 프로세서 로직의 정교함을 높이는 대신.

(예: out-of-order and speculative operations(추측연산))
트랜지스터 수를 늘려 프로세서에 더 많은 코어를 추가.

- 트랜지스터 증가 활용: 트랜지스터 수가 증가함에 따라 이를 활용하여 더 많은 코어를 추가함으로써 병렬 처리 성능을 향상시킴.
- 단순화된 코어 설계: 각 코어는 단일 명령 스트림을 처리하는 데 있어 약간 느릴 수 있지만, 전체적으로 더 많은 작업을 병렬로 처리할 수 있음.
- 병렬 프로그래밍의 중요성: 병렬성을 표현하지 않는 프로그램은 멀티 코어의 이점을 충분히 활용할 수 없으므로, 병렬 프로그래밍 기법이 필수적

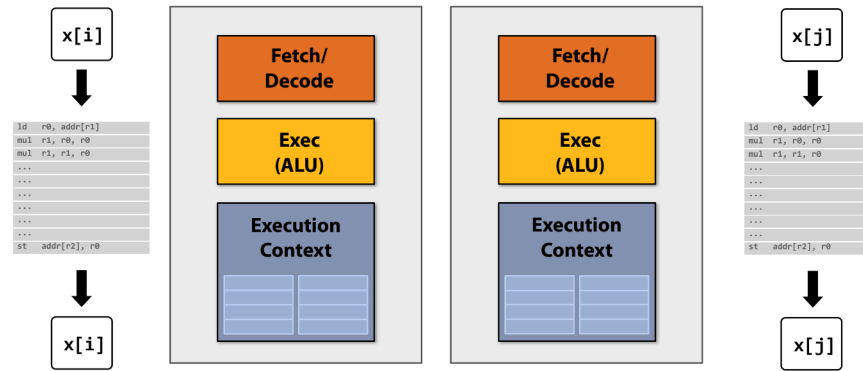
Two cores: compute two elements in parallel



더 단순한 코어: 각 코어는 원래의 “멋진” 코어보다 단일 명령어 스트림을 실행하는 속도가 느릴 수 있음(예: 25% 느림).

하지만 이제 코어가 두 개 되었습니다: $2 \times 0.75 = 1.5$ (속도 향상 가능성!).

Two cores: compute two elements in parallel



- 멀티코어 프로세서는 추가 코어를 활용하여 동시에 여러 작업을 병렬로 처리할 수 있음
 - 두 코어를 사용하는 시스템에서는 각각의 코어가 독립적으로 실행되며, 두 개의 작업을 동시에 병렬로 수행. 이는 프로세서의 전체적인 효율성을 높이며 단일 작업 처리에 의존하지 않음.

효율성 및 제한 사항

1. 단순한 코어 설계:

- ✓ 각 코어는 개별적으로 더 복잡한 작업을 처리하는 데 약간 느릴 수 있지만, 병렬 실행을 통해 전체 성능이 향상.
- ✓ 예를 들어, 단일 고급 코어에 비해 각 코어는 약 25% 느리게 작동할 수 있지만 두 코어를 함께 사용하면 최대 1.5배의 속도 향상을 얻을 가능성이 있음.

2. 프로그램 병렬화 필요성:

- ✓ 프로세스가 병렬 실행을 지원하지 않으면 멀티코어 프로세서의 잠재적 이점을 충분히 활용할 수 없음
- ✓ 따라서 프로그래머는 병렬성을 효과적으로 표현하는 기술이 필요.

But our program expresses no parallelism

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

이 C 프로그램은 하나의 프로세서 코어에서 하나의 스레드로 실행되는 명령어 스트림으로 컴파일됩니다.

단순한 프로세서 코어 각각이 원래의 복잡한 단일 프로세서 코어보다 25% 느렸다면, 이제 우리 프로그램은 이전보다 25% 느리게 실행됩니다.



프로그램이 멀티코어 프로세서의 병렬 처리 잠재력을 활용하지 못한다는 것 ➔ 현재 대부분의 멀티코어로 되어있음에도 불구하고 제대로 사용하고 있지 못함

Example: expressing parallelism using C++ threads

```
typedef struct {
    int N;
    int terms;
    float* x;
    float* y;
} my_args;

void my_thread_func(my_args* args)
{
    sinx(args->N, args->terms, args->x, args->y); // do work
}
```

```
void parallel_sinx(int N, int terms, float* x, float* y)
{
    std::thread my_thread;
    my_args args;

    args.N = N/2;
    args.terms = terms;
    args.x = x;
    args.y = y;

    my_thread = std::thread(my_thread_func, &args); // launch thread
    sinx(N - args.N, terms, x + args.N, y + args.N); // do work on main thread
    my_thread.join(); // wait for thread to complete
}
```

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * numer / denom
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

Data-parallel expression

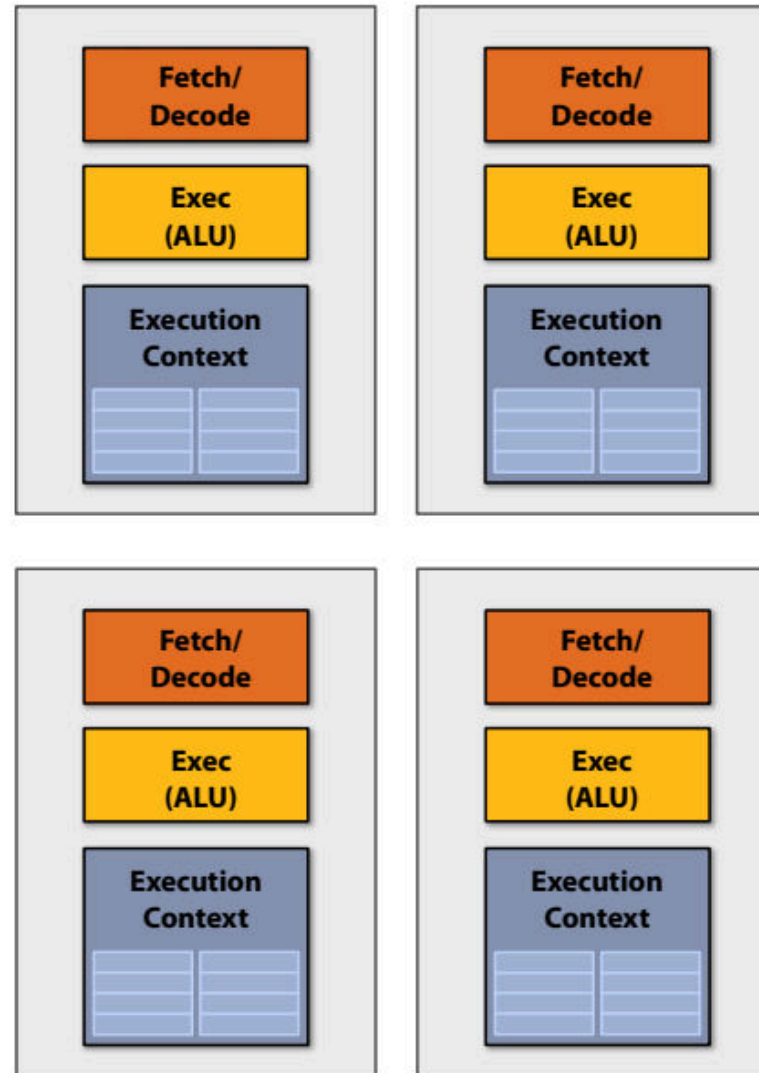
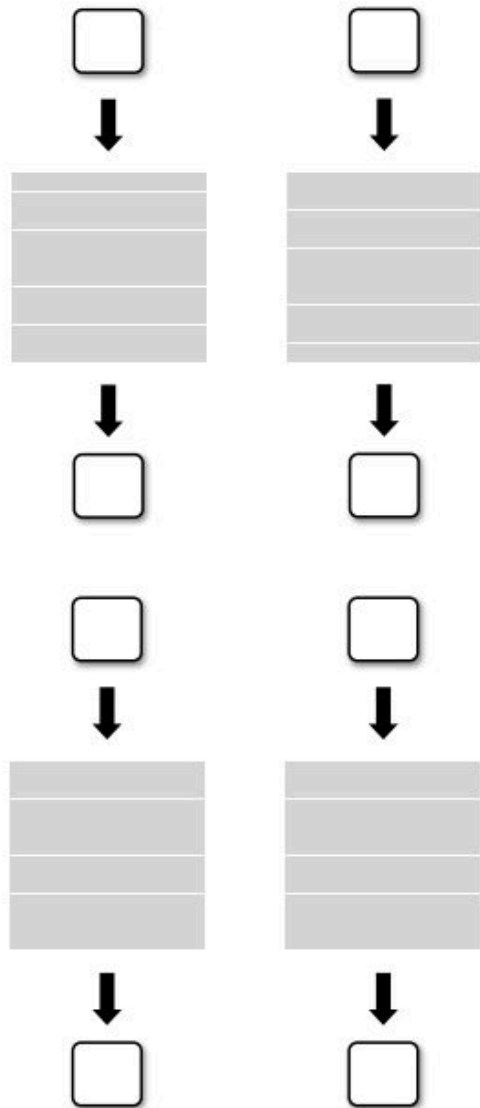
```
void sinx(int N, int terms, float* x, float* y)
{
    // declares that loop iterations are independent
    forall (int i from 0 to N)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

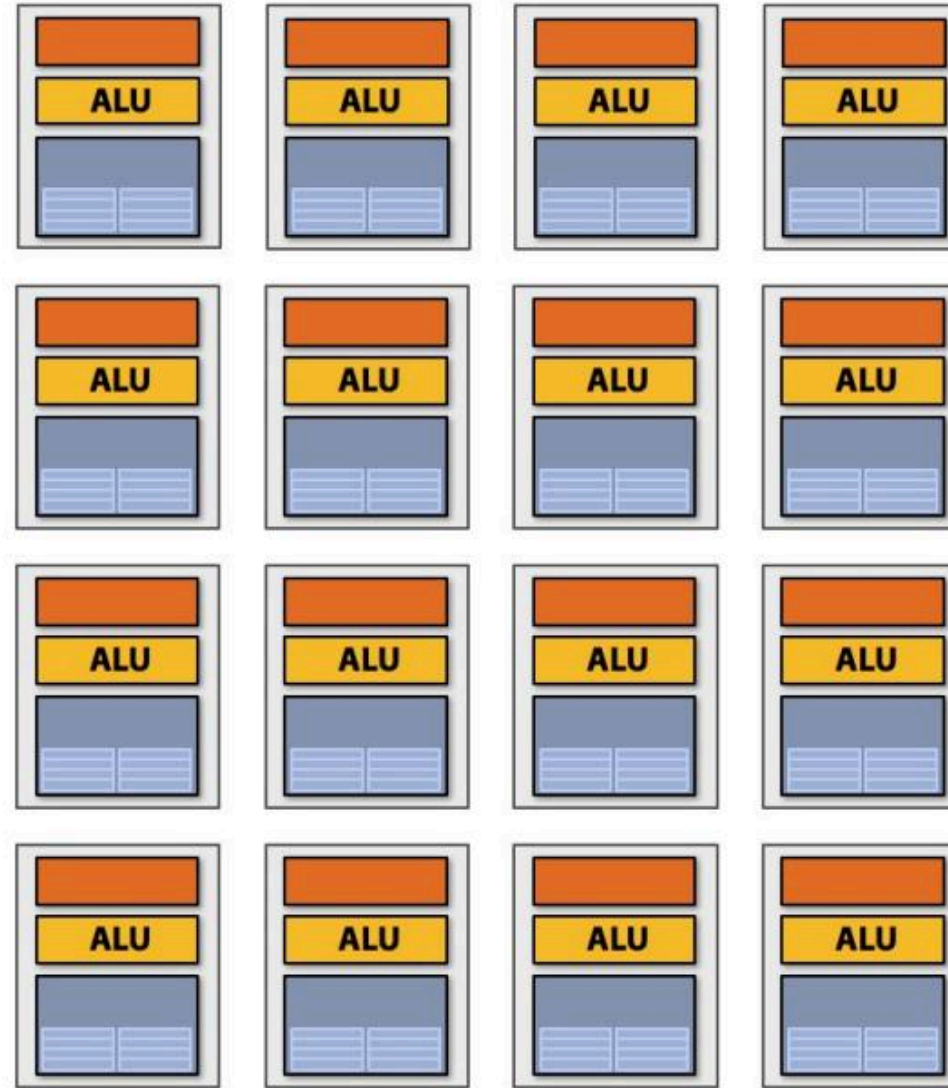
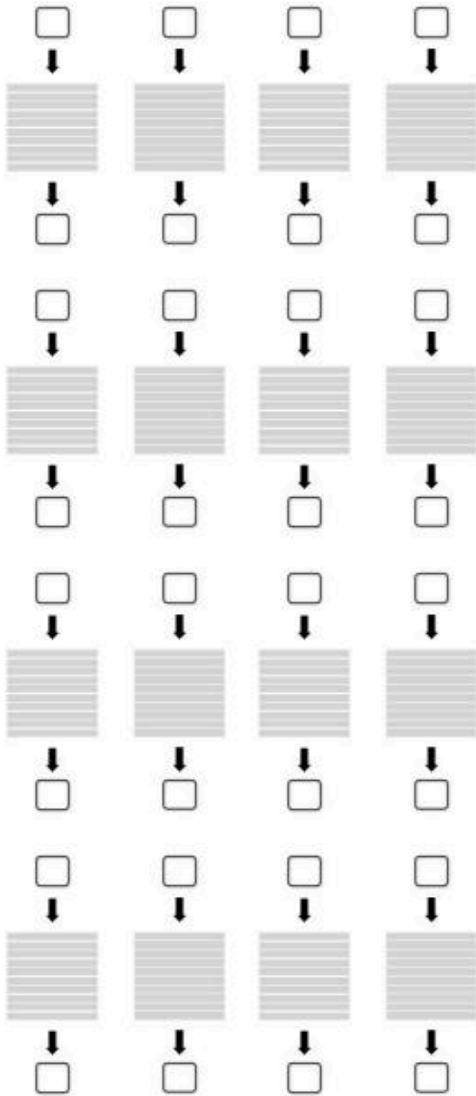
        y[i] = value;
    }
}
```

이 코드에서 프로그래머는 루프 반복을 독립적으로 선언합니다('forall' 참조). 이 정보를 통해 컴파일러가 스레드 코드를 자동으로 생성하는 방법을 상상해볼 수 있습니다.

Four cores: compute four elements in parallel



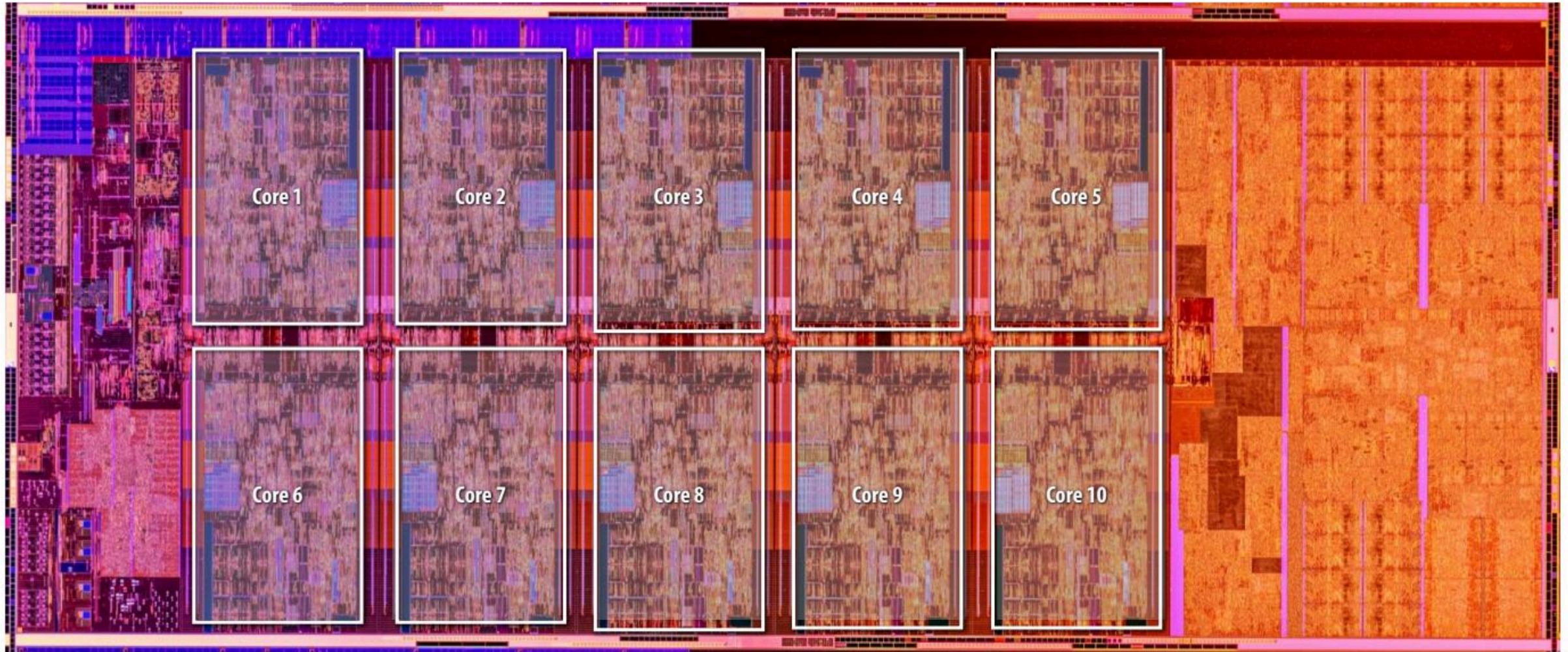
Sixteen cores: compute sixteen elements in parallel



Sixteen cores, sixteen simultaneous instruction streams

Example: multi-core CPU

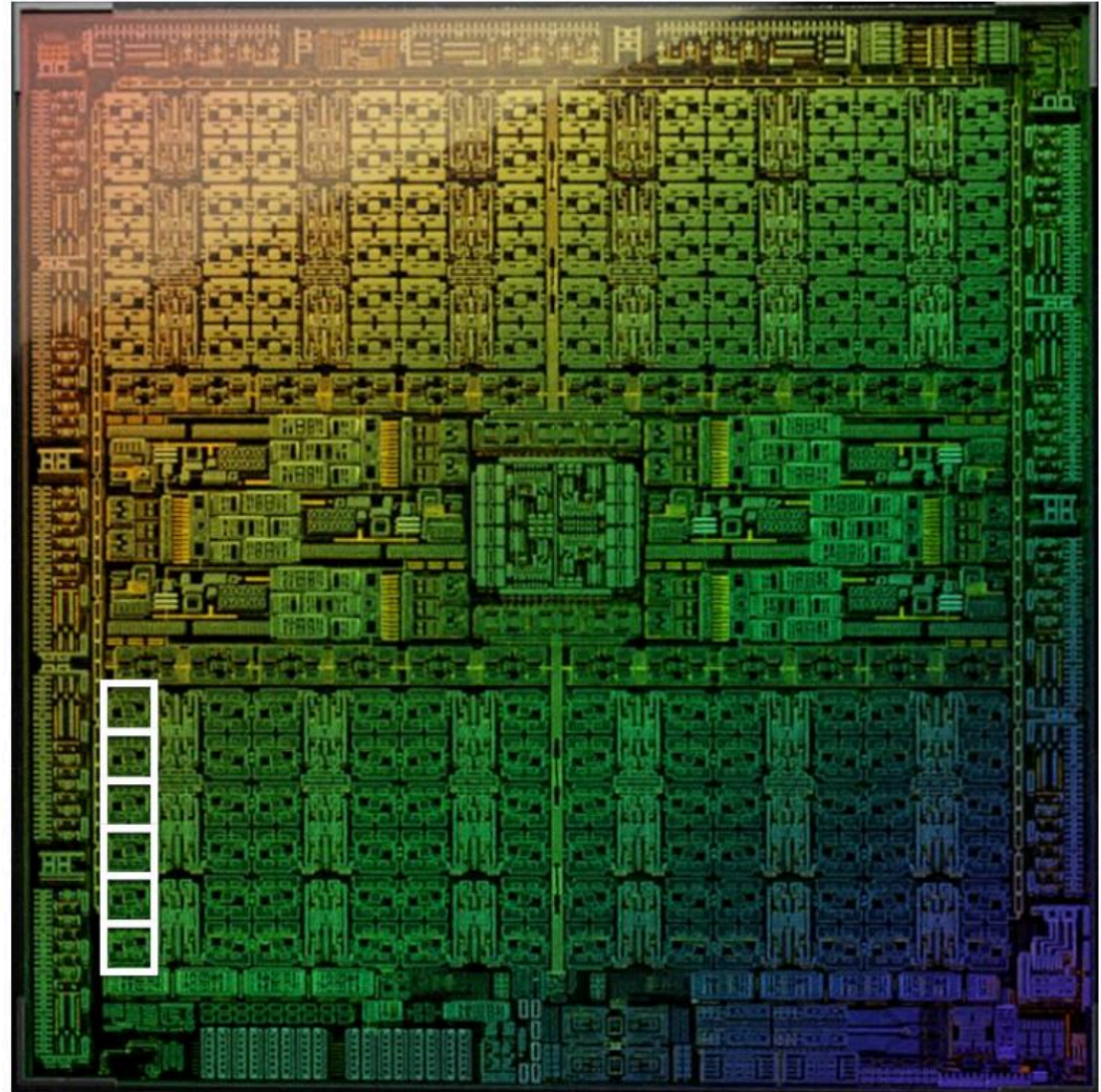
Intel "Comet Lake" 10th Generation Core i9 10-core CPU (2020)



Multi-core GPU

GeForce RTX 4090 (2022)

144 processing blocks (called SMs)



Apple A15 Bionic

Two “big cores” + four “small” cores

2 “big” CPU cores

4 “small” CPU cores



Image Credit: TechInsights Inc.

Apple M1 Silicon (also heterogenous cores)

Four “big cores” + four “small” CPU cores *

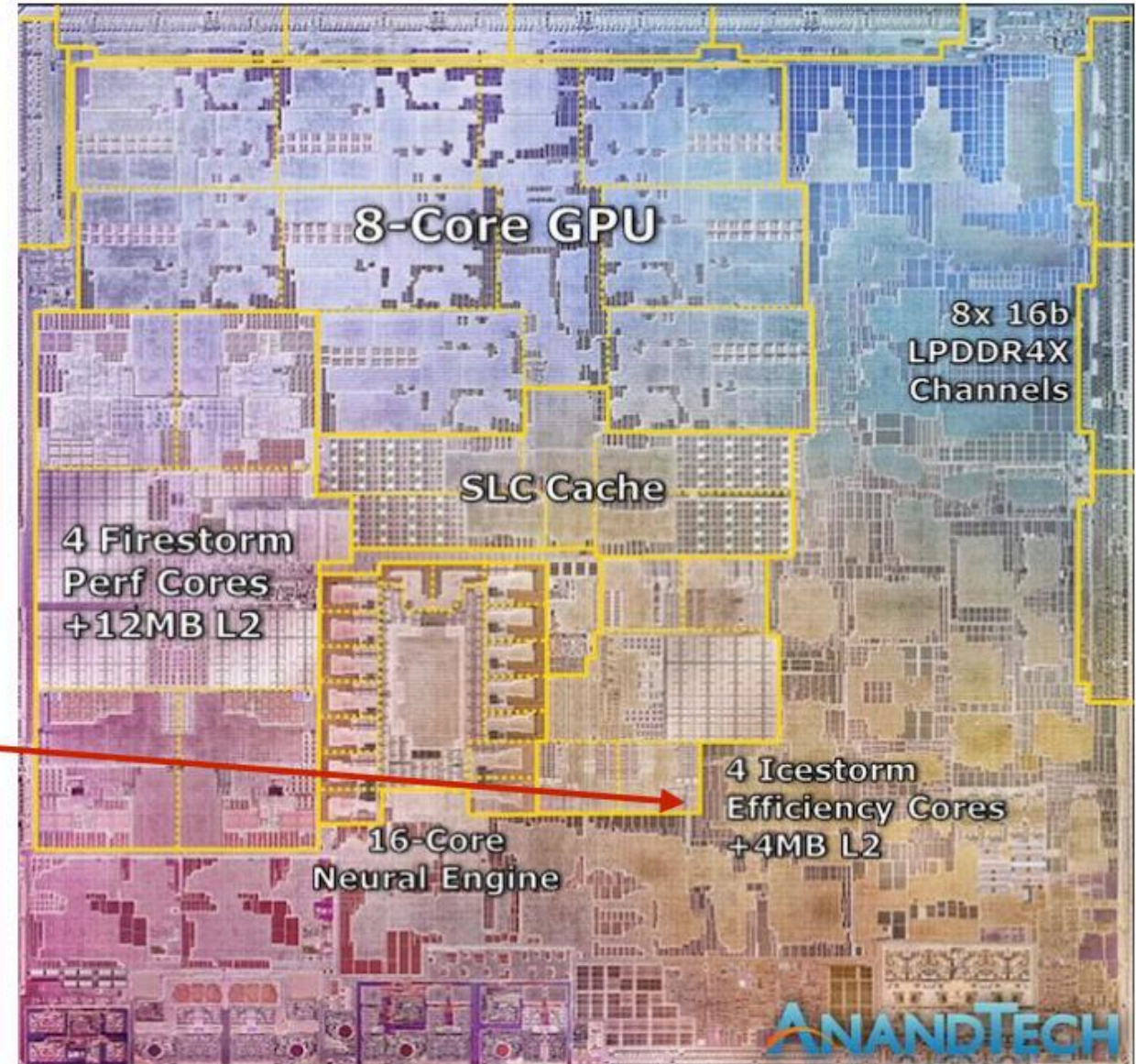
4 “big” cores



4 “small” CPU cores



* not even counting the GPU cores or the neural acceleration hardware



Data-parallel expression

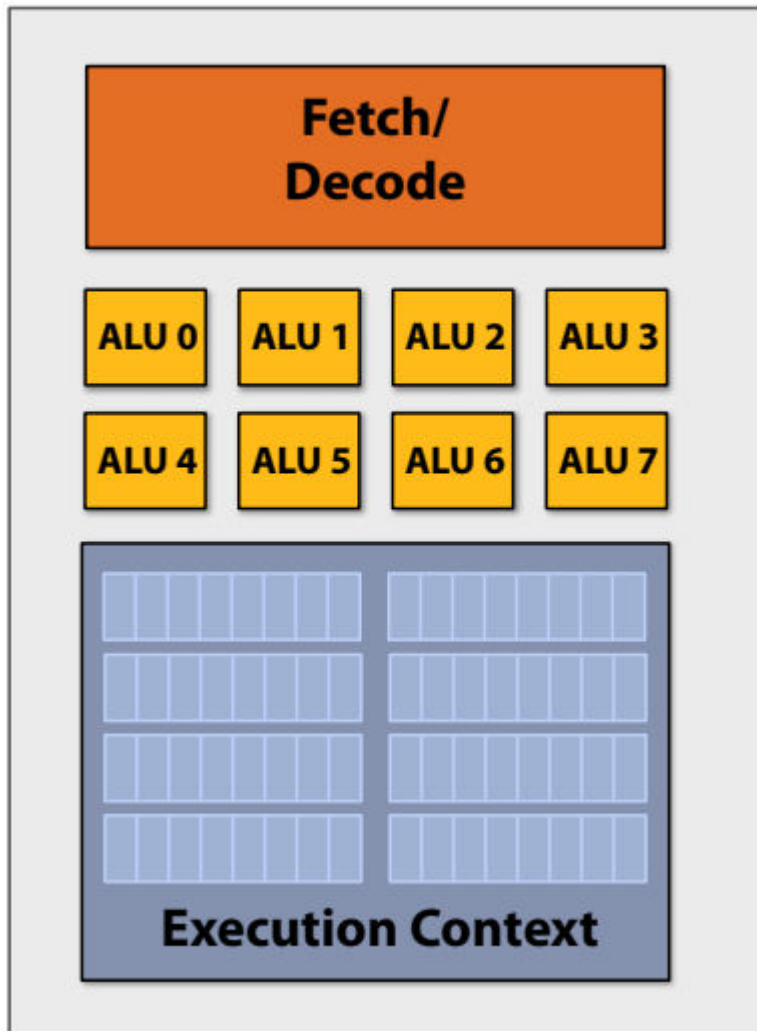
```
void sinx(int N, int terms, float* x, float* result)
{
    // declares that loop iterations are independent
    forall (int i from 0 to N)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        result[i] = value;
    }
}
```

- 이 코드의 또 다른 흥미로운 속성은 루프의 반복에 걸쳐 병렬성이 존재한다는 점입니다.
- 루프의 모든 반복은 정확히 동일한 명령어 시퀀스(루프 본문에 의해 정의됨)를 수행하지만, $x[i]$ (루프 본문에 의해 $\sin(x[i])$ 계산)로 주어진 서로 다른 입력 데이터에 대해 수행됩니다.
- 독립적 루프 반복: 반복문 내의 작업이 서로 독립적. 이는 모든 반복(iteration)이 다른 데이터 항목에 대해 동일한 작업을 수행하므로 병렬 실행이 가능하다는 것을 의미.
 - ✓ forall 키워드를 사용하여 반복문이 독립적임을 명시.
 - ✓ 독립성을 기반으로 컴파일러는 병렬 코드를 자동으로 생성할 수 있음.
- 병렬 처리의 이점: 데이터 병렬성은 동일한 작업을 대규모 데이터 집합에 대해 병렬로 수행가능. 이는 루프의 각 반복이 서로 다른 입력 데이터($x[i]$)에 대해 동일한 계산(예: sine 계산)을 실행할 때 특히 효과적.

Add execution units (ALUs) to increase compute capability



- 아이디어 #2: 여러 ALU에서 명령어 스트림을 관리하는 데 드는 비용/복잡성 완화

- **SIMD processing**

Single instruction, multiple data

- 모든 ALU에 동일한 명령어 브로드캐스팅
- 이 연산은 모든 ALU에서 병렬로 실행.



이 방식은 대량의 데이터셋에서 동일한 연산을 반복적으로 수행해야 하는 작업(예: 배열 연산, 그래픽 처리 등)에 매우 효과적

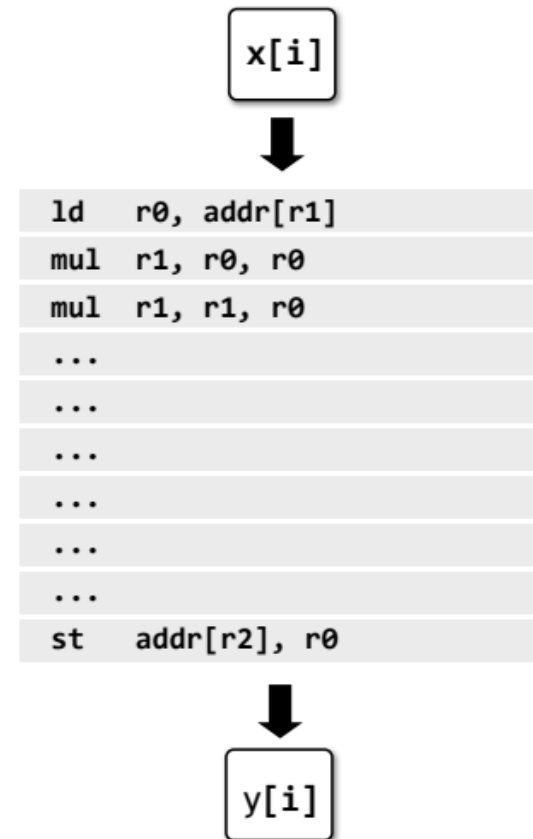
Recall our original scalar program

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

- 원래 컴파일된 프로그램:
- 스칼라 레지스터의 스칼라 명령어(예: 32비트 “float”)를 사용하여 하나의 배열 요소를 처리합니다.



Vector program (using AVX intrinsics)

```
#include <immintrin.h>

void sinx(int N, int terms, float* x, float* y)
{
    float three_fact = 6; // 3!
    for (int i=0; i<N; i+=8)
    {
        __m256 origx = _mm256_load_ps(&x[i]);
        __m256 value = origx;
        __m256 numer = _mm256_mul_ps(origx, _mm256_mul_ps(origx, origx));
        __m256 denom = _mm256_broadcast_ss(&three_fact);
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            // value += sign * numer / denom
            __m256 tmp = _mm256_div_ps(_mm256_mul_ps(_mm256_set1ps(sign), numer), denom);
            value = _mm256_add_ps(value, tmp);

            numer = _mm256_mul_ps(numer, _mm256_mul_ps(origx, origx));
            denom = _mm256_mul_ps(denom, _mm256_broadcast_ss(((2*j+2) * (2*j+3))));
            sign *= -1;
        }
        _mm256_store_ps(&y[i], value);
    }
}
```

- C 프로그래머가 사용할 수 있는 고유 데이터 유형 및 함수
- 내재 함수는 8개의 32비트 값으로 구성된 벡터(예: 8개의 부동 소수점 벡터)에서 작동합니다.

- 벡터 명령어: 여러 데이터 요소를 동시에 처리할 수 있는 병렬 처리를 지원. AVX는 256비트 레지스터를 사용하여 8개의 32비트 부동 소수점 데이터를 병렬로 처리 가능
- 데이터 병렬성 활용: 프로그램이 독립적인 데이터 집합(예: 배열)을 반복적으로 처리할 때, 데이터 병렬성을 활용하여 성능을 극대화. 이는 특히 반복 작업이 많은 과학 계산 및 그래픽 처리에 유용.

- `_mm256_load_ps`: 입력 배열 x에서 8개의 데이터를 로드.
- `_mm256_mul_ps` 및 `_mm256_add_ps`: 벡터 요소 간 병렬 곱셈 및 덧셈 수행.
- `_mm256_store_ps`: 계산된 결과를 출력 배열 y에 병렬로 저장합니다.

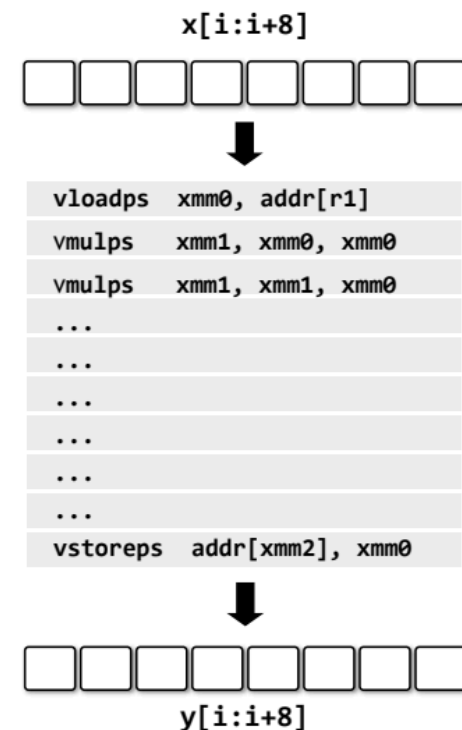
Vector program (using AVX intrinsics)

```
#include <immintrin.h>

void sinx(int N, int terms, float* x, float* y)
{
    float three_fact = 6; // 3!
    for (int i=0; i<N; i+=8)
    {
        __m256 origx = _mm256_load_ps(&x[i]);
        __m256 value = origx;
        __m256 numer = _mm256_mul_ps(origx, _mm256_mul_ps(origx, origx));
        __m256 denom = _mm256_broadcast_ss(&three_fact);
        int sign = -1;

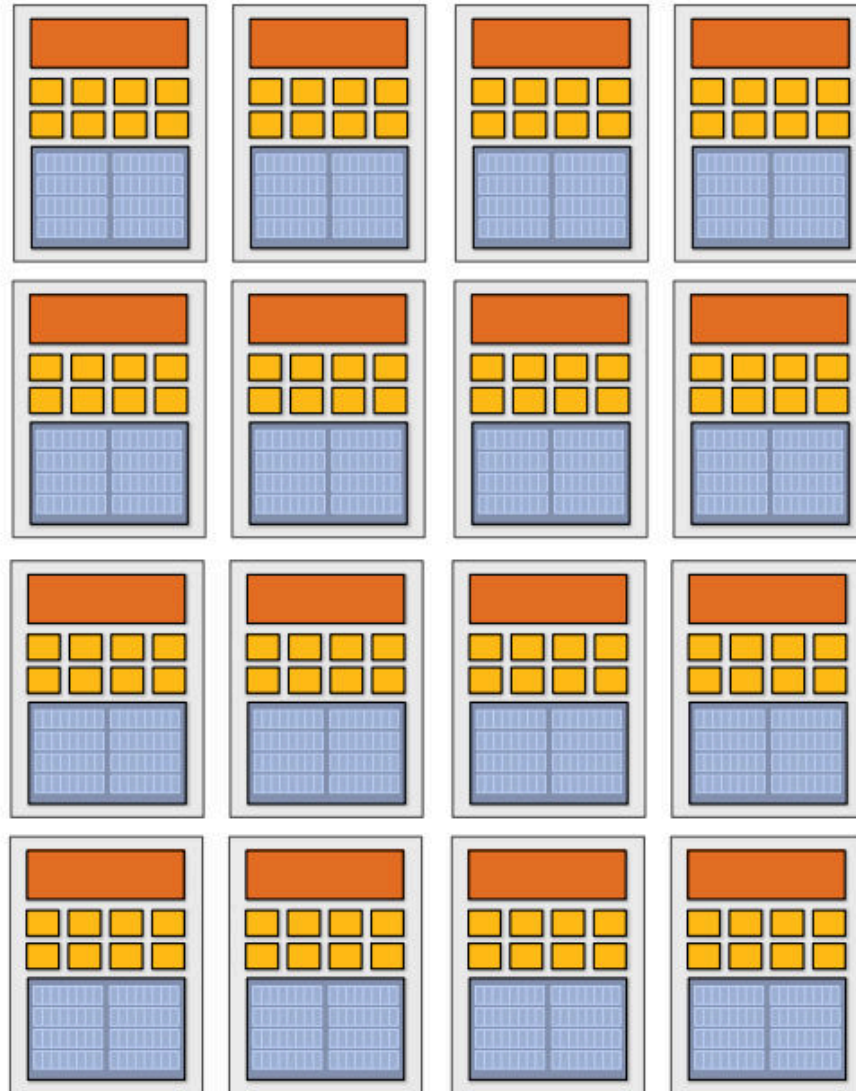
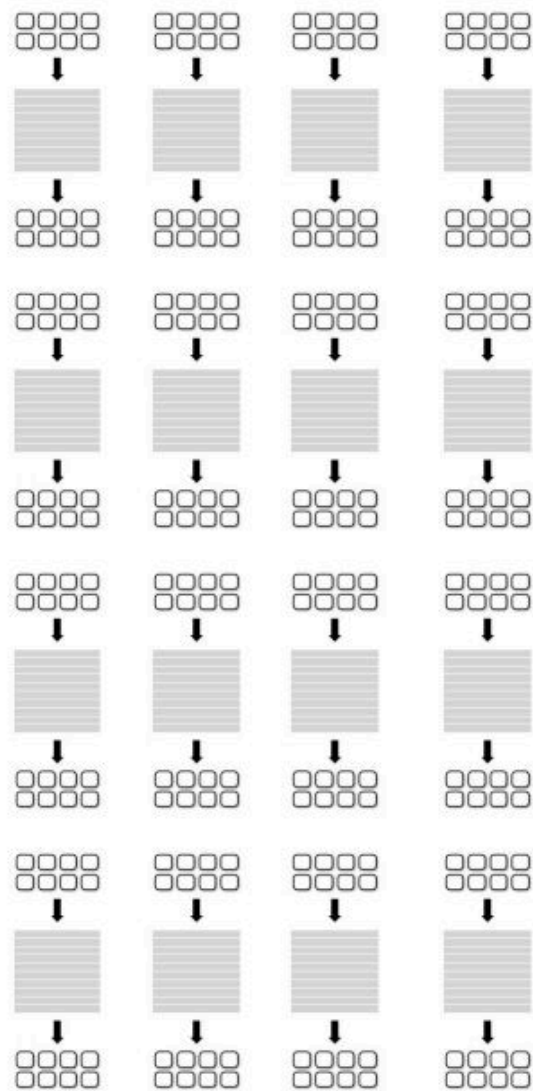
        for (int j=1; j<=terms; j++)
        {
            // value += sign * numer / denom
            __m256 tmp = _mm256_div_ps(_mm256_mul_ps(_mm256_set1ps(sign), numer), denom);
            value = _mm256_add_ps(value, tmp);

            numer = _mm256_mul_ps(numer, _mm256_mul_ps(origx, origx));
            denom = _mm256_mul_ps(denom, _mm256_broadcast_ss((2*j+2) * (2*j+3)));
            sign *= -1;
        }
        _mm256_store_ps(&y[i], value);
    }
}
```



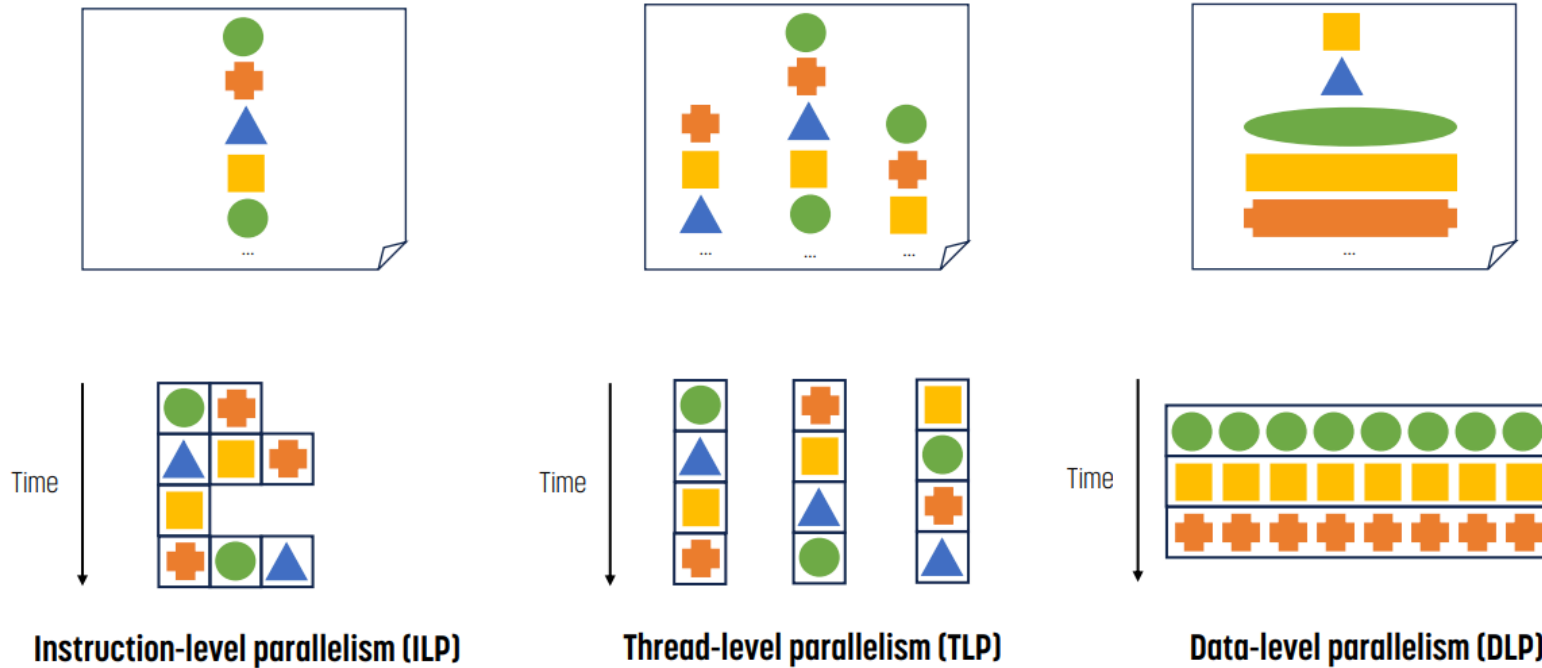
- 컴파일된 프로그램:
256비트 벡터 레지스터에서 벡터 명령어를 사용하여 8개의 배열 요소를 동시에 처리합니다.

16 SIMD cores: 128 elements in parallel



16 cores, 128 ALUs, 16 simultaneous instruction streams

Parallelism의 종류



위쪽 부분은 programmer view를, 아래쪽 부분은 실제 작동 방식을 나타낸다.

- **ILP**의 경우 independent한 instruction을 적당히 동시에 실행시킨다. hardware가 수행하기에 programmer는 이에 대해 알 수 없다.
- **TLP**의 경우 programmer가 instruction을 나누어 배치했기 때문에 해당 instruction들끼리는 parallel하게 실행되는 모습이다. programmer가 instruction을 나누었기 때문에 이를 인지하고 있다.
- **DLP**의 경우 같은 종류의 data는 같은 instruction이 실행되는 모습이다. TLP의 variation인 만큼 programmer도 이를 인지하고 있다.

Parallelism의 종류

● ILP, Instruction Level Parallelism

- instruction들끼리 **independent**하다.
- hardware는 instruction window size 내에서 implicit하게 찾을 수 있다.
- compiler들은 window 내에 있는 명령어들이 independent할 수 있도록 찾아서 순서를 바꾼다. **independent**한 순서로 재배열 한 후 실행하는 방식이다.

● TLP, Thread Level Parallelism

- compiler와 programmer가 program을 명시적으로 나눈다.

● DLP, Data Level Parallelism

- TLP의 variation 중 하나로, 같은 instruction을 실행하는 여러 개의 thread가 다른 data에 대해 동작하는 방식이다.

Hardware가 parallelism을 달성하는 방법

- ILP

- Superscalar : single thread에서 여러 개의 instruction을 실행하는 방식이다. instruction dependency가 없는 instruction만 하나의 cycle에 동시에 실행되기 때문에 dependency에 크게 영향을 받는다.

- TLP

- Find Grain Multithreading : thread 간 context switching이 빠르다. 매 cycle마다 다른 thread를 교체해서 실행하는 방식이다. (single core)
- SMT, Simultaneous MultiThreading : 같은 cycle에 여러 개의 thread의 instruction을 실행하는 방식이다. (multi core)
- CMP, Chip Multi Processors : 하나의 chip에 여러 개의 core를 탑재하고 각 thread를 다른 core에서 실행하는 방식이다.

- DLP

- Vector Processors : data의 여러 부분에 작동하는 single instruction을 실행하는 방식이다.

blog.hyelie.com/category/CS/Parallel%20Computing


```
void sinx(int N, int terms, float* x, float* result)
{
    // declares that loop iterations are independent
    forall (int i from 0 to N)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        result[i] = value;
    }
}
```

- 프로그램에서 “forall”을 사용하면 루프 반복이 독립적이며 동일한 루프 본문이 많은 수의 데이터 요소에서 실행될 것임을 컴파일러에 선언합니다.
- 이러한 추상화를 통해 멀티코어 병렬 코드와 벡터 명령어를 자동으로 생성하여 코어 내에서 SIMD 처리 기능을 활용할 수 있습니다.

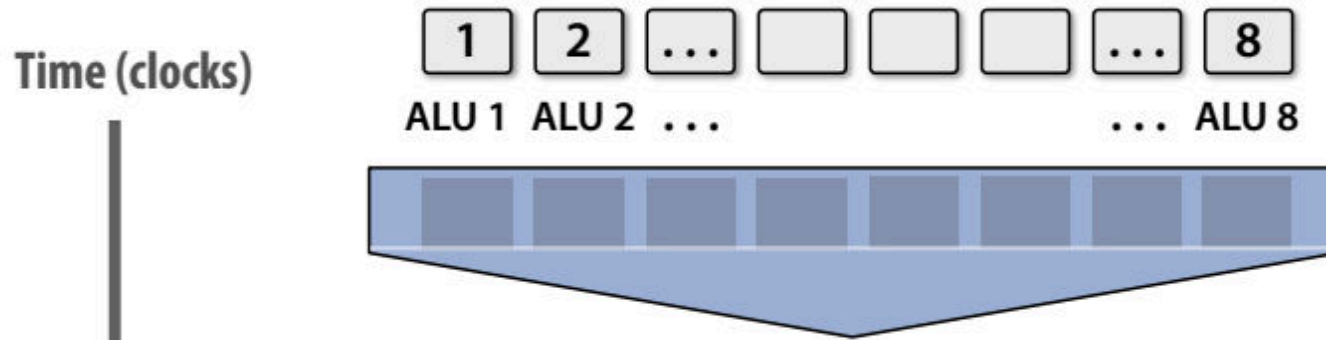
```
void sinx(int N, int terms, float* x, float* result)
{
    // declares that loop iterations are independent
    forall (int i from 0 to N)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        result[i] = value;
    }
}
```

- 프로그램에서 “forall”을 사용하면 루프 반복이 독립적이며 동일한 루프 본문이 많은 수의 데이터 요소에서 실행될 것임을 컴파일러에 선언합니다.
- 이러한 추상화를 통해 멀티코어 병렬 코드와 벡터 명령어를 자동으로 생성하여 코어 내에서 SIMD 처리 기능을 활용할 수 있습니다.

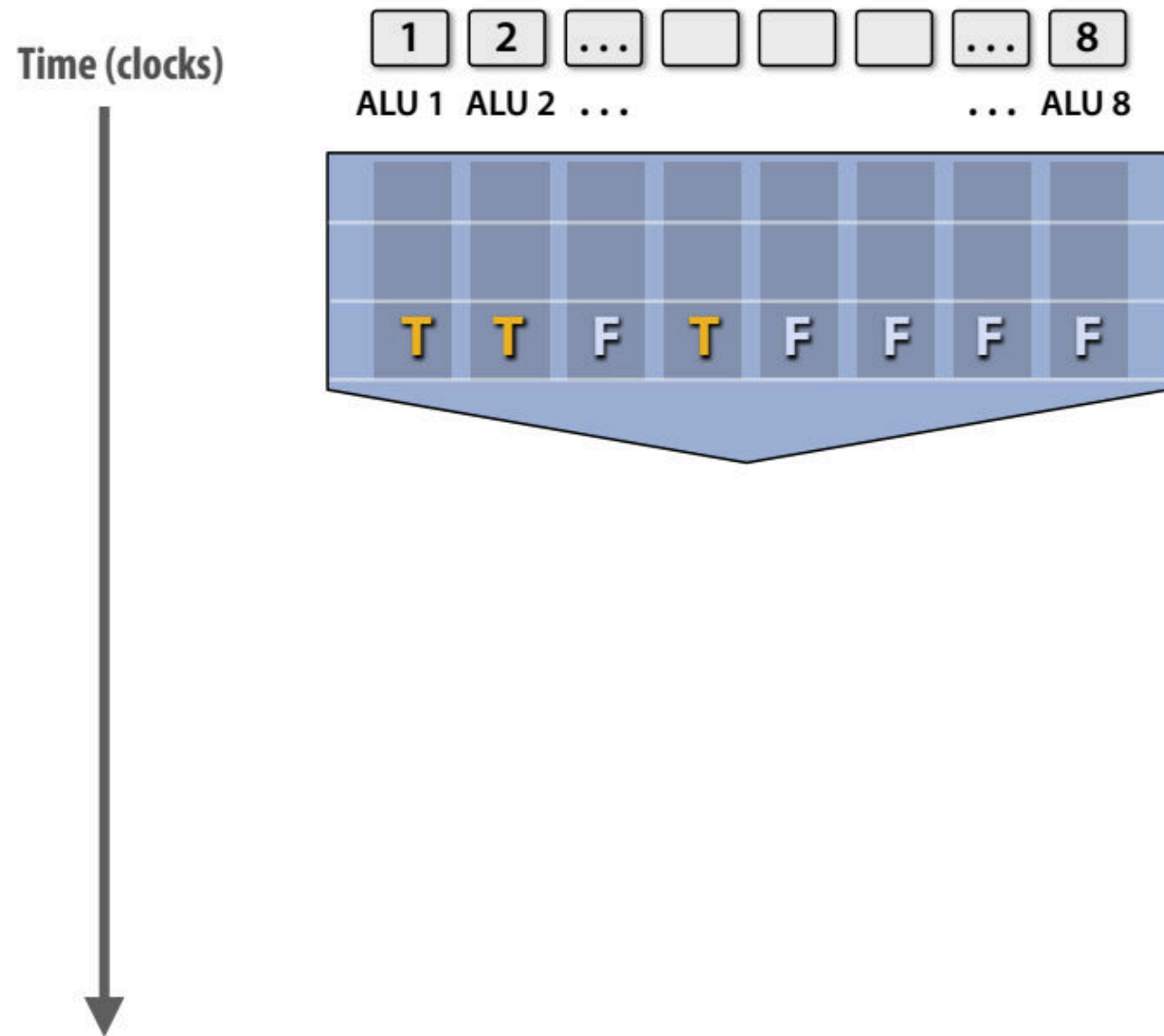
조건부 실행은 어떻게 되나요?(조건부 실행에서의 병렬성 문제)



- 조건부 분기와 데이터 병렬성:
 - 조건부 실행은 병렬 컴퓨팅에서 데이터를 다루는 각 ALU(Arithmetic Logic Unit)가 동일한 작업을 수행하지 않게 만들 수 있음
 - 예를 들어, 조건문 if-else는 각 데이터 요소에 대해서 다른 경로를 선택하게 하므로, 병렬 실행이 비효율적으로 이루어질 수 있음.
- ALU의 활용 저하:
 - SIMD(Single Instruction, Multiple Data)와 같은 병렬 처리에서는 ALU들이 동일한 명령어를 실행하지만, 조건부 분기가 발생하면 일부 ALU가 일을 멈추거나 불필요한 일을 수행하게 될 수 있음.
 - 최악의 경우, ALU의 활용도가 크게 떨어져 최고 성능의 일부만 발휘될 가능성이 있습니다.

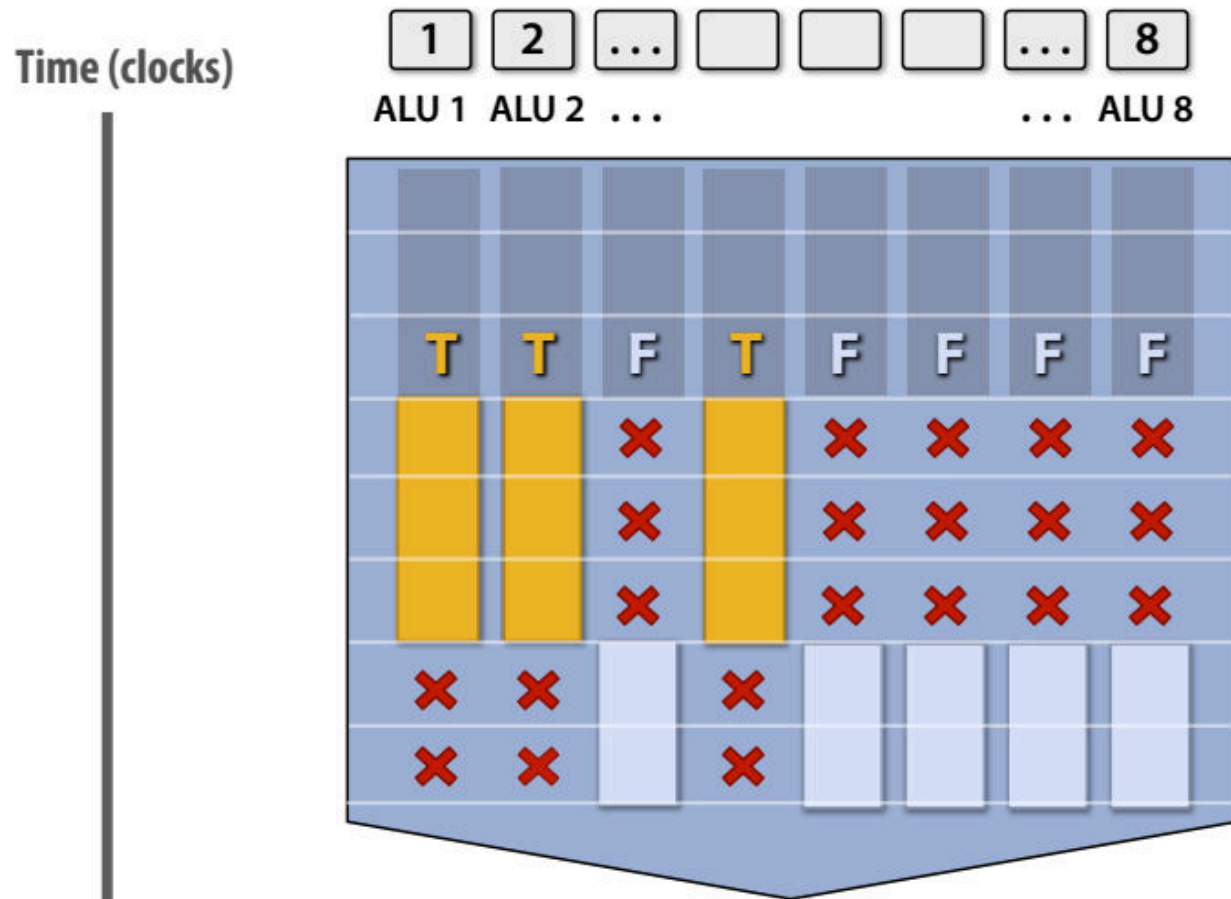
```
forall (int i from 0 to N) {  
    float t = x[i];  
    <unconditional code>  
  
    if (t > 0.0) {  
        t = t * t;  
        t = t * 50.0;  
        t = t + 100.0;  
    } else {  
        t = t + 30.0;  
        t = t / 10.0;  
    }  
  
    <resume unconditional code>  
    y[i] = t;  
}
```

조건부 실행은 어떻게 되나요?



```
forall (int i from 0 to N) {  
    float t = x[i];  
    <unconditional code>  
  
    if (t > 0.0) {  
        t = t * t;  
        t = t * 50.0;  
        t = t + 100.0;  
    } else {  
        t = t + 30.0;  
        t = t / 10.0;  
    }  
  
    <resume unconditional code>  
    y[i] = t;  
}
```

ALU의 마스크(무시, 즉 폐기) 출력



모든 ALU가 유용한 작업을 수행하는 것은 아닙니다!

Worst case: 1/8 peak performance

```
forall (int i from 0 to N) {  
    float t = x[i];  
    <unconditional code>  
  
    if (t > 0.0) {  
        t = t * t;  
        t = t * 50.0;  
        t = t + 100.0;  
    } else {  
        t = t + 30.0;  
        t = t / 10.0;  
    }  
  
    <resume unconditional code>  
    y[i] = t;  
}
```

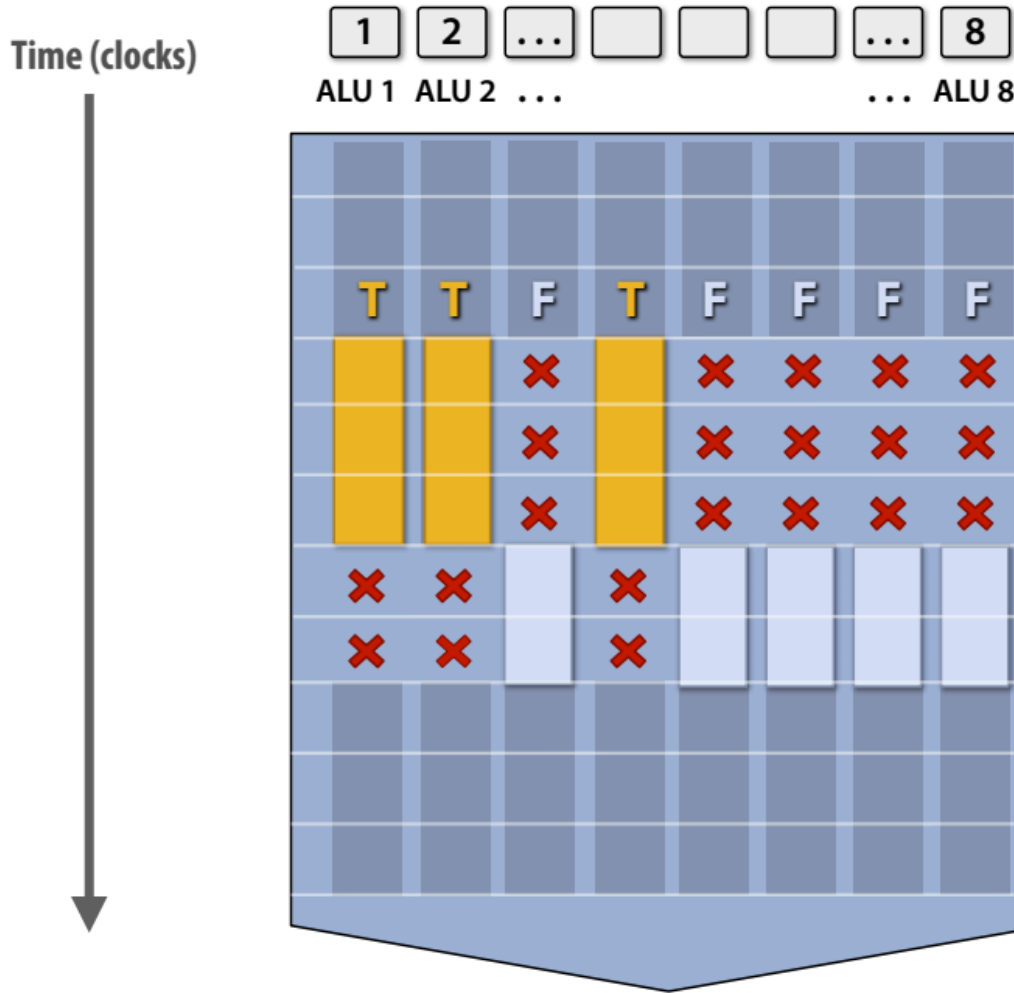
- **ALU의 유향 상태:**

- SIMD 명령어는 동일한 작업을 여러 데이터 요소에 동시에 적용해야 하지만, 조건부 코드가 포함되면 데이터에 따라 다른 분기가 발생
- 특정 ALU는 작업을 수행하지 않고 "유향 상태"로 남게 되며, 이는 병렬 처리 효율성을 감소

- **최악의 성능 사례:**

- 조건부 실행으로 인해 데이터의 일부만 처리되고 나머지는 무시(discard)되면, ALU가 전체적으로 비효율적으로 동작
- 예를 들어, 8개의 ALU를 가진 SIMD 프로세서에서 최악의 경우 1/8의 성능으로 작업을 수행할 수 있음.

분기 후: 최대 성능으로 계속 진행



```
forall (int i from 0 to N) {  
    float t = x[i];  
    <unconditional code>  
  
    if (t > 0.0) {  
        t = t * t;  
        t = t * 50.0;  
        t = t + 100.0;  
    } else {  
        t = t + 30.0;  
        t = t / 10.0;  
    }  
  
    <resume unconditional code>  
    y[i] = t;  
}
```

- 만약 코드 내에 조건문이 존재한다면 어떻게 될까? 결과적으로 일부 ALU는 조건문에 의해서 제한되기 때문에 모든 ALU의 성능을 사용하지 못한다. 하지만 분기 부분을 지나게 되면 정상적으로 모든 ALU의 성능을 사용할 수 있다.

Breakout question

8폭 SIMD 실행이 가능한 프로세서에서 최악의 성능을 내는 코드 조각을 생각해 볼 수 있나요?

힌트: 하나의 "if" 문만 사용하여 만들 수 있나요?

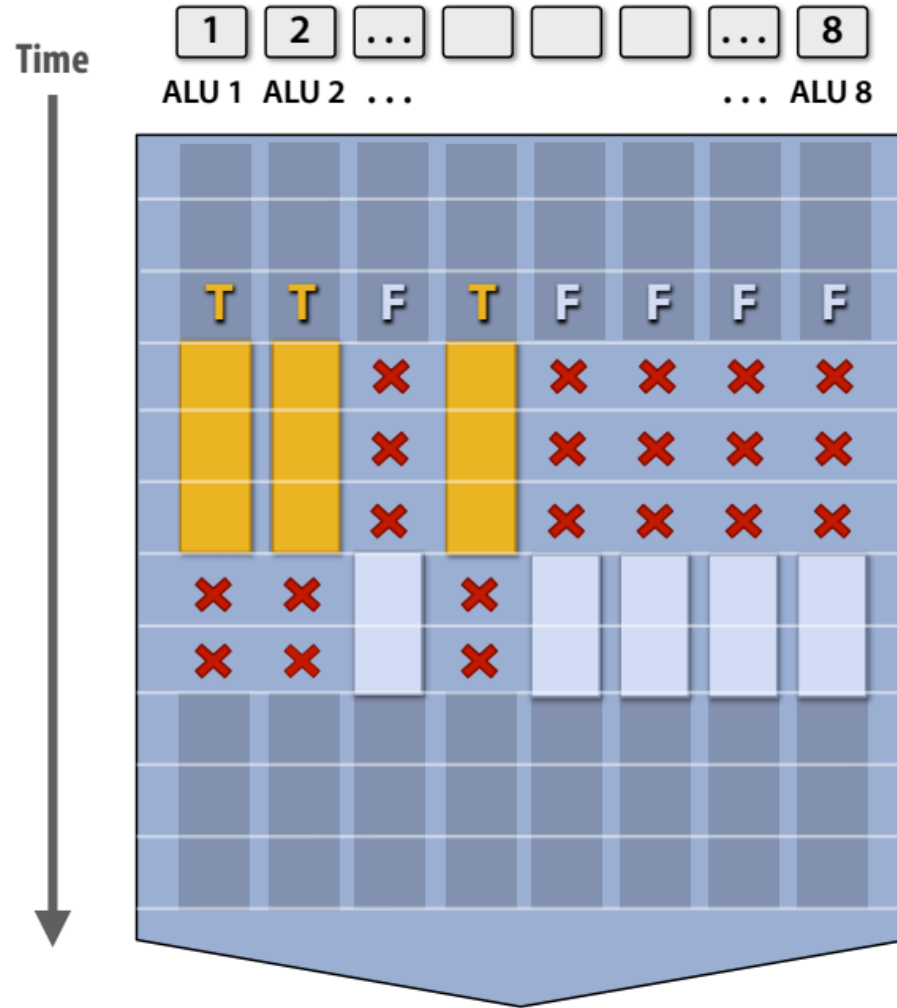
```
forall (int i from 0 to N) {  
    float t = x[i];  
  
    if (i % 8 == 0) { // 특정 조건을 만족하는 경우만 작업 수행  
        t = t * t;  
    }  
  
    y[i] = t;  
}
```

- $i \% 8 == 0$ 조건으로 인해 일부 데이터만 조건을 만족.
- 조건을 만족하지 않는 ALU는 "유향 상태"에 머물며 작업하지 않음.
- 결과적으로 SIMD ALU 8개 중 일부만 활성화되어 최악의 경우 1/8의 성능을 초래.

↓ 해결책 예시(이런 방법도 있음)

Branchless Programming: 조건문을 제거하고 모든 ALU가 동일한 작업을 수행하도록 수학적 표현을 활용

```
t = (i % 8 == 0) * (t * t) + (i % 8 != 0) * t;
```



```
forall (int i from 0 to N) {  
    float t = x[i];  
    <unconditional code>  
  
    if (t > 0.0) {  
        ???  
    } else {  
        ???  
    }  
  
    <resume unconditional code>  
    y[i] = t;  
}
```

몇 가지 일반적인 전문 용어

- 명령 스트림 일관성[Instruction stream coherence,] (“coherent execution”)
 - 동일한 명령어 순서가 여러 데이터 요소에 적용되는 프로그램의 특성.
 - SIMD 처리 리소스를 효율적으로 사용하려면 일관된 실행이 필요합니다.
 - 각 코어는 스레드의 명령어 스트림에서 다른 명령어를 가져오고 디코딩할 수 있는 기능이 있기 때문에 서로 다른 코어 간 효율적인 병렬화를 위해 일관된 실행이 필요하지 않습니다.
- “분기” 실행(“Divergent” execution)
 - 프로그램에서 명령어 스트림의 일관성이 부족함

SIMD execution: modern CPU examples

- 인텔 AVX2 지침: 256비트 연산: 8x32비트 또는 4x64비트(8폭 부동 소수점 벡터)
- 인텔 AVX512 명령어: 512비트 연산: 16x32비트...
- ARM Neon 명령어: 128비트 연산: 4x32비트...
- 명령어는 컴파일러에 의해 생성됨
 - 프로그래머가 내재성을 사용하여 명시적으로 요청한 병렬 처리
 - 병렬 언어 시맨틱을 사용하여 전달되는 병렬성(예: forall 예제)
 - “자동 벡터화” 컴파일러에 의한 루프의 의존성 분석으로 추론된 병렬성
- 용어: “명시적 SIMD”: 컴파일 타임에 SIMD 병렬화가 수행됩니다.
 - 프로그램 바이너리를 검사하고 SIMD 명령어(vstoreps, vmulps 등)를 확인할 수 있습니다.

SIMD execution on many modern GPUs

- “Implicit SIMD (암시적 SIMD)”

- Compiler는 **Scalar Binary(Scalar Instructions)**을 생성함
- 하지만, Program의 **N개의 Instance**는 Processor에서 항상 함께 실행됨

```
execute(my_function, N) // execute my_function N times
```

- (즉, Hardware 자체의 Interface는 Data-Parallel방식)
 - Compiler가 아닌 **Hardware**는 여러 SIMD ALU에서 다른 데이터를 가진 여러 **Instance의 동일한 Instruction**을 동시에 실행하는 것에 책임이 있음
- 대부분의 최신 GPU의 SIMD 폭은 8~32입니다.
 - ✓ Divergence(분기, 예 if문)는 큰 문제가 될 수 있음
 - ✓ 잘 못 짜여진 Code는 Machine의 최고 Capability의 1/32에서 실행될 수도 있음

Summary: three different forms of parallel execution

1. Superscalar Execution (명령어 수준 병렬성)

- Superscalar 처리 방식은 단일 명령어 스트림 내의 독립적인 명령어들을 동시에 실행함으로써 병렬성을 극대화. 이는 하드웨어가 실행 중에 자동으로 명령어들의 독립성을 발견하여 구현.
- 활용 분야: 동일한 명령어 스트림의 실행 단위에서 성능을 최적화하고자 하는 경우.
- 특징: 프로세서가 명령어 간 의존성을 분석해 동시에 처리 가능한 영역을 실행.

2. SIMD Execution (데이터 병렬성)

- SIMD(Single Instruction, Multiple Data)는 동일한 명령어를 여러 데이터 요소에 동시에 적용할 수 있도록 설계된 방식. 여러 ALU(Arithmetic Logic Unit)가 한 번의 명령어로 병렬 연산을 수행.
- 이점:
 - ✓ 데이터 병렬 워크로드에 효율적이며, 제어 비용을 여러 ALU에 걸쳐 분산시켜 처리 효율을 높임.
 - ✓ 병렬화는 컴파일러에 의해 명시적으로 수행될 수도 있고, 하드웨어가 실행 시간에 자동으로 처리할 수도 있음.
- 활용 분야: 과학 계산, 그래픽 처리, 머신 러닝 등 대규모 데이터 작업.

3. Multi-core Execution (스레드 수준 병렬성)

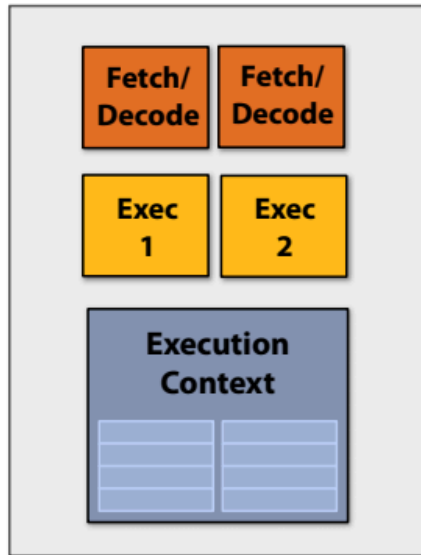
- Multi-core 처리 방식은 여러 프로세서 코어를 사용하여 서로 다른 명령어 스트림을 동시에 실행. 이를 통해 병렬성을 노출하고 최적화하려면 소프트웨어에서 스레드를 생성하여 하드웨어에 제공해야 함.
- 이점: 여러 독립적인 명령어 스트림을 동시에 처리할 수 있어 성능이 크게 향상.
- 활용 분야: 서버 애플리케이션, 고성능 컴퓨팅(HPC), 병렬 프로그램 설계.

Summary: three different forms of parallel execution

병렬 실행 형태의 차이점

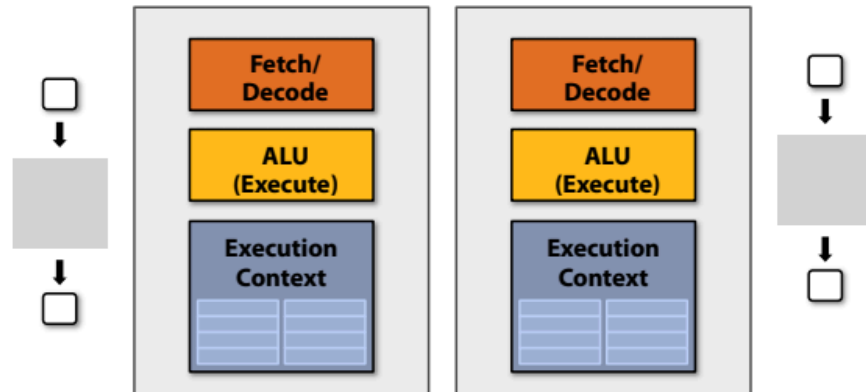
병렬 실행 형태	병렬 수준	하드웨어 의존성	주요 활용
Superscalar	명령어 수준 병렬성 (ILP)	단일 코어 내부	단일 스트림 성능 최적화
SIMD	데이터 병렬성	다중 ALU 사용	대규모 데이터 작업 병렬화
Multi-core	스레드 수준 병렬성	여러 프로세서 코어	독립적인 작업 동시 처리

Summary: three different forms of parallel execution

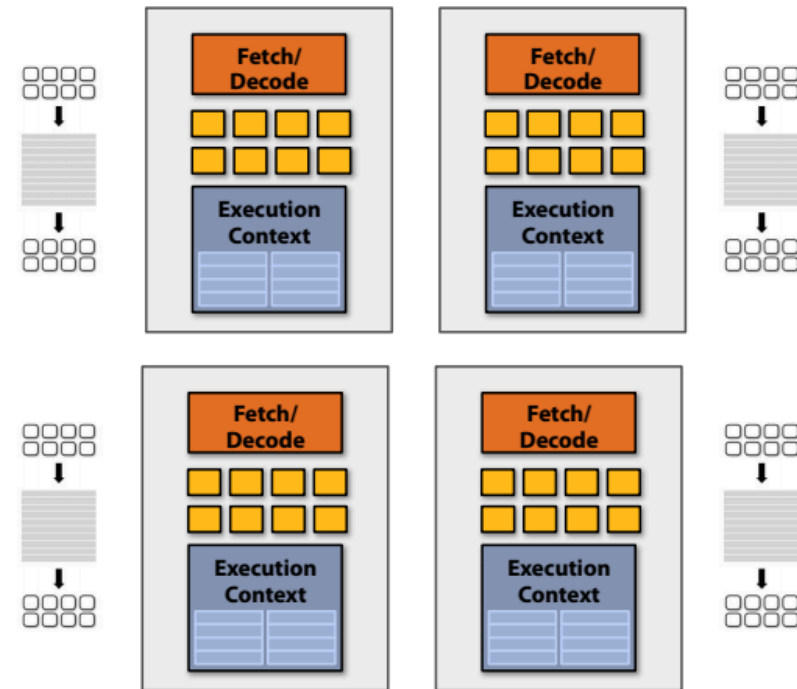


My single core, superscalar processor:
executes up to two instructions per clock
from a single instruction stream (if the
instructions are independent)

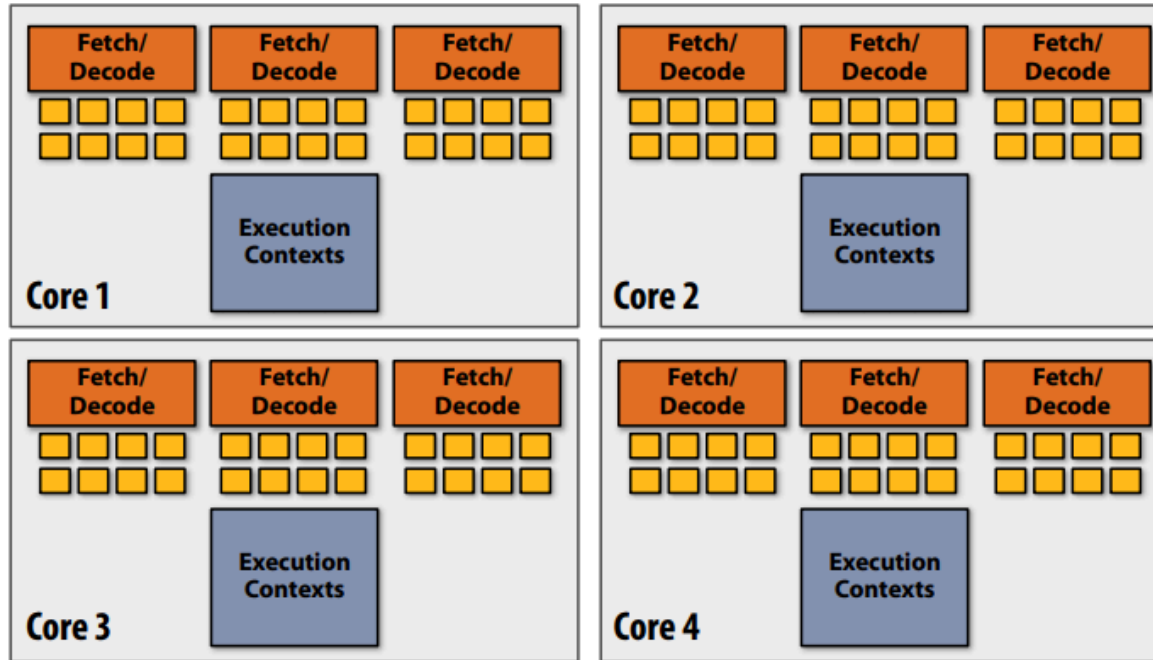
My dual-core processor:
executes one instruction per clock
from one instruction stream on each core.



My SIMD quad-core processor:
executes one 8-wide SIMD instruction per clock
from one instruction stream on each core.



Example: four-core Intel i7-7700K CPU



4 core processor

**Three 8-wide SIMD ALUs per core
(AVX2 instructions)**

4 cores x 8-wide SIMD x 3 x 4.2 GHz = 400 GFLOPs

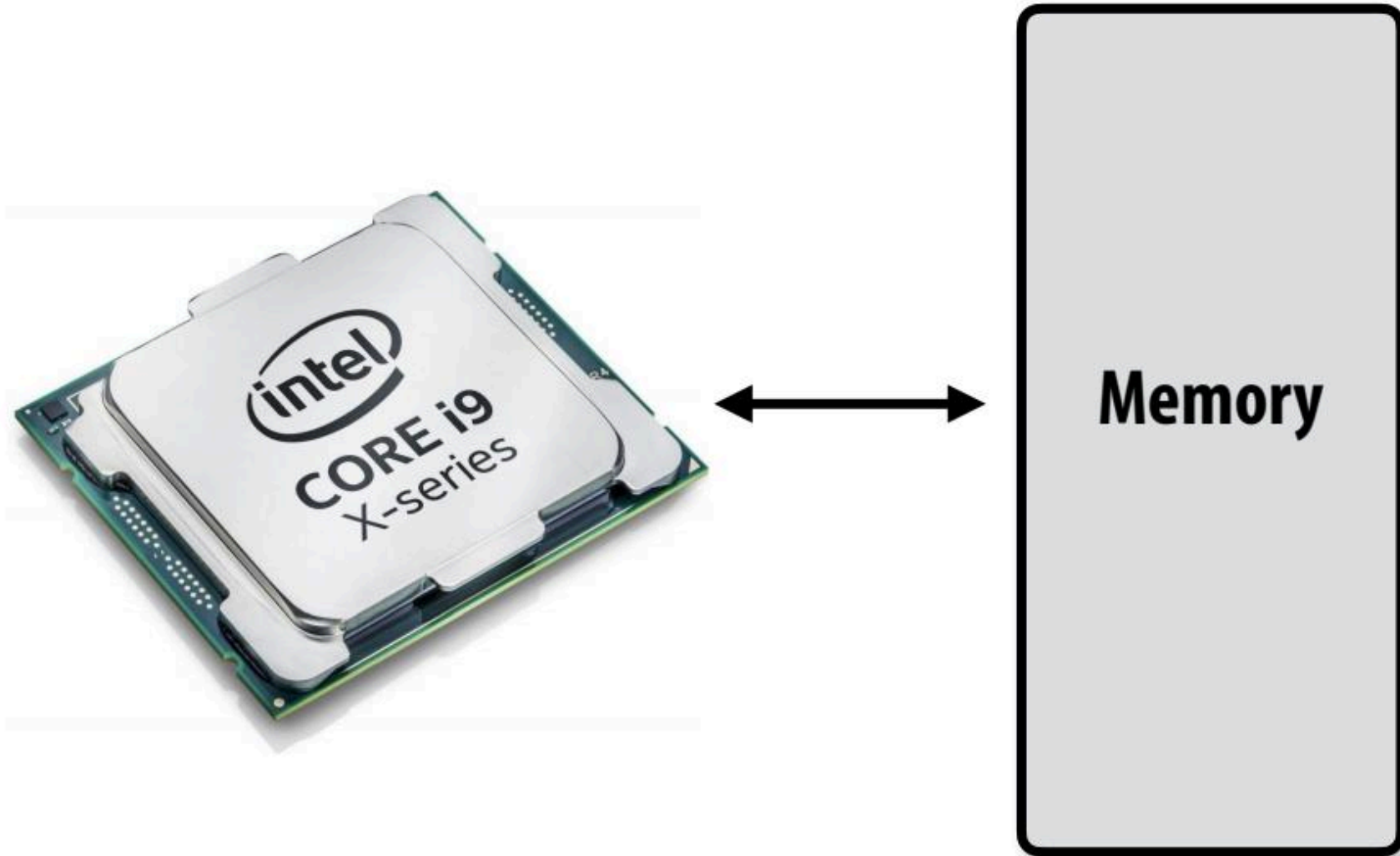
Example: NVIDIA V100 GPU



80 "SM" cores

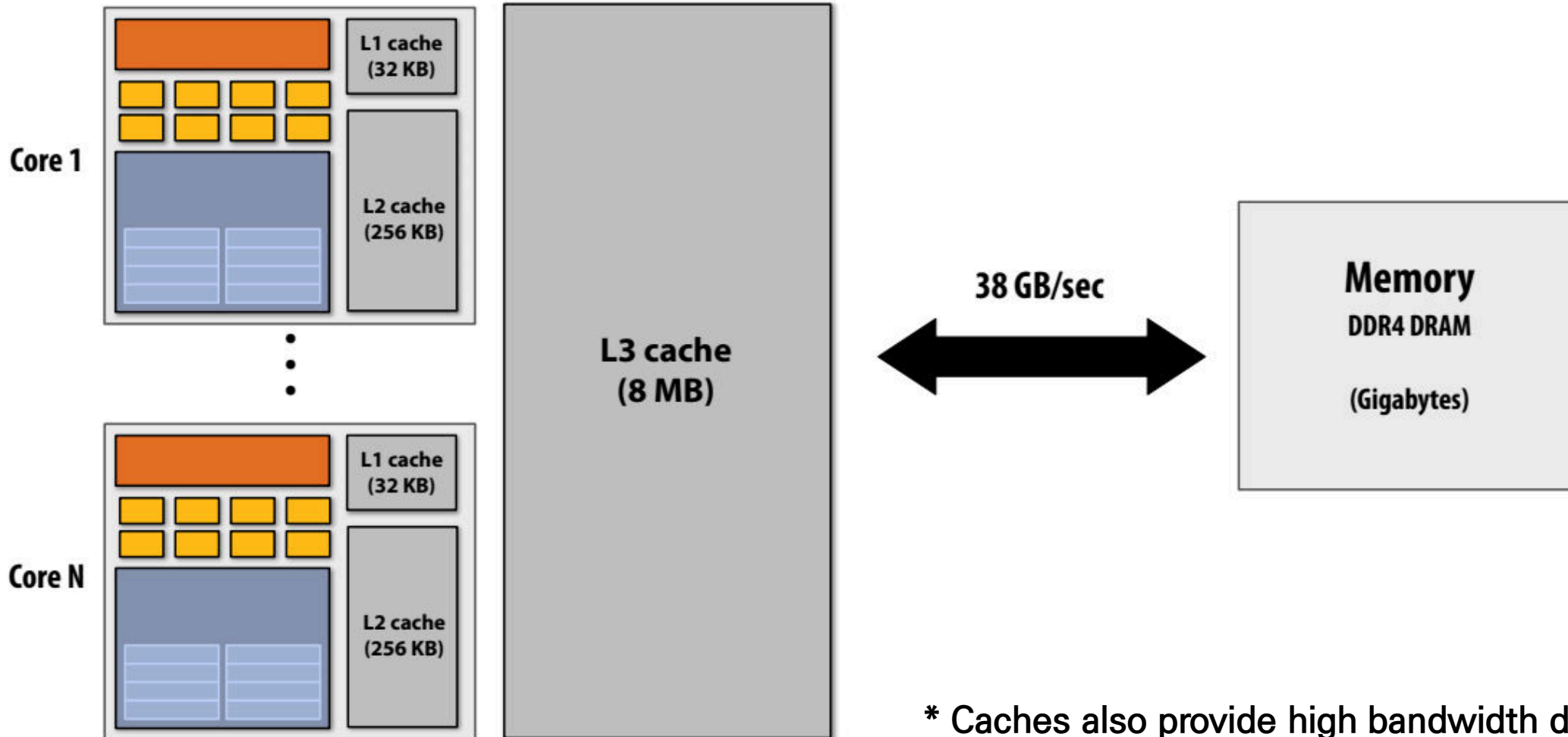
128 SIMD ALUs per "SM" (@1.6 GHz) = 16 TFLOPs (~250 Watts)

Part 2: accessing memory



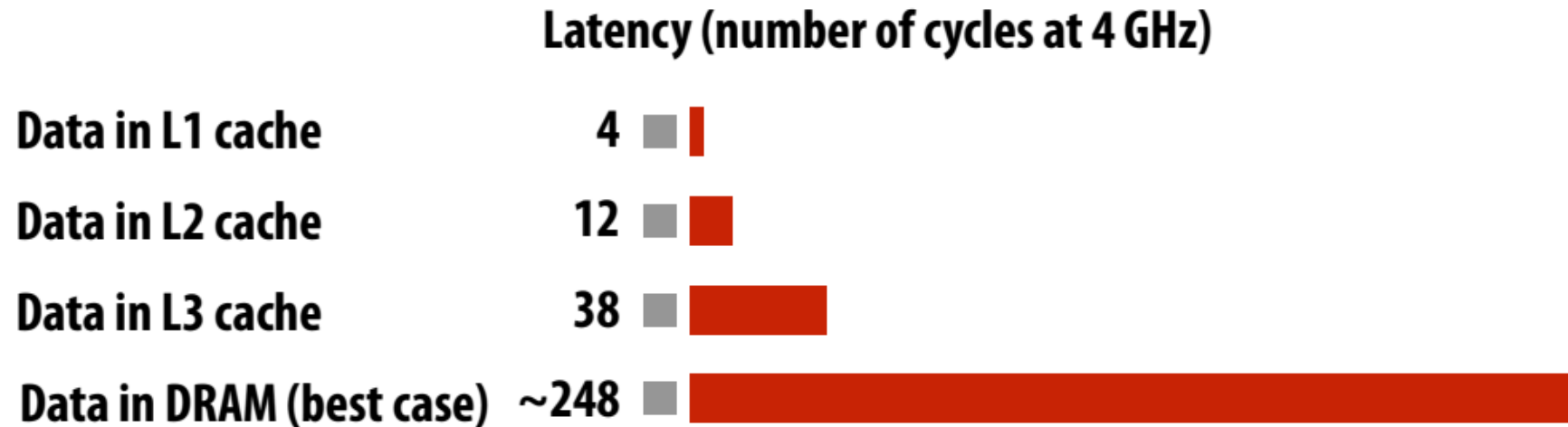
캐시(cache)는 스톨(stall) 시간을 감소(메모리 액세스 지연 시간 감소).

- 왜 일반적인 Processor에는 여러 Cache를 가지고 있을까?
→ 그 이유는 Latency를 줄이는 것(Reducing)에 목적을 두고 있다.
- Processor는 Memory와의 상호작용보다 **Cache에 존재하는 데이터를 효율적으로 실행시킬** 수 있다.
- 또한 Cache는 CPU에 데이터를 전달하기에 **높은 Bandwidth를 가지고 있어 결과적으로 Latency를 줄일** 수 있다.



Recall: [very] long latency of data access

(Kaby Lake CPU)



Recall this access pattern

데이터 읽기 패턴: 프로그램이 16바이트 배열을 순차적으로 읽고 다시 읽는 상황을 가정

총 캐시 용량 = 8바이트

- 캐시에는 4바이트 캐시 라인. (따라서 캐시에 2줄이 들어감)
- 최소 최근 사용량(LRU) - 교체 정책

토론 질문:

- 주소 0x0의 두 번째 읽기에 '히트'가 없는 이유는 무엇인가요?
- 주소 0x4의 두 번째 읽기는 어떨까요?
- 캐시의 용량이 4줄이라면 답이 달라질까요?

데이터 액세스 패턴과 캐시 성능

1.캐시 메모리의 구조와 용량:

- 캐시는 CPU와 메인 메모리 간의 데이터 액세스를 빠르게 하기 위해 설계됨.
- 총 캐시 용량이 8바이트이고, 4바이트 캐시 라인을 사용하는 시스템을 가정. 따라서 두 개의 캐시 라인이 캐시에 저장될 수 있음.

2.LRU(Least Recently Used) 대체 정책:

- 캐시는 최근에 사용된 데이터를 유지하고, 오래 사용하지 않은 데이터를 대체하는 LRU 정책을 사용.
- 이는 액세스 패턴에 따라 캐시의 히트(hit) 또는 미스(miss)가 발생하는지 결정함.

•왜 두 번째 읽기에서 히트가 발생하지 않는가?

- ✓ 순차적으로 데이터를 읽으면서, 배열 크기가 캐시 용량을 초과하면 이전 데이터를 캐시에서 대체합니다.
- ✓ 따라서 동일한 데이터를 반복적으로 사용하는 경우에도 효율이 떨어집니다.

•캐시 라인이 4개로 늘어나면 어떻게 되는가?

- ✓ 캐시의 용량이 증가하면 더 많은 데이터를 한 번에 유지할 수 있어, 두 번째 읽기에서 히트율이 높아질 것입니다.
- ✓ 이는 프로그램의 메모리 접근 효율을 향상시키는 중요한 요소입니다.

		Address accessed	Cache action
Line 0x0	0x0	0x0	"cold miss", load 0x0
	0x1	0x1	hit
	0x2	0x2	hit
	0x3	0x3	hit
Line 0x4	0x4	0x4	"cold miss", load 0x4
	0x5	0x5	hit
	0x6	0x6	hit
	0x7	0x7	hit
Line 0x8	0x8	0x8	"cold miss", load 0x8 (evict 0x0)
	0x9	0x9	hit
	0xA	0xA	hit
	0xB	0xB	hit
Line 0xC	0xC	0xC	"cold miss", load 0xC (evict 0x4)
	0xD	0xD	hit
	0xE	0xE	hit
	0xF	0xF	hit
		0x0	"capacity miss", load 0x0 (evict 0x8)

Data prefetching reduces stalls (hides latency)

- 모든 일반적인 CPU는 Cache에서 데이터를 Prefetch하기 위한 Logic을 가지고 있음.
 - ➔ 동적으로 프로그램 접근 패턴을 분석하여 다음에 어떠한 데이터가 접근할 것인지를 예측.
 - ➔ 이것은 위의 Cache와는 다르게 직접적으로 Latency를 줄이는 것보다 줄여지는 것(Hiding)처럼 보이게 한다.
- 만약 접근하려는 데이터가 Cache에 존재한다면 Stalls 현상을 줄일 수도 있다.

```
predict value of r2, initiate load  
predict value of r3, initiate load
```

...

...

...

...

...

...

```
ld r0 mem[r2]
```

```
ld r1 mem[r3]
```

```
add r0, r0, r1
```

These loads are cache hits

data arrives in cache

data arrives in cache

Note: Prefetching can also reduce performance if the guess is wrong (consumes bandwidth, pollutes caches)

- ➔ 다만, 예측을 Logic에 의한 예측이 잘못된 경우 오히려 성능을 떨어뜨릴 수 있다.
- ➔ 즉, Bandwidth를 낭비하거나 Cache를 불필요한 데이터로 채울 수 있는 문제가 발생한다.

Data prefetching reduces stalls (hides latency)

**But what if data hasn't been read recently,
so does not reside in cache?**

**And the next piece of data to
read is not easily predictable?**

```
int x = some_function();  
int y = A[x];
```

Consider doing your laundry...



Credit: <https://www.theodysseyonline.com/the-dos-and-donts-of-dorm-laundry>

Or cooking a meal...

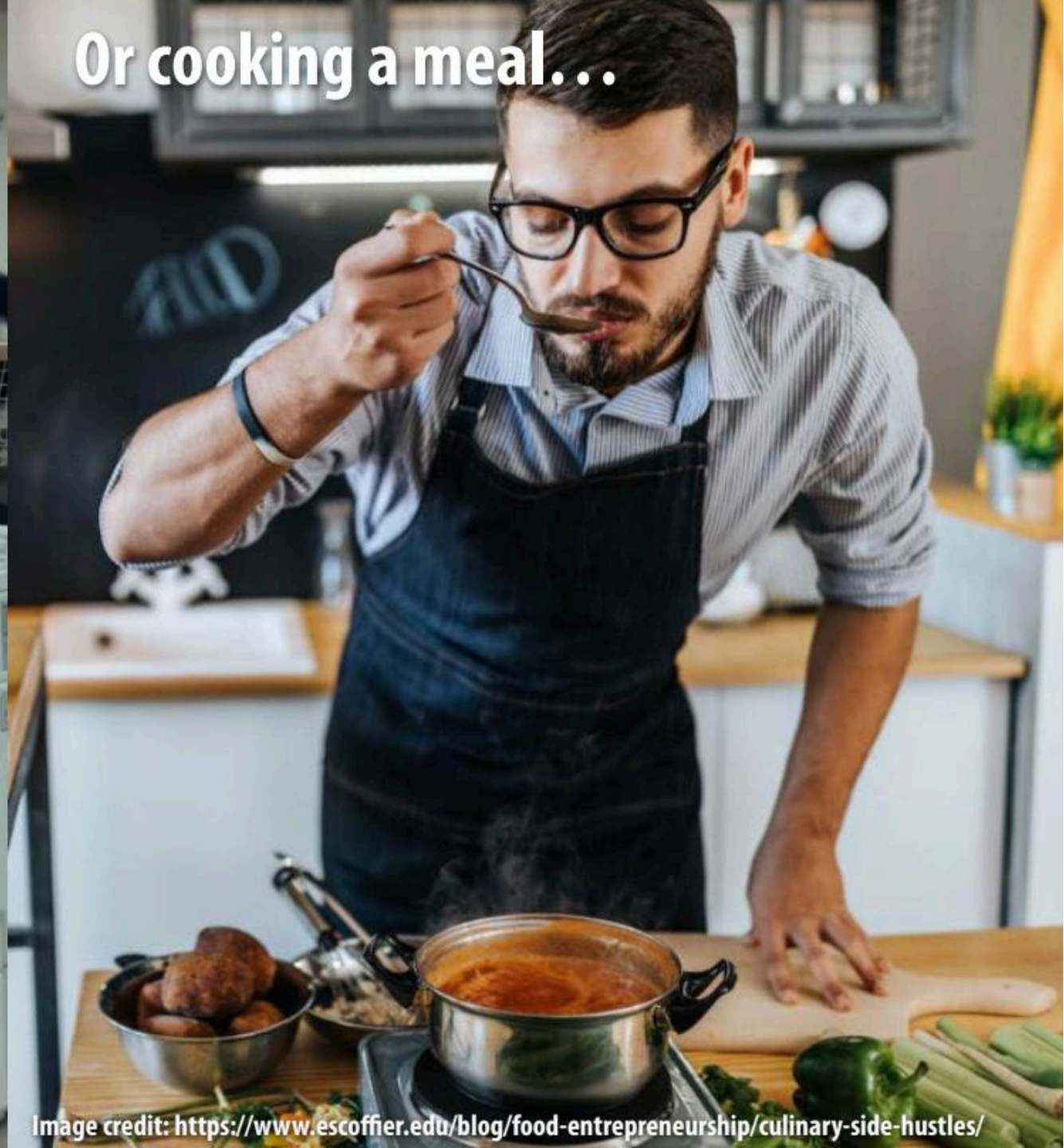
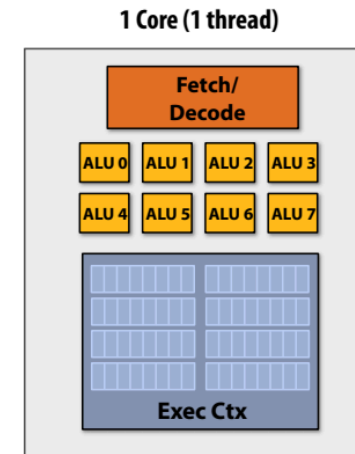
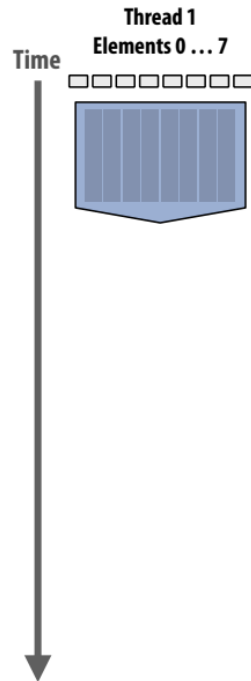


Image credit: <https://www.escoffier.edu/blog/food-entrepreneurship/culinary-side-hustles/>

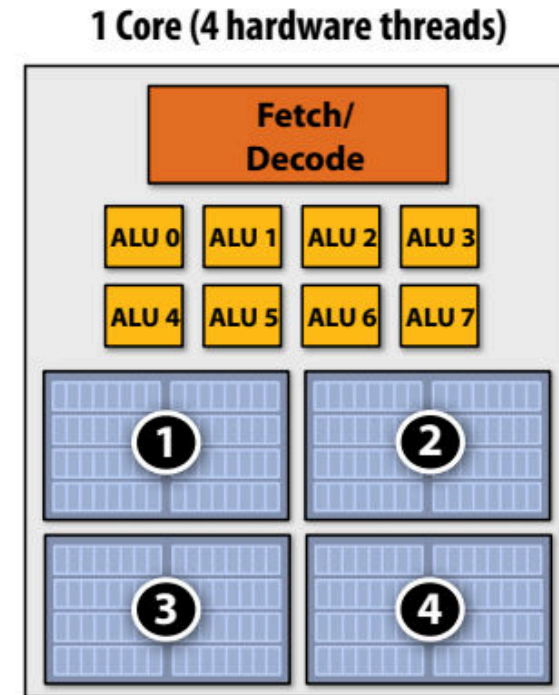
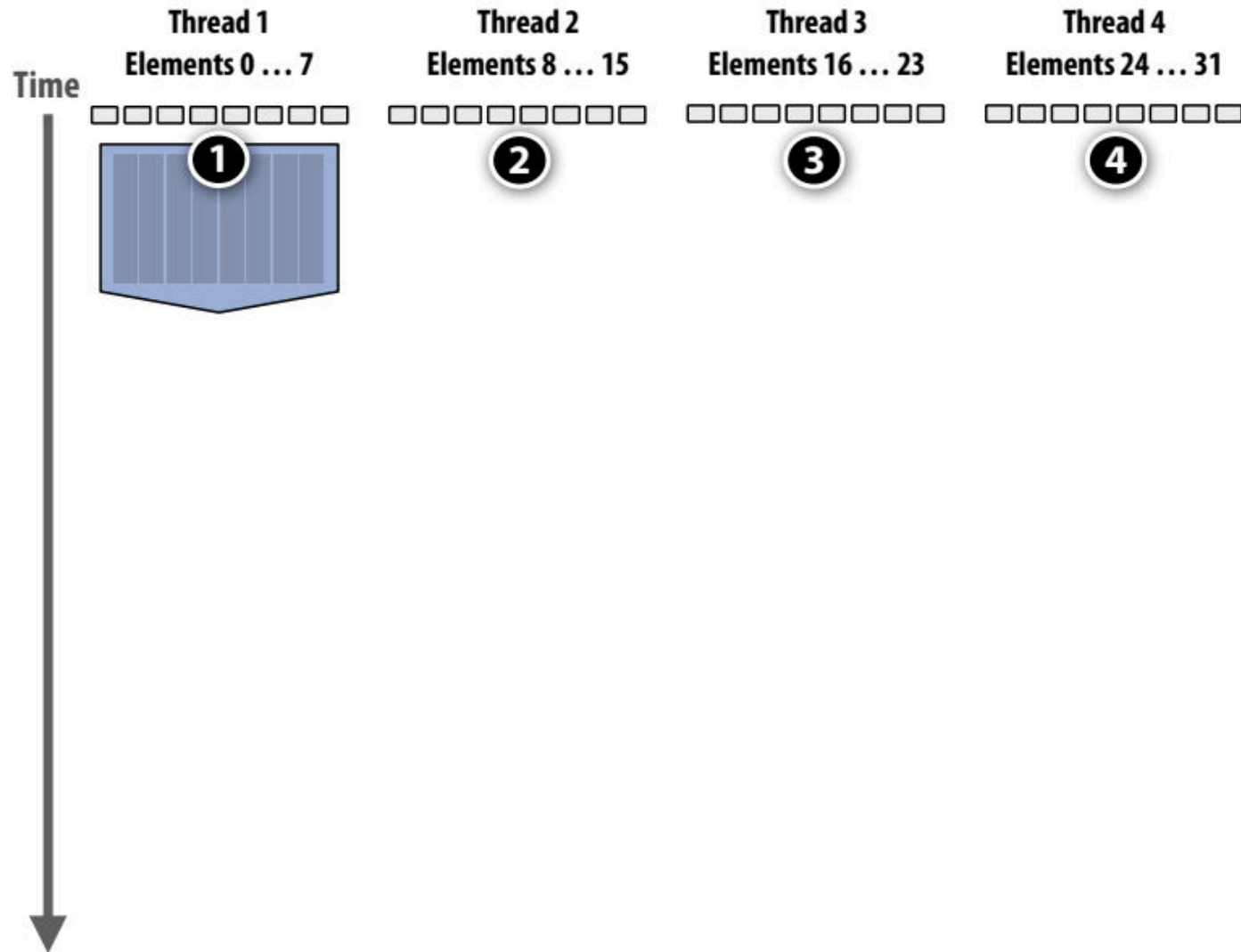
Multi-threading reduces stalls

- Multi-Threading은 Stalls 상황을 줄이기 위해 동일한 Core에서 여러 Thread의 Interleave Processing을 수행
 - ➔또한 Prefetching과 마찬가지로 Latency를 줄이는게 아니라 줄이는 것처럼 보이게 함.
 - ➔Throughput-Oriented System의 핵심적인 요소는 여러 Thread를 실행할 때 전체적인 System Throughput을 늘리기 위해 하나의 Thread에서 하나의 작업을 완료하는데 걸리는 시간을 잠재적으로 늘린다.

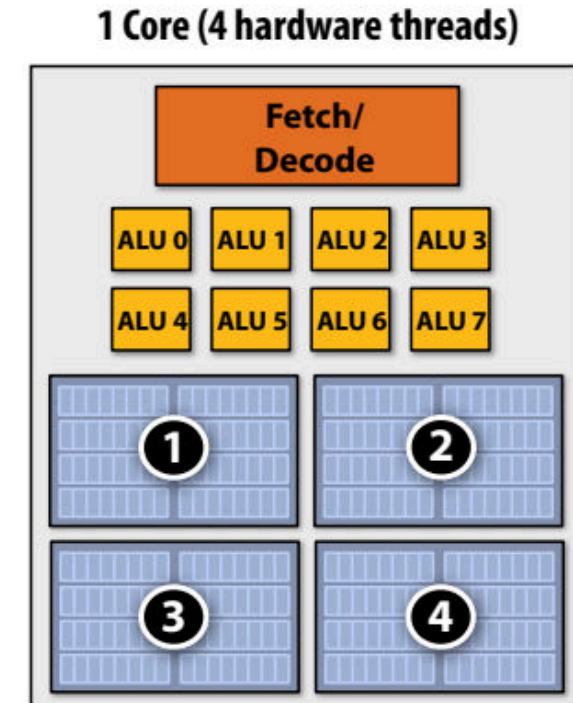
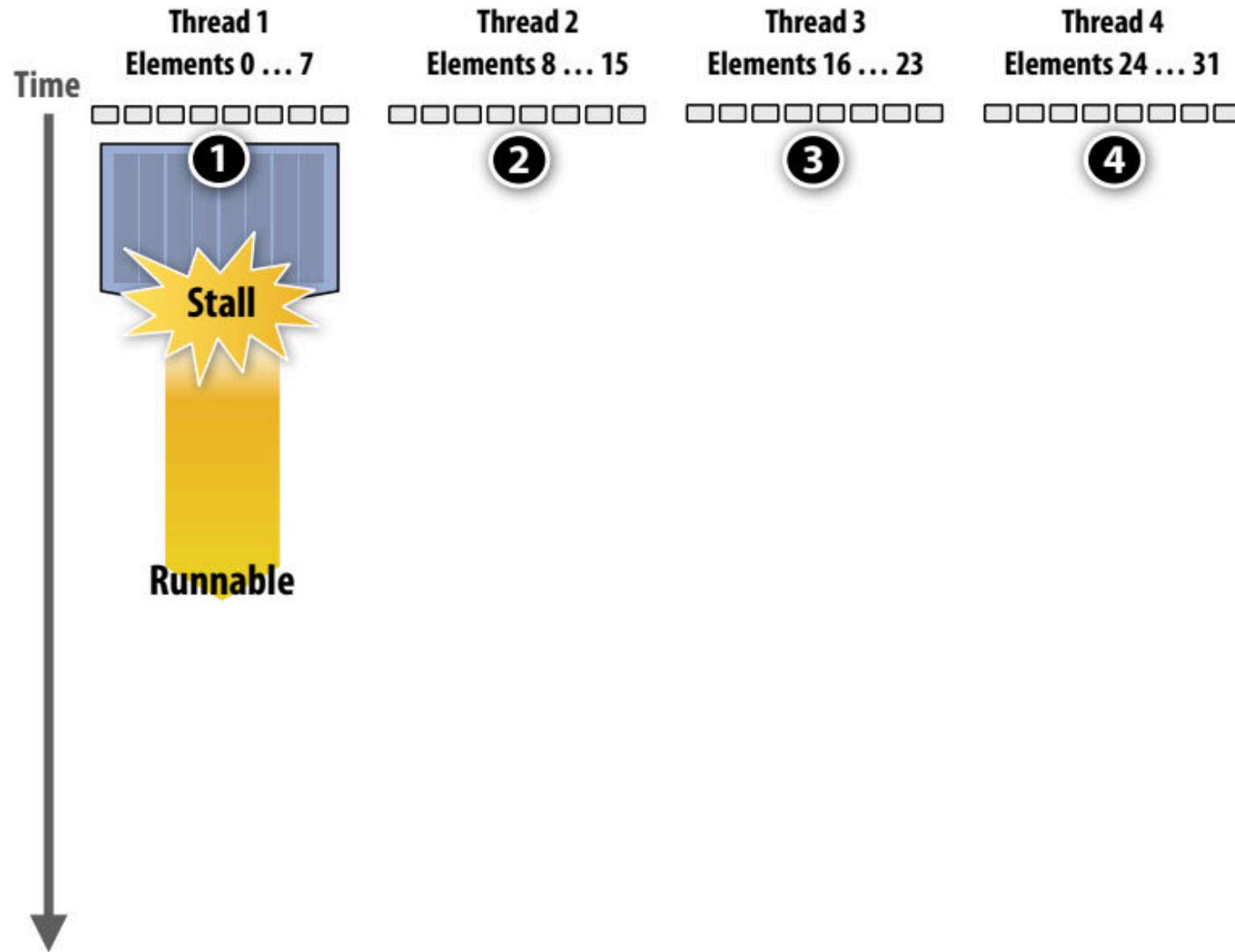
Hiding stalls with multi-threading



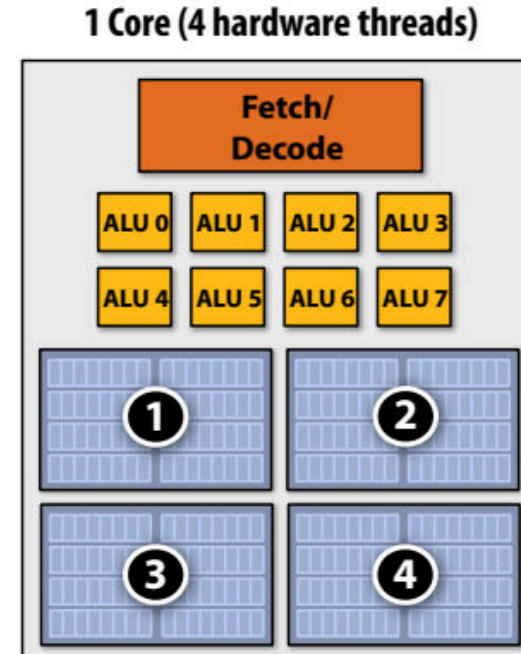
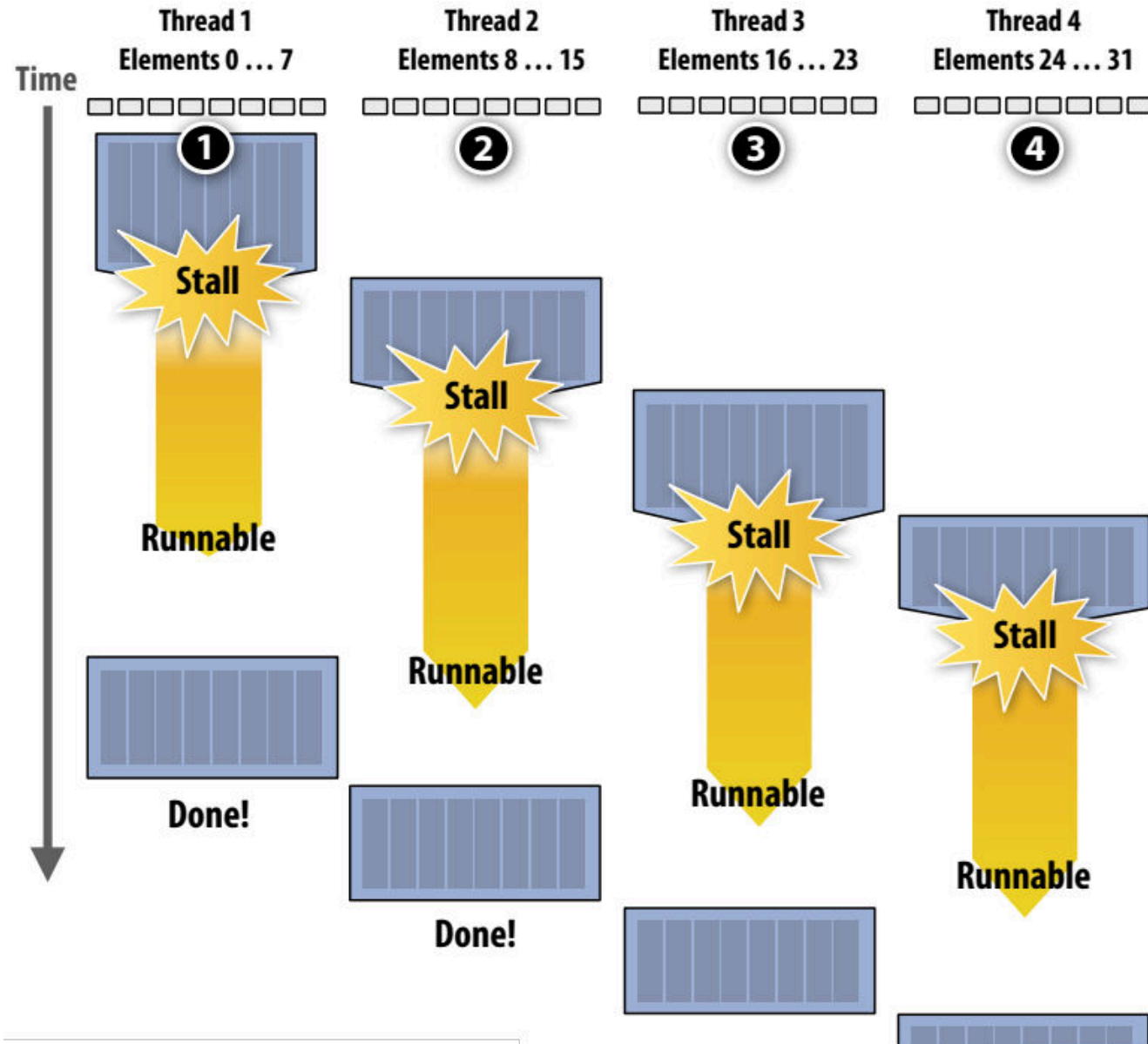
Hiding stalls with multi-threading



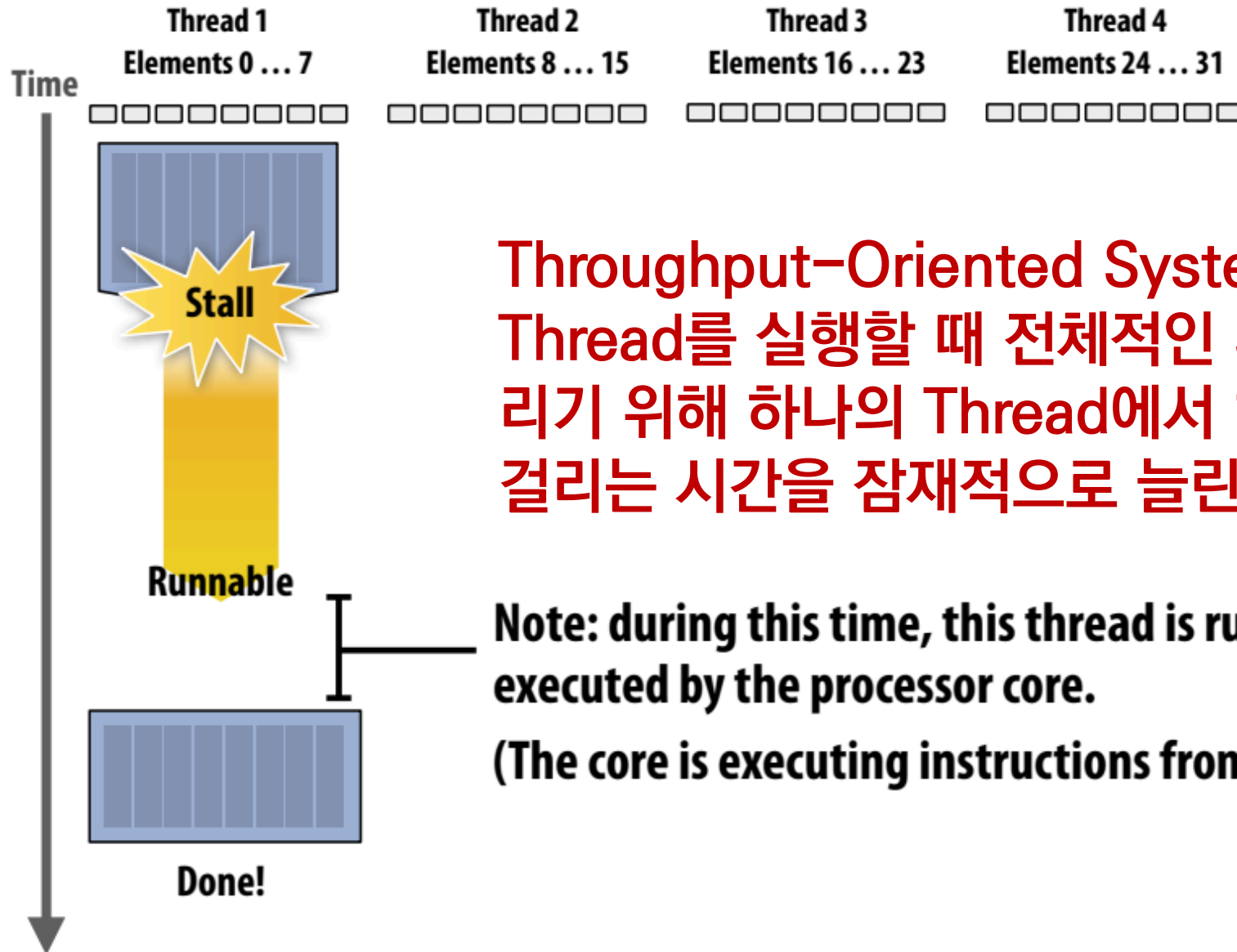
Hiding stalls with multi-threading



Hiding stalls with multi-threading



Throughput computing: a trade-off



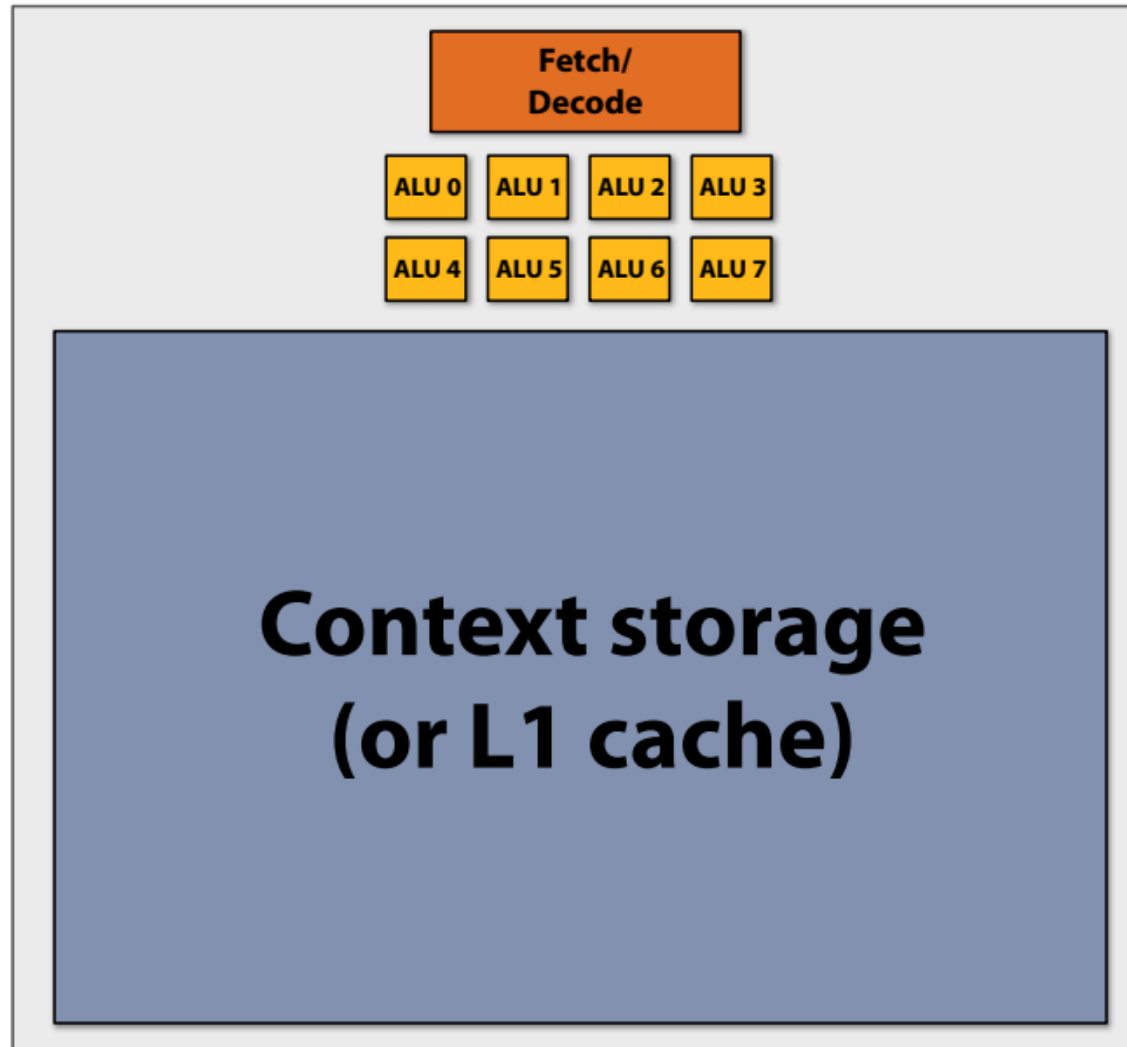
Throughput-Oriented System의 핵심적인 요소는 여러 Thread를 실행할 때 전체적인 System Throughput을 늘리기 위해 하나의 Thread에서 하나의 작업을 완료하는데 걸리는 시간을 잠재적으로 늘린다.

Note: during this time, this thread is runnable, but it is not being executed by the processor core.

(The core is executing instructions from another thread.)

No free lunch: storing execution contexts

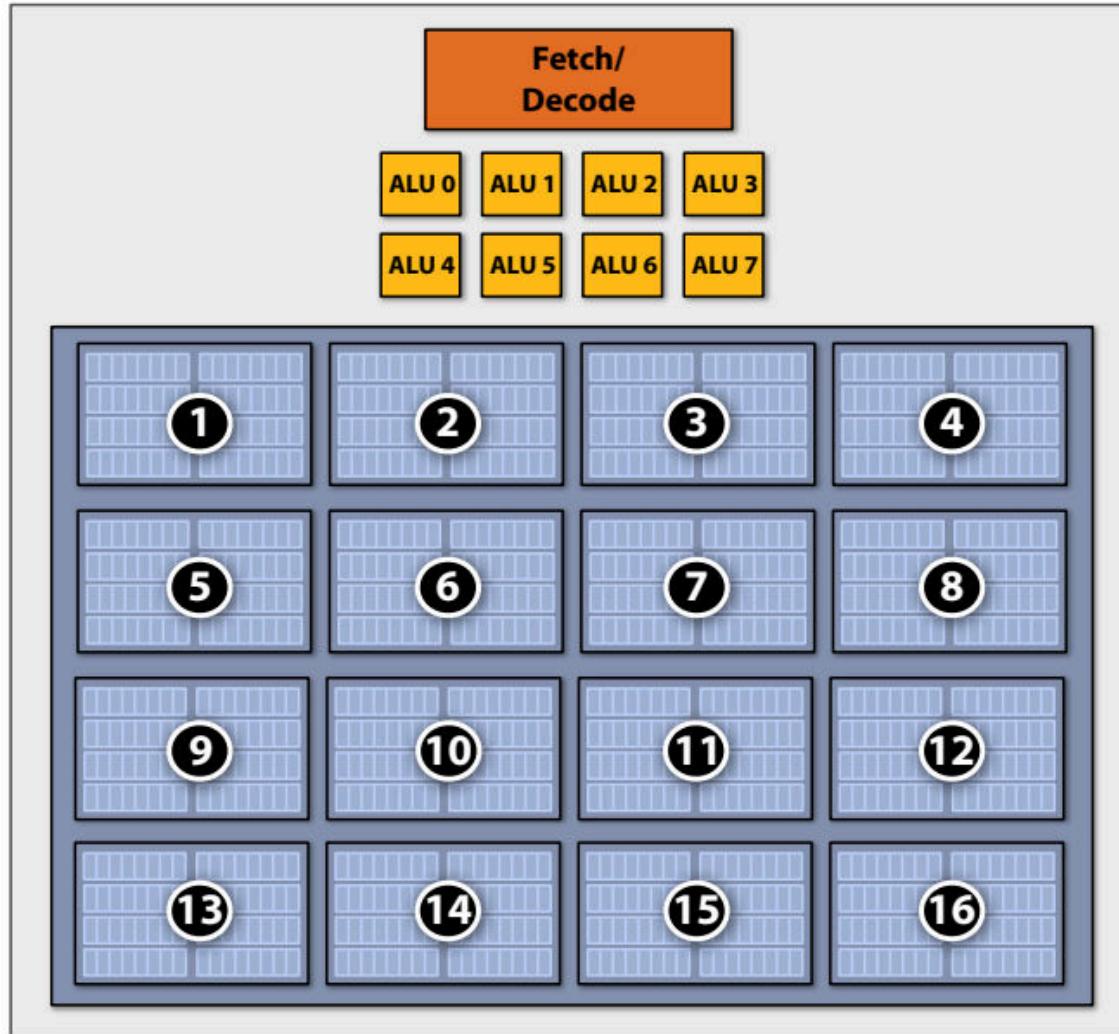
실행 컨텍스트(execution contexts)의 온칩 스토리지를 유한한 리소스로 간주



Many small contexts (high latency hiding ability)

많은 작은 컨텍스트(높은 지연 시간 숨기기 기능)

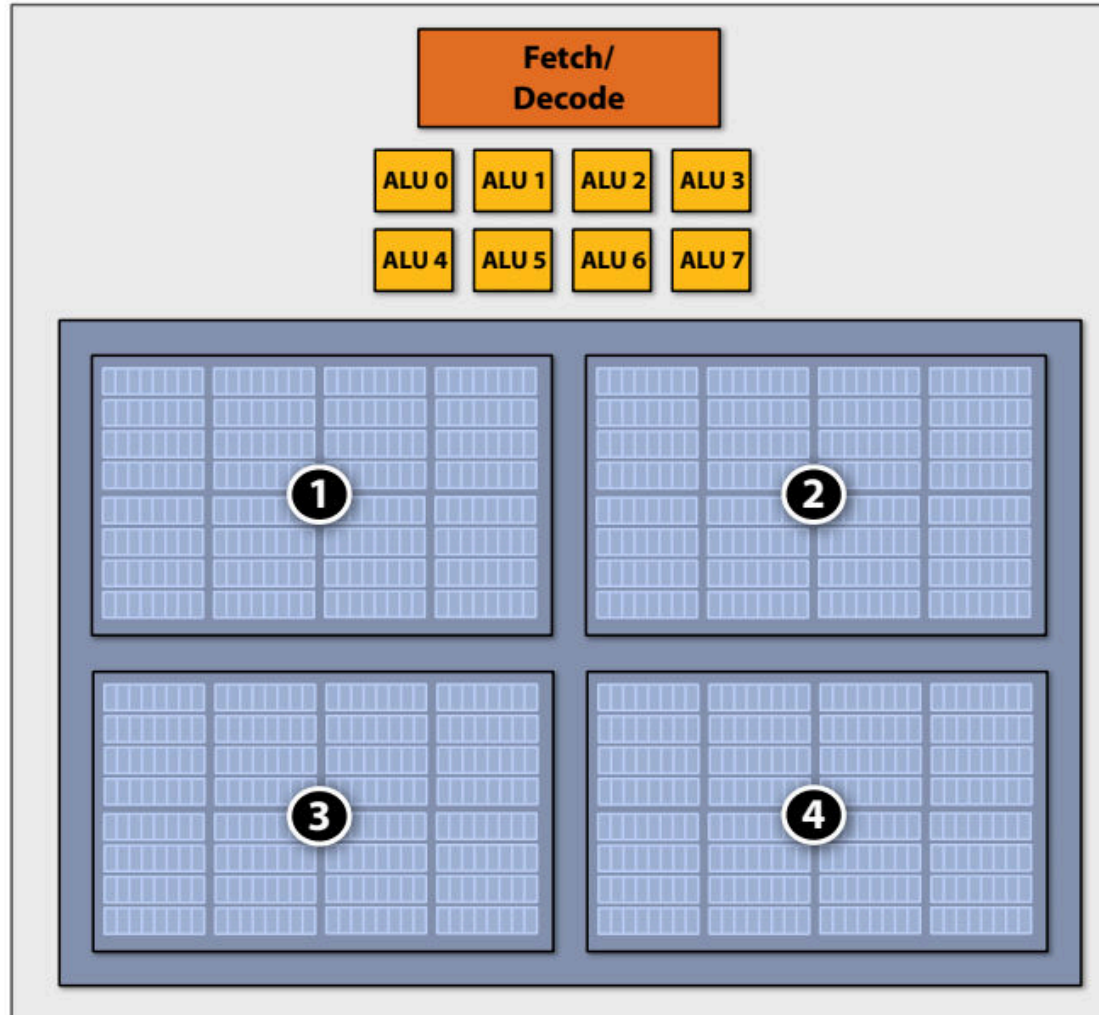
16개의 하드웨어 스레드: 스레드당 소규모 작업 세트를 위한 스토리지



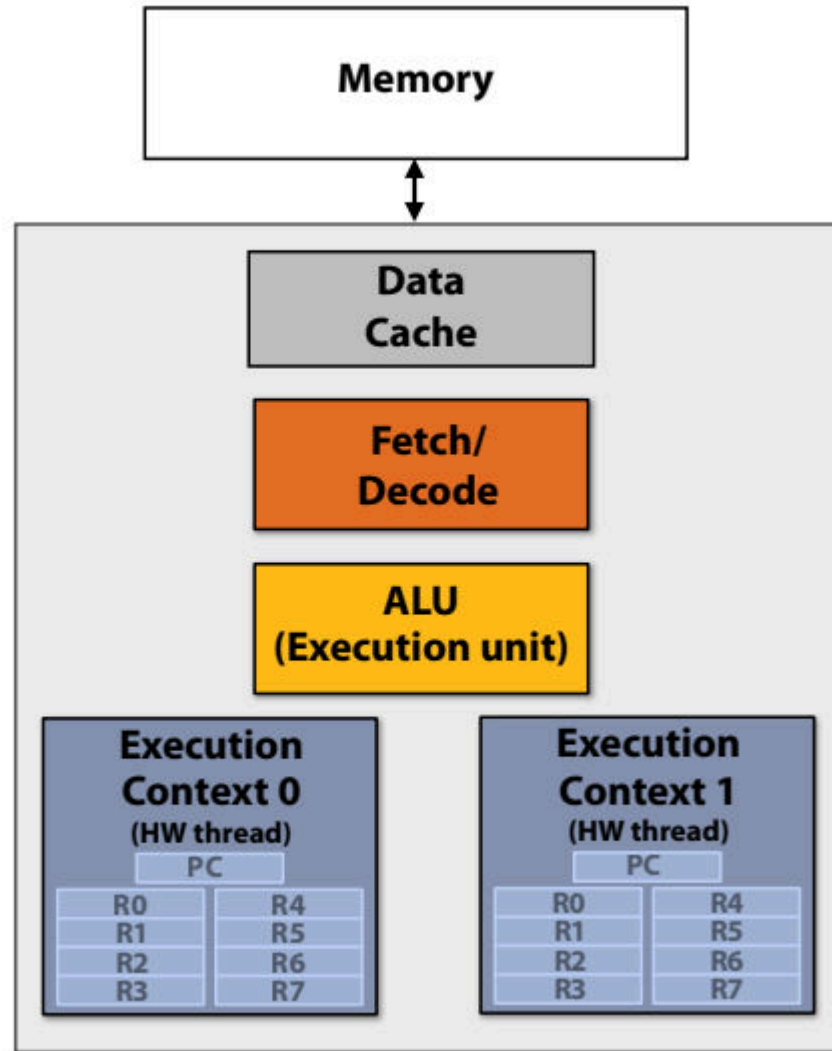
Four large contexts (low latency hiding ability)

4개의 큰 컨텍스트(낮은 지연 시간 숨김 기능)

4개의 하드웨어 스레드: 스레드당 대규모 작업 세트를 위한 스토리지

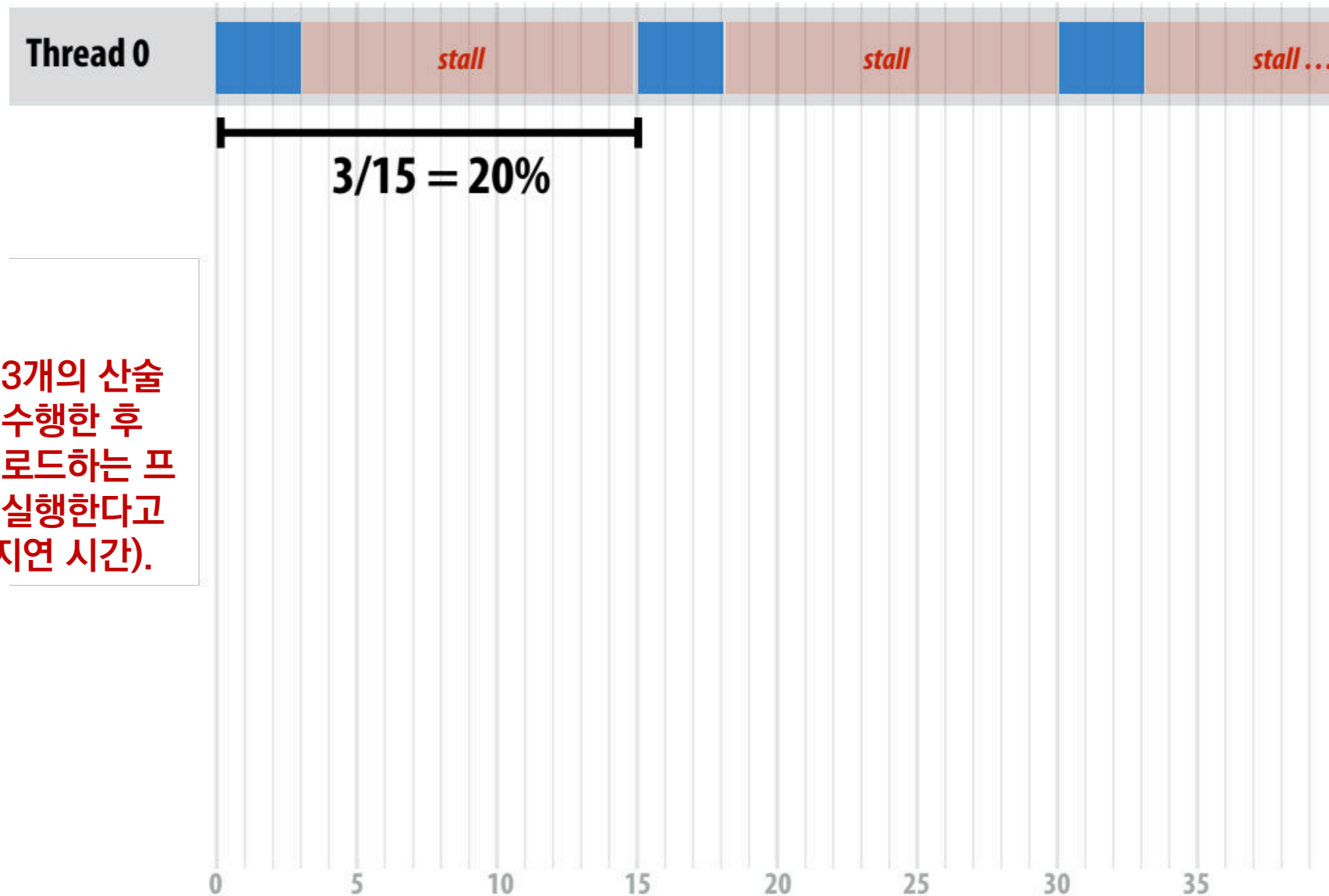


Exercise: consider a simple two-threaded core



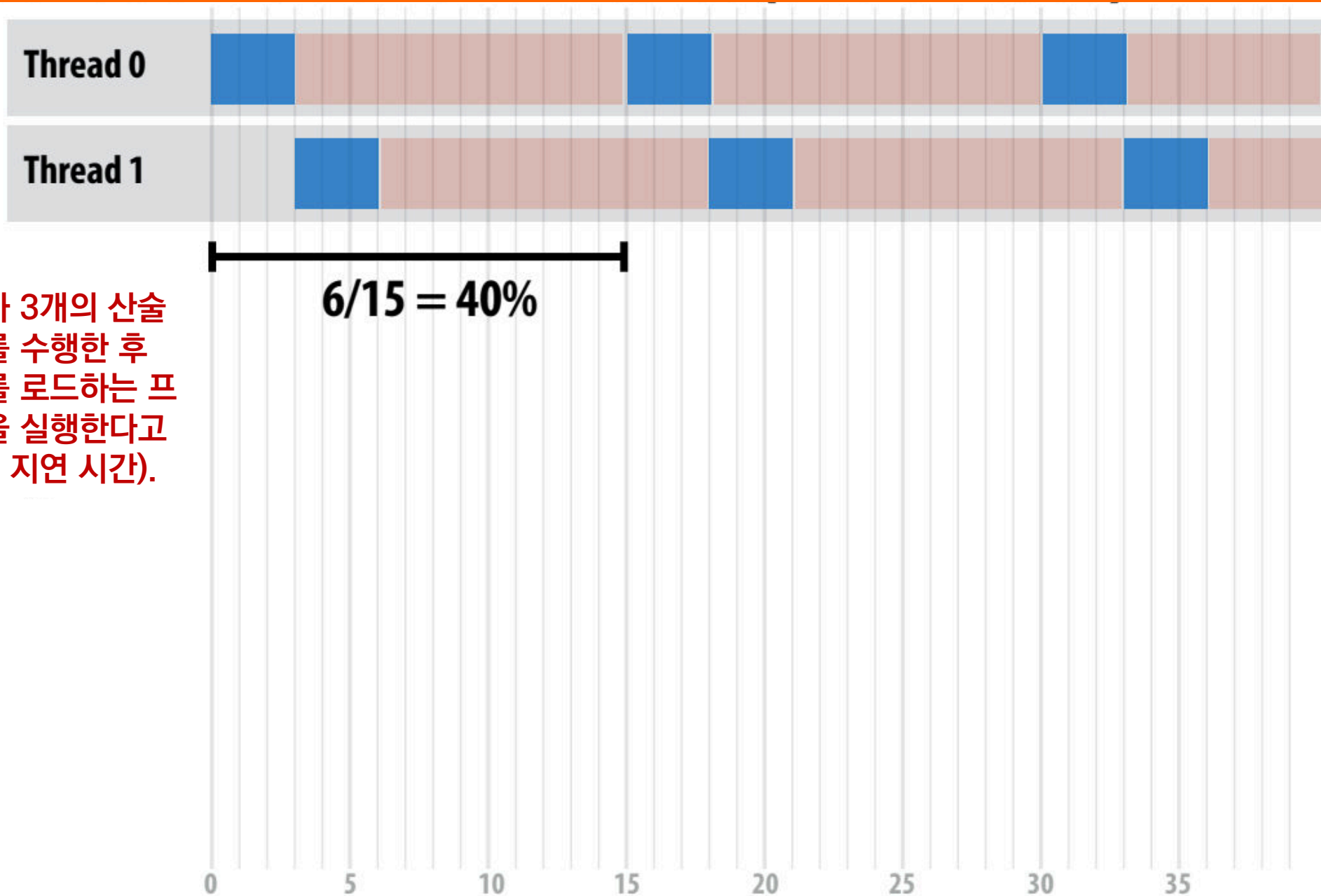
Single core processor, multi-threaded core (2 threads).
Can run one scalar instruction per clock from one of the hardware threads

What is the utilization of the core? (one thread)



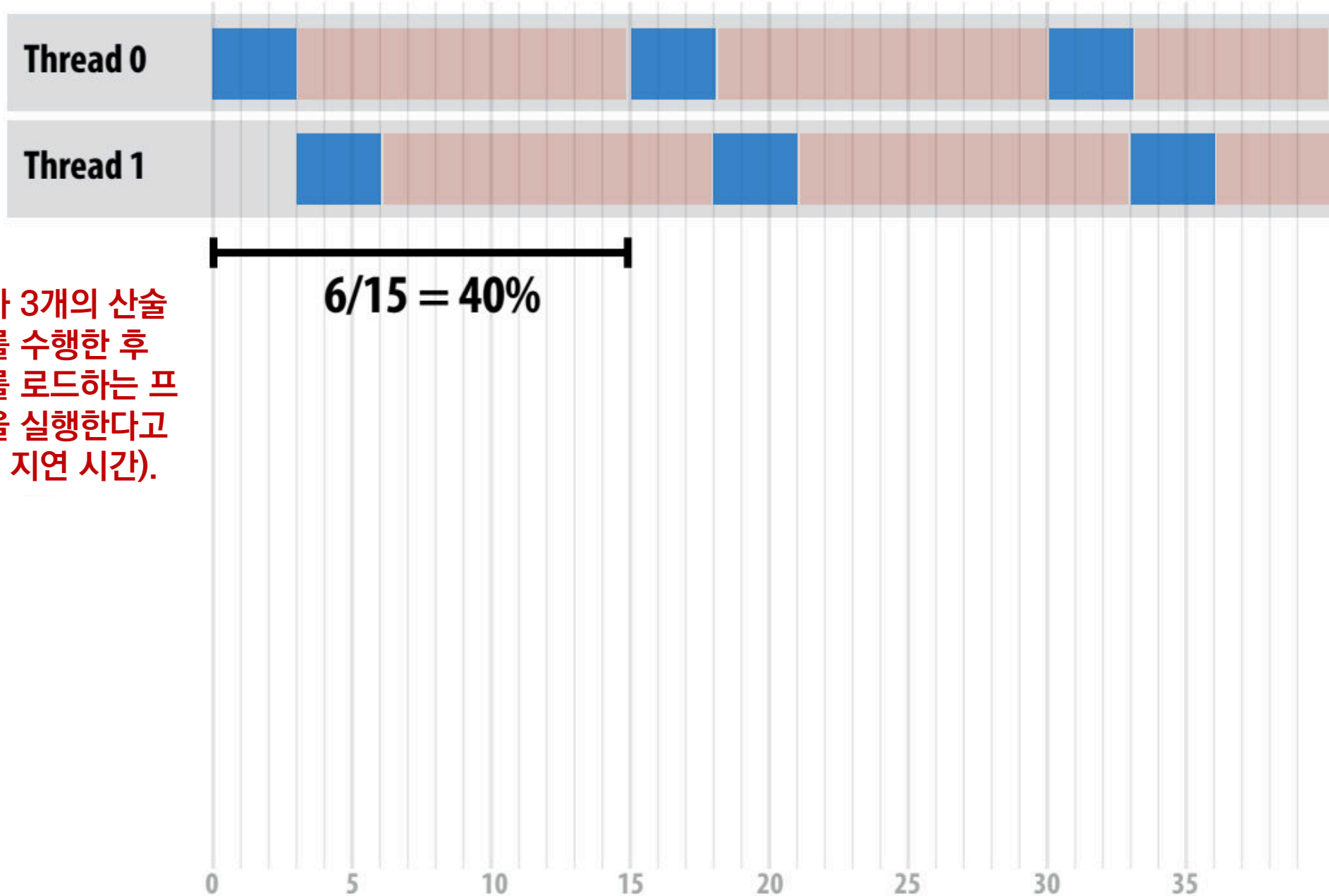
스레드가 3개의 산술 명령어를 수행한 후 메모리를 로드하는 프로그램을 실행한다고 (12주기 지연 시간).

What is the utilization of the core? (one thread)



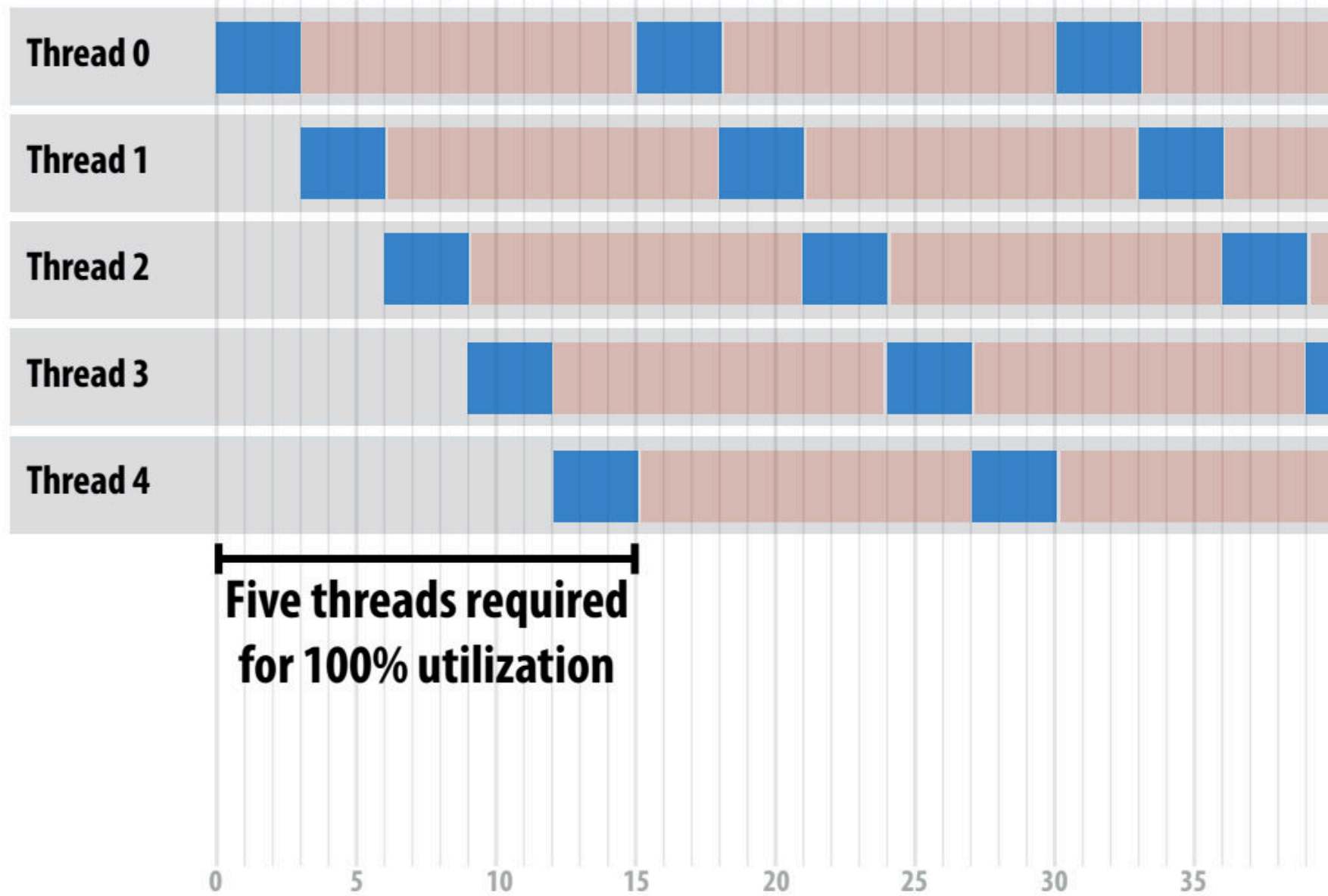
스레드가 3개의 산술 명령어를 수행한 후 메모리를 로드하는 프로그램을 실행한다고 (12주기 지연 시간).

How many threads are needed to achieve 100% utilization?

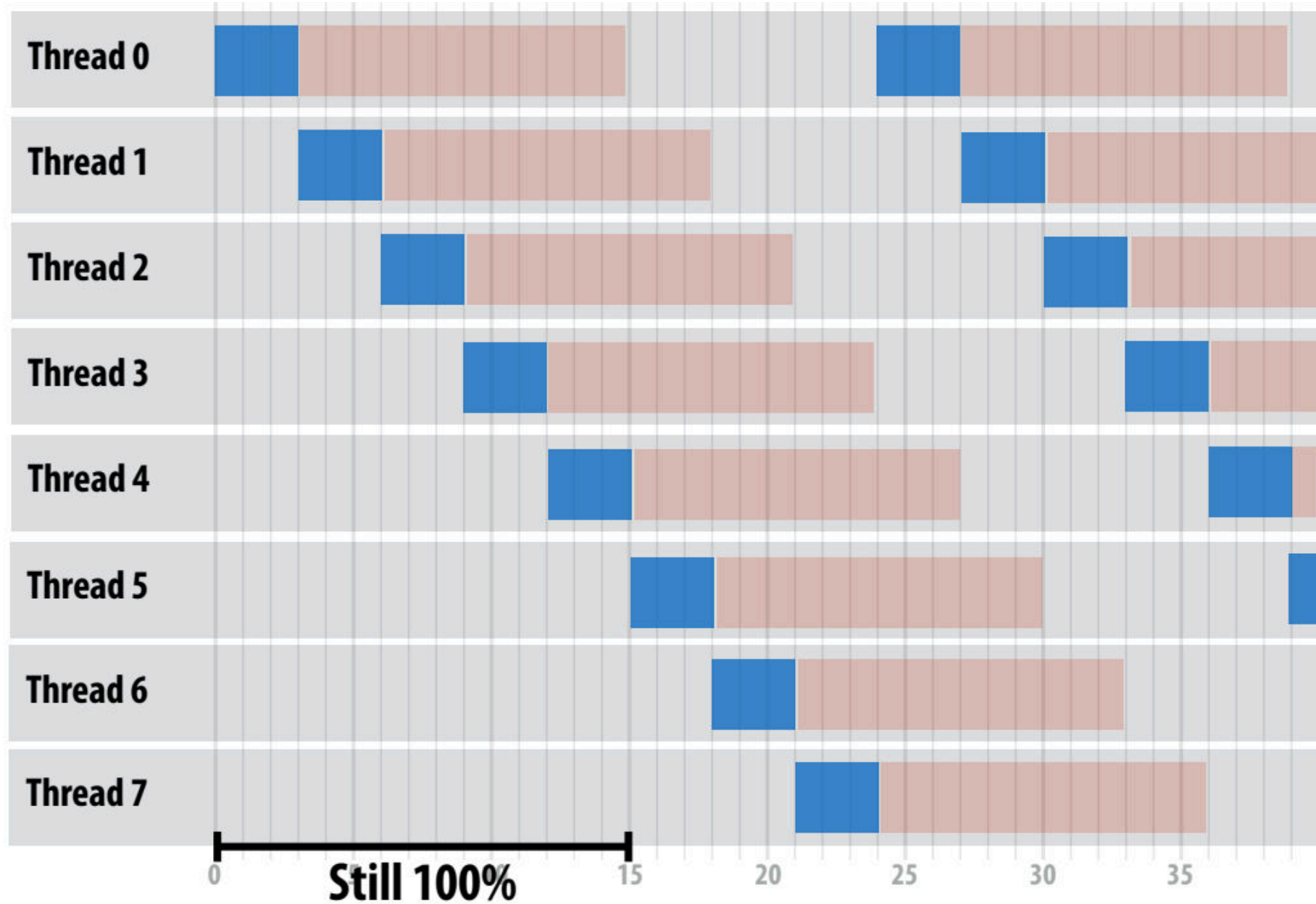


스레드가 3개의 산술 명령어를 수행한 후 메모리를 로드하는 프로그램을 실행한다고 (12주기 지연 시간).

Five threads needed to obtain 100% utilization

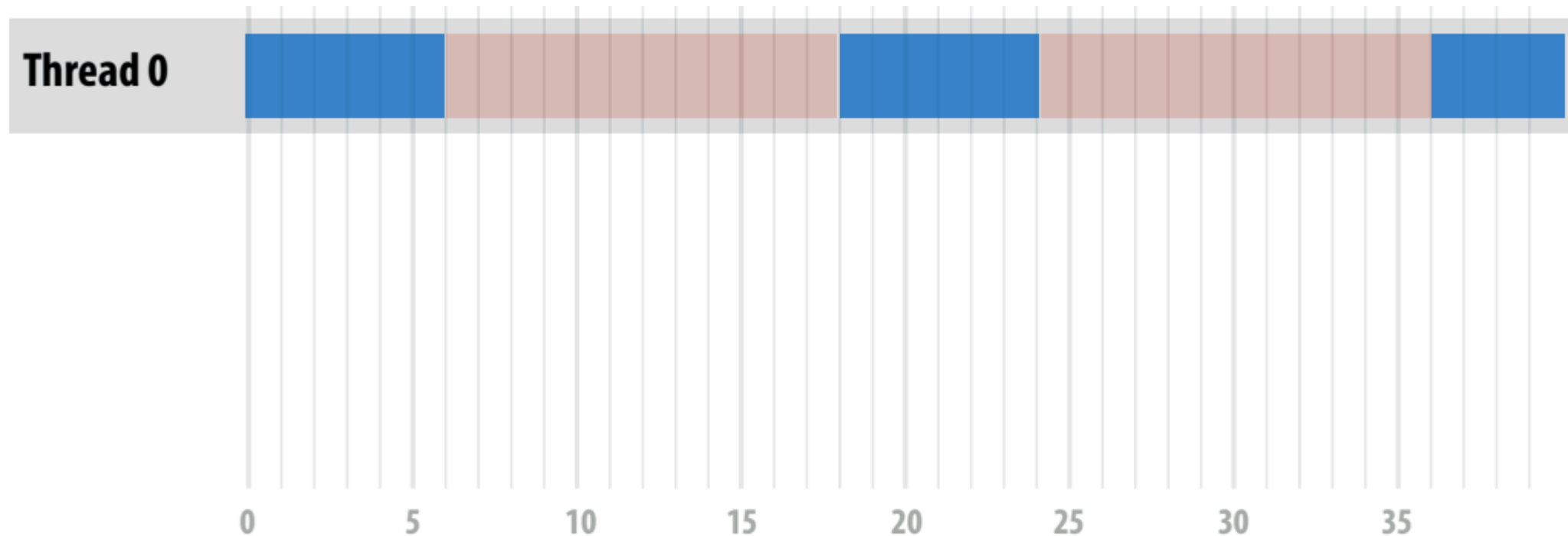


Additional threads yield no benefit (already 100% utilization)



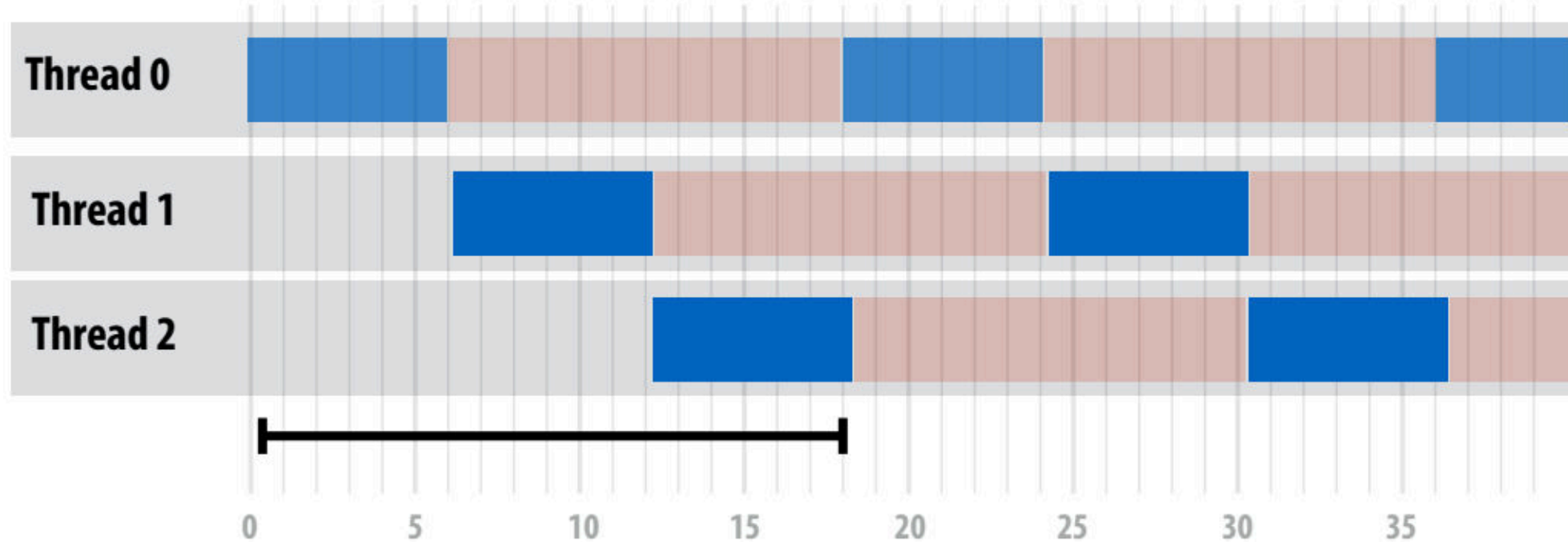
How many threads are needed to achieve 100% utilization?

- 100% 사용률을 달성하려면 몇 개의 스레드가 필요합니까?
- 이제 스레드는 6개의 산술 명령어를 수행한 후 메모리 로드(12주기 지연 시간)



Now only three threads needed for 100% utilization?

- 이제 스레드는 6개의 산술 명령어를 수행한 후 메모리 로드(12주기 지연 시간)



**100% utilization using
only three threads**

메모리 지연 시간 대비 수학 명령어 비율이 높으면 지연 시간 숨기기에 필요한 스레드 수에 어떤 영향을 미치나요?

Takeaway (point 1):

- 하드웨어 스레드가 여러 개 있는 프로세서는 하나의 스레드가 긴 대기 시간 연산을 완료할 때까지 기다려야 할 때 다른 스레드의 명령을 수행하여 멈춤을 방지할 수 있습니다.
- 참고: 메모리 작업의 지연 시간은 멀티스레딩으로 인해 변경되지 않으며, 단지 프로세서 사용률이 더 이상 감소하지 않을 뿐입니다.

Takeaway (point 2):

- 멀티스레드 프로세서는 다른 스레드에서 연산을 수행하여 메모리 지연을 숨깁니다.
- 메모리 액세스당 더 많은 연산을 수행하는 프로그램은 메모리 지연을 숨기기 위해 더 적은 수의 스레드가 필요합니다.

Hardware-supported multi-threading

- Hardware-Supported Multi-Threading은 여러 Thread를 Core가 Execution Context를 관리.
 - ➔ 즉, OS가 Thread를 관리하는 것이 아니라 매 Clock마다 어떠한 Thread를 실행시킬지를 Processor가 결정
 - ➔ 또한 Core는 여전히 동일한 개수의 ALU를 가지고 있어 Multi-Threading은 Memoery Access처럼 High-Latecny Operation에서 효율적으로 사용할 수 있도록 하는 역할을 함.
- Temporal Multi-Threading이라고 불리는 Interleaved Multi-Threading은 매 Clock마다 Core에 의해 선택된 하나의 Thread는 여러 ALU에서 Instruction을 실행하게 된다.
- Simultaneous Multi-Threading(SMT)는 매 Clock마다 Core가 여러 ALU에서 여러 Thread의 Instructions을 선택.
 - ➔ 이것은 SuperScalar CPU 디자인의 확장으로 볼 수 있다.

fictitious multi-core chip(가상 멀티-코어 칩)

16 cores

8 SIMD ALUs per core

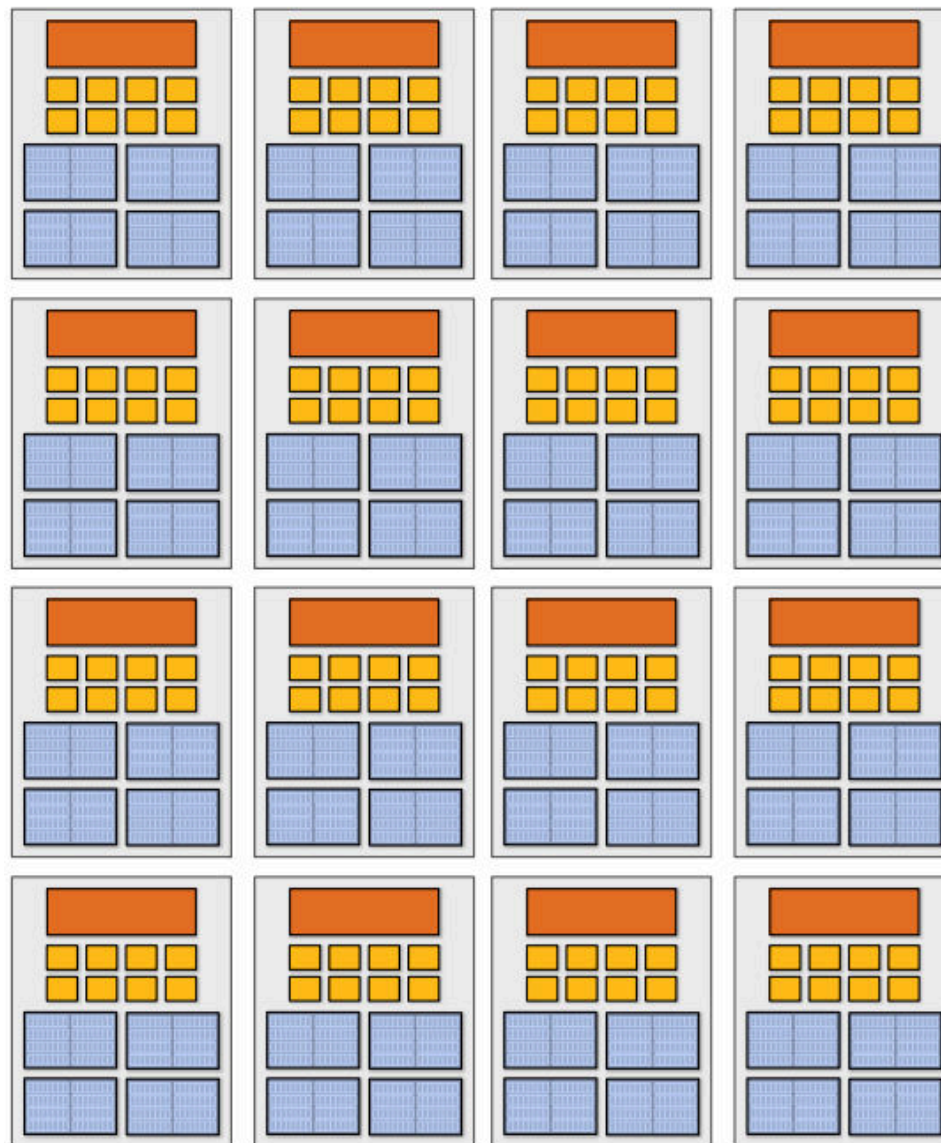
$16 \times 8 = (128 \text{ total})$

4 threads per core

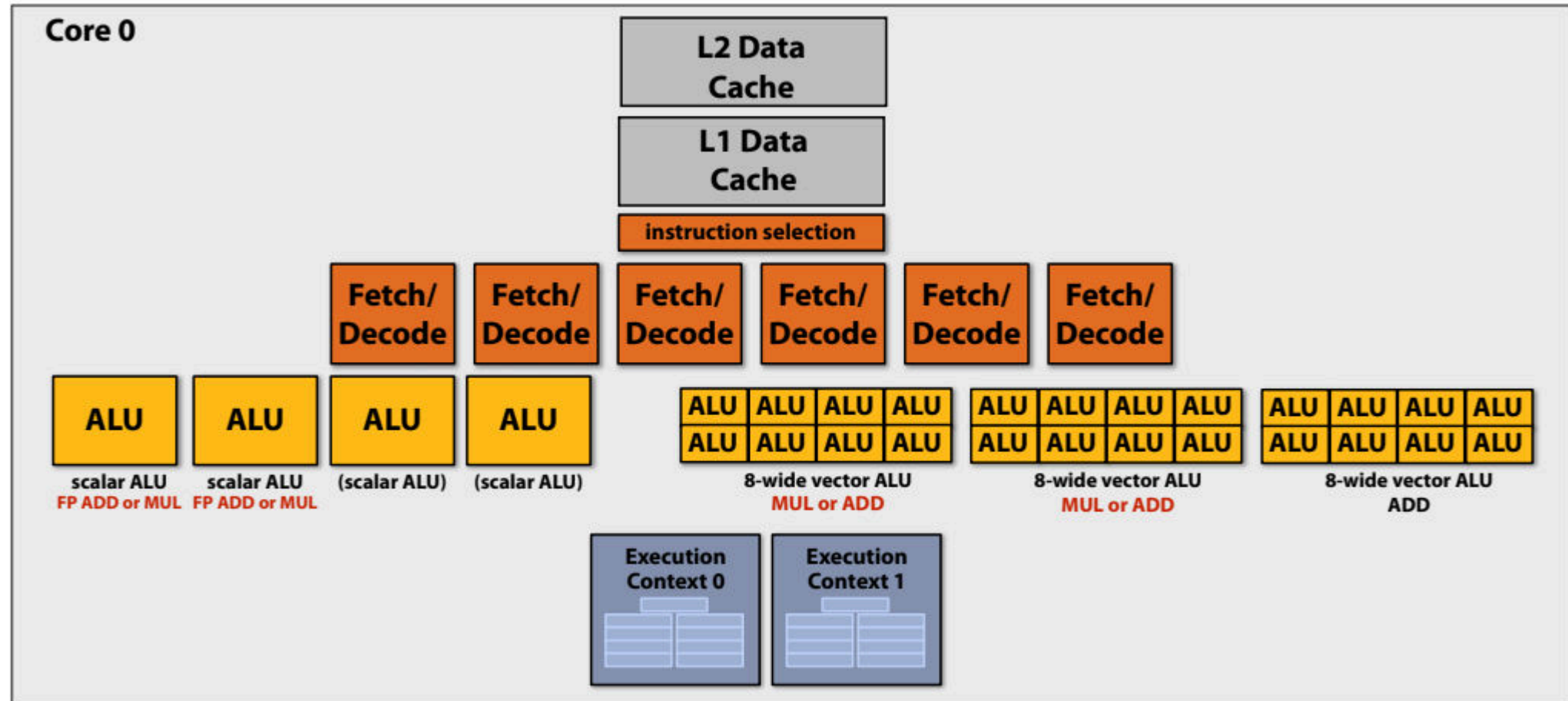
$16 \text{ core} = 16 \text{ simultaneous}$
instruction streams

$16 \times 4 = 64 \text{ total concurrent}$
instruction streams

$16 \times 4 \times 8 = 512 \text{ independent pieces of}$
work are needed to run chip
with maximal latency
hiding ability

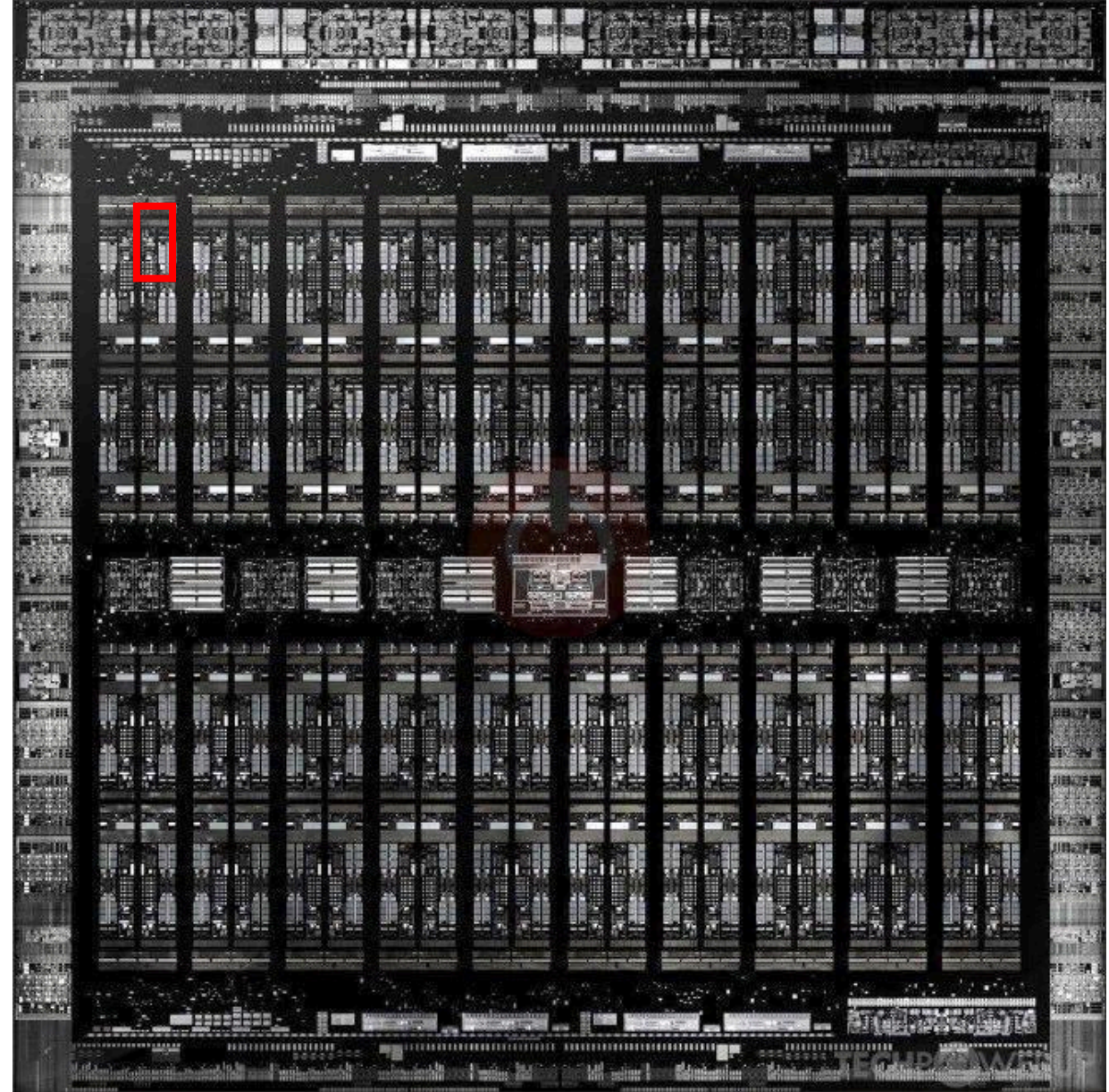


Example: Intel Skylake/Kaby Lake core



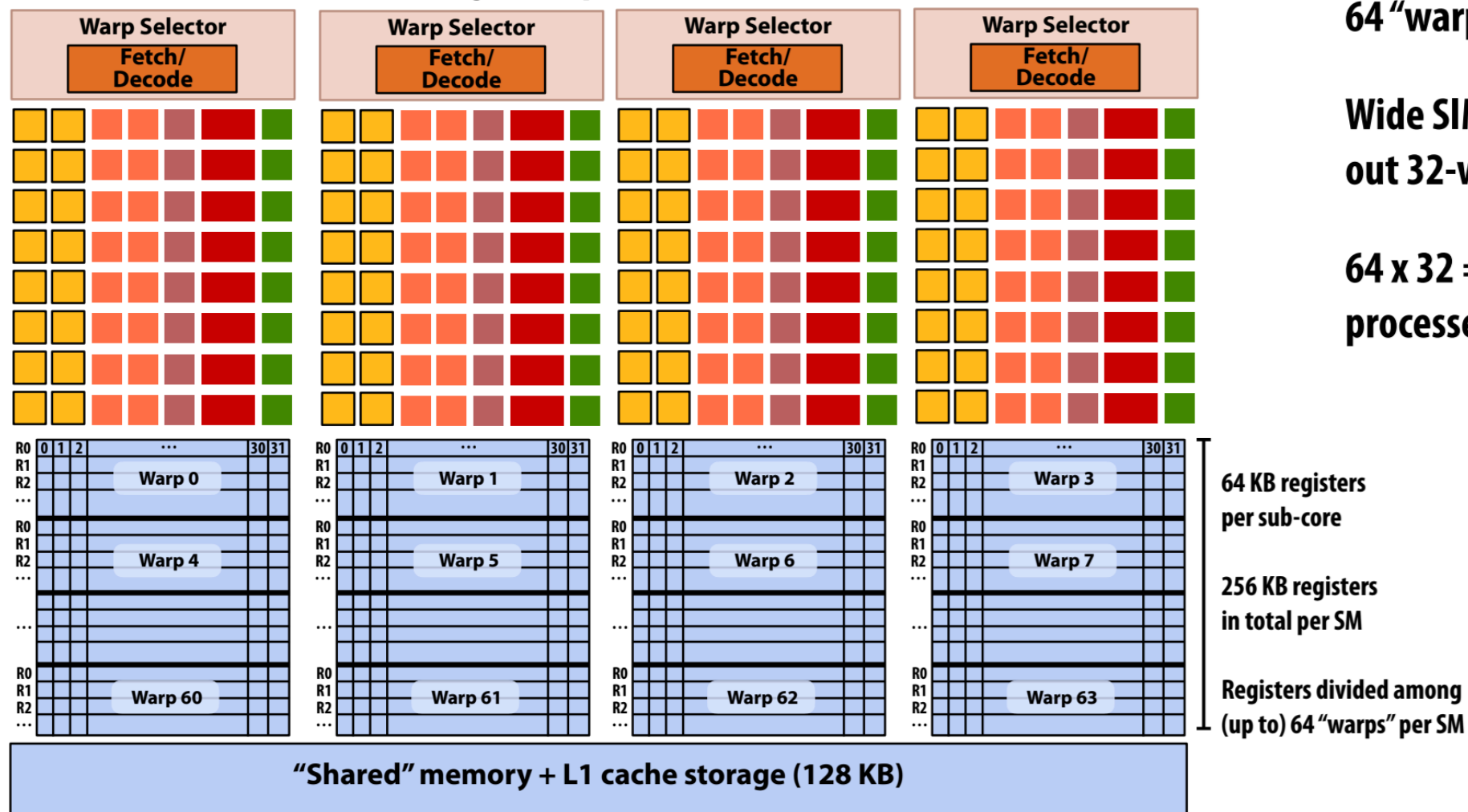
Two-way multi-threaded cores (2 threads).
Each core can run up to four independent scalar instructions
and up to three 8-wide vector instructions
(up to 2 vector mul or 3 vector add)

SM = "Streaming Multi-processor"



GPUs: extreme throughput-oriented processors

This is one NVIDIA V100 streaming multi-processor (SM) unit



64 "warp" execution contexts per SM

Wide SIMD: 16-wide SIMD ALUs (carry out 32-wide SIMD execute over 2 clocks)

64 x 32 = up to 2048 data items processed concurrently per "SM" core

 = SIMD fp32 functional unit, control shared across 16 units (16 x MUL-ADD per clock *)

 = SIMD int functional unit, control shared across 16 units (16 x MUL/ADD per clock *)

 = SIMD fp64 functional unit, control shared across 8 units (8 x MUL/ADD per clock **)

 = Tensor core unit
 = Load/store unit

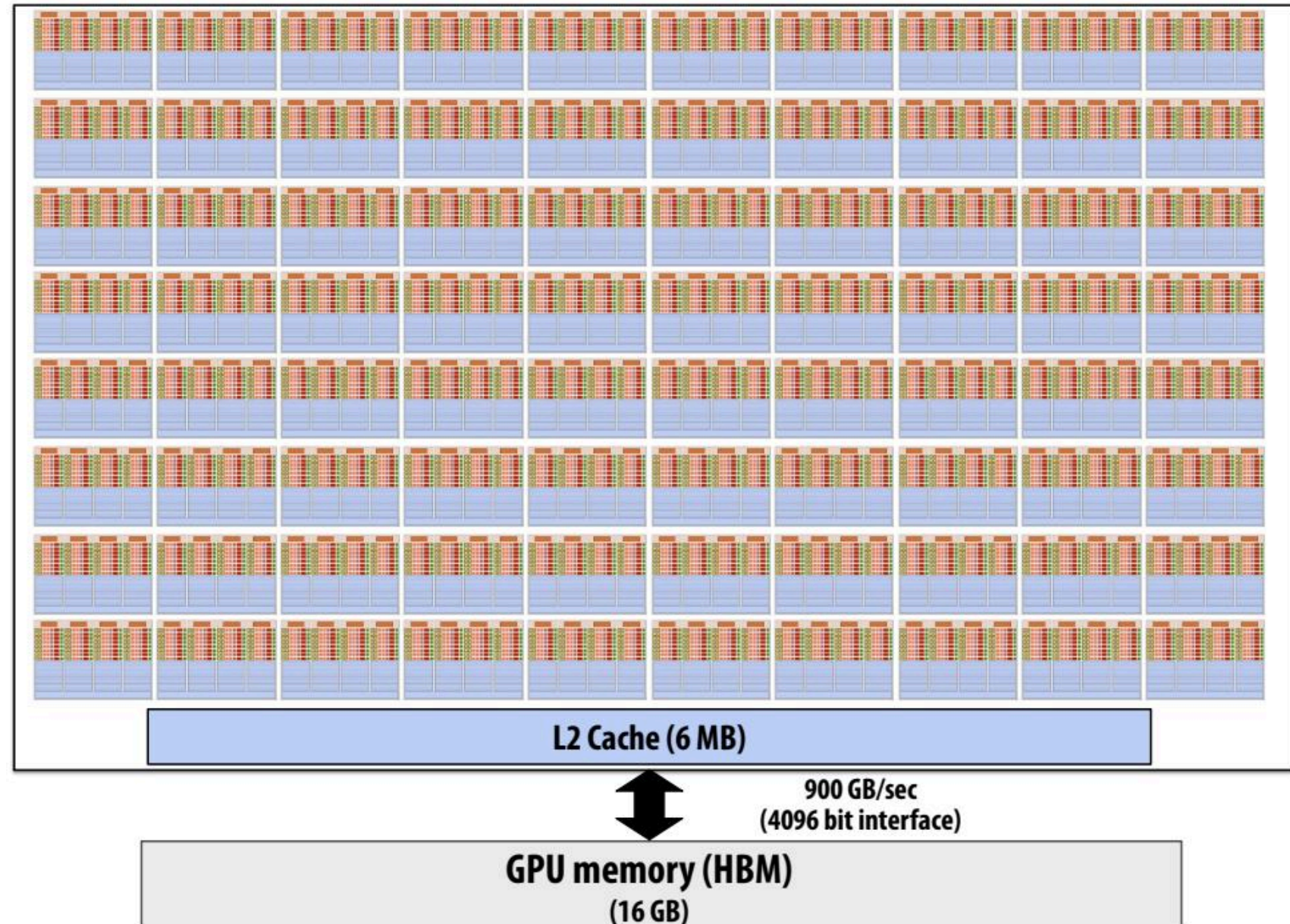
* one 32-wide SIMD operation every 2 clocks

** one 32-wide SIMD operation every 4 clocks

NVIDIA V100

V100에는 80개의 SM 코어가 있음:

이는 163,840개의 데이터를 동시에
처리하여 지연 시간을 최대한 줄일
수 있다는 뜻입니다!



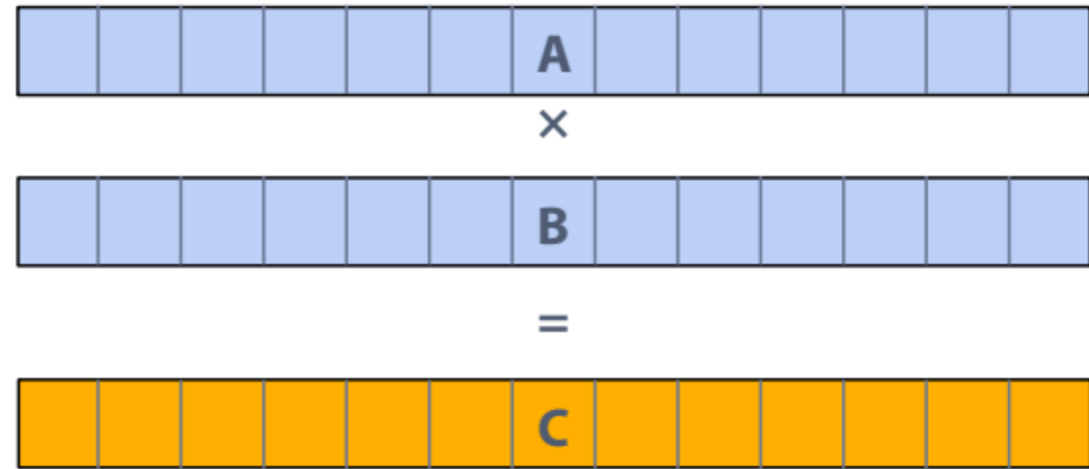
지금까지의 이야기...

최신 병렬 프로세서를 효율적으로 활용하려면, 다음과 같은 애플리케이션이 있어야 합니다:

1. 사용 가능한 모든 실행 유닛을 활용할 수 있는 **충분한 병렬 작업**(여러 코어 및 코어당 많은 실행 유닛) 보유
2. **병렬 작업 항목 그룹은 동일한 명령어 순서를 필요로 해야 합니다**(SIMD 실행을 활용하려면).
3. 프로세서 ALU보다 더 많은 병렬 작업을 노출하여 작업 인터리빙을 통해 메모리 정체를 숨길 수 있습니다.

Thought experiment

1. 작업: 두 벡터 A와 B의 요소별 곱하기
2. 벡터에 수백만 개의 요소가 포함되어 있다고 가정.
 - Load input A[i]
 - Load input B[i]
 - Compute $A[i] \times B[i]$
 - Store result into C[i]

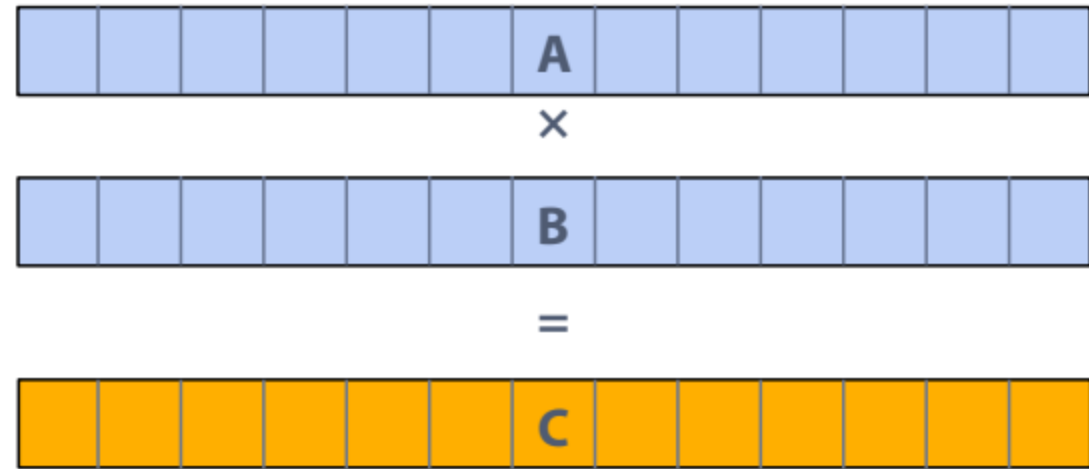


이 애플리케이션이 최신 처리량 지향 병렬 프로세서에서 실행하기에 좋은 애플리케이션인가요?



Thought experiment

1. 작업: 두 벡터 A와 B의 요소별 곱하기
2. 벡터에 수백만 개의 요소가 포함되어 있다고 가정.
 - Load input A[i]
 - Load input B[i]
 - Compute $A[i] \times B[i]$
 - Store result into C[i]



이 애플리케이션이 최신 처리량 지향 병렬 프로세서에서 실행하기에 좋은 애플리케이션인가요?



이 질문에 답하려면 먼저 지연 시간과 대역폭의 차이를 이해해야 함.

Thought experiment

학생을 위한 제안: 다음 용어를 숙지하세요.

- **Instruction stream**
- **Multi-core processor**
- **SIMD execution**
- **Coherent control flow**
- **Hardware multi-threading**
 - **Interleaved multi-threading**
 - **Simultaneous multi-threading**

REVIEW

모든 것이 어떻게 조화를 이루는지 살펴보세요:

**superscalar execution,
SIMD execution,
multi-core execution,
and hardware multi-threading**

Running code on a simple processor

C program source

```
void sinx(int N, int terms, float* x, float* y)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value;
    }
}
```

compiler

Compiled instruction stream (scalar instructions)

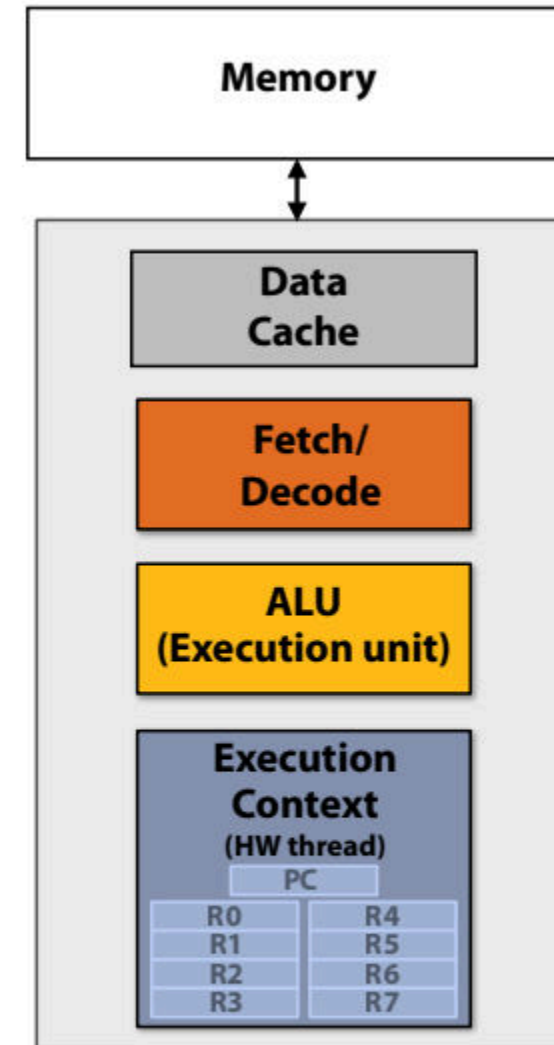
ld	r0, addr[r1]
mul	r1, r0, r0
add	r2, r0, r0
mul	r3, r1, r2
...	
...	
...	
...	
...	
st	addr[r2], r0

Running code on a simple processor

Instruction stream



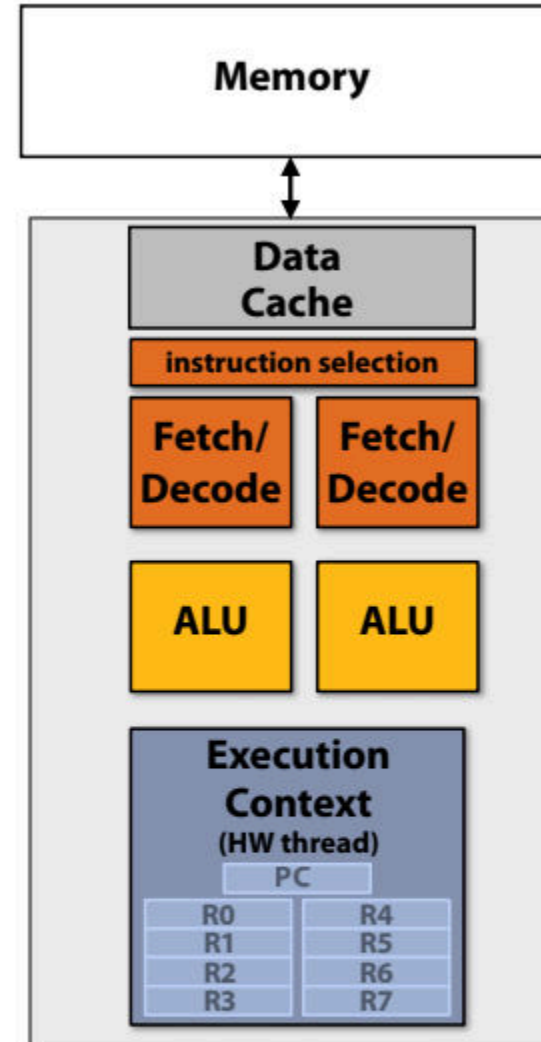
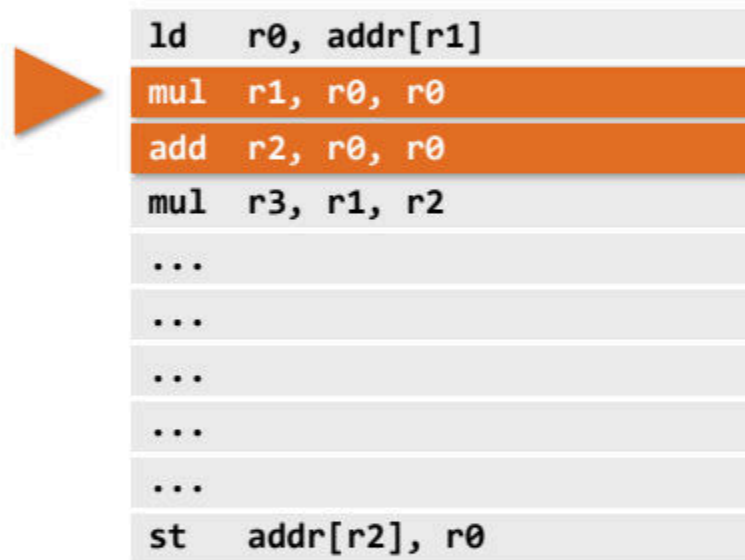
ld	r0, addr[r1]
mul	r1, r0, r0
add	r2, r0, r0
mul	r3, r1, r2
...	
...	
...	
...	
...	
st	addr[r2], r0



**Single core processor, single-threaded core.
Can run one scalar instruction per clock**

Superscalar core

Instruction stream



Single core processor, single-threaded core.

Two-way superscalar core:

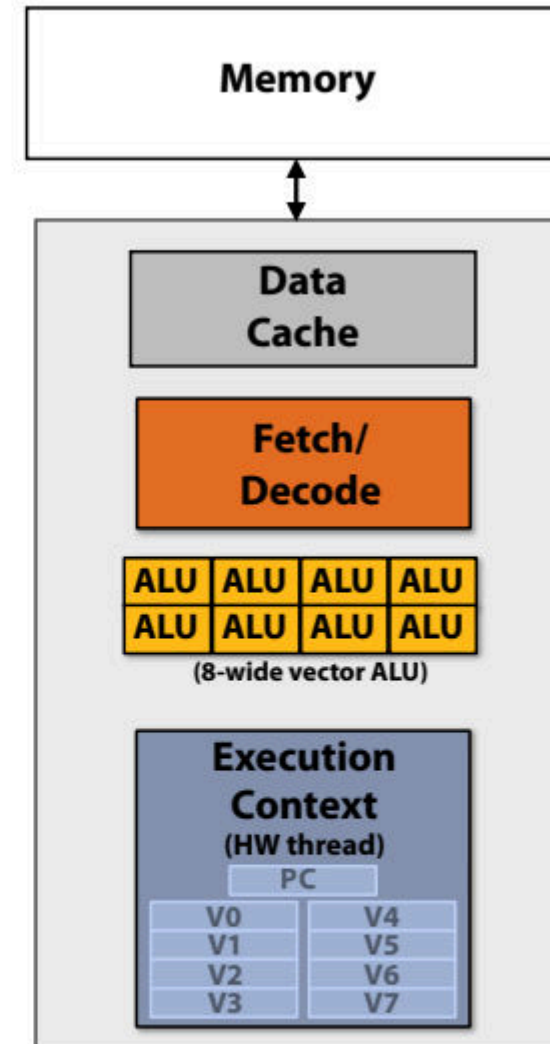
can run up to two independent scalar instructions
per clock from one instruction stream (one hardware thread)

SIMD execution capability

Instruction stream (now with vector instructions)



vector_ld	v0, vector_addr[r1]
vector_mul	v1, v0, v0
vector_add	v2, v0, v0
vector_mul	v3, v1, v2
...	
...	
...	
...	
...	
vector_st	addr[r2], v0

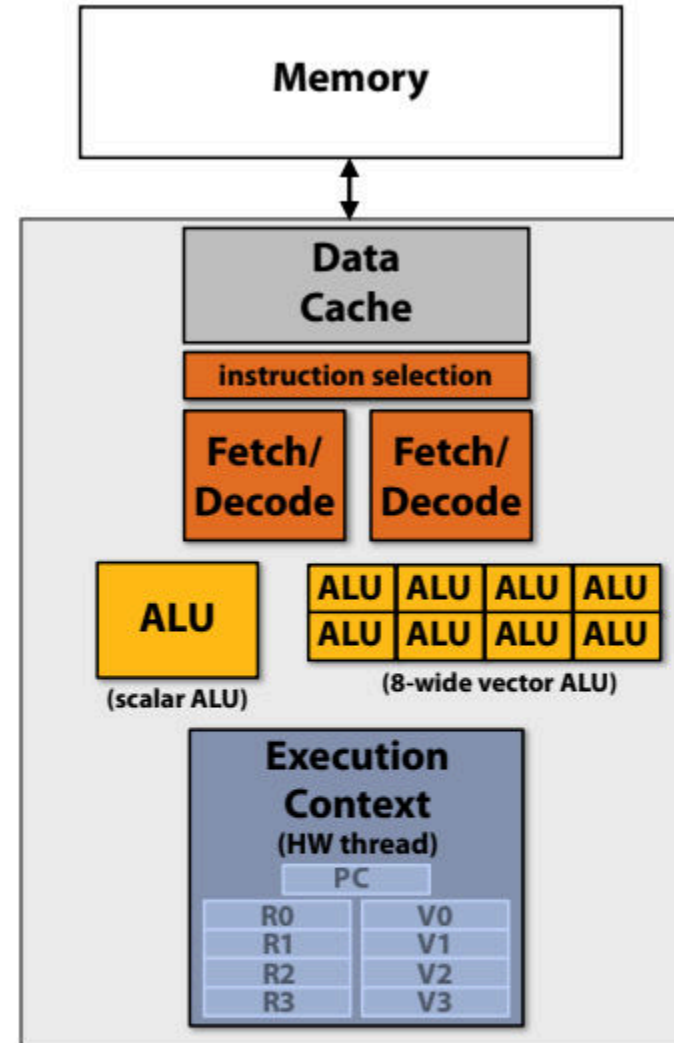


**Single core processor, single-threaded core.
can run one 8-wide SIMD vector instruction from
one instruction stream**

Heterogeneous superscalar (scalar + SIMD)

Instruction stream

vector_ld v0, vector_addr[r1]
vector_mul v1, v0, v0
add r2, r1, r0
vector_add v2, v0, v0
vector_mul v3, v1, v2...
...
...
...
...
vector_st addr[r2], v0

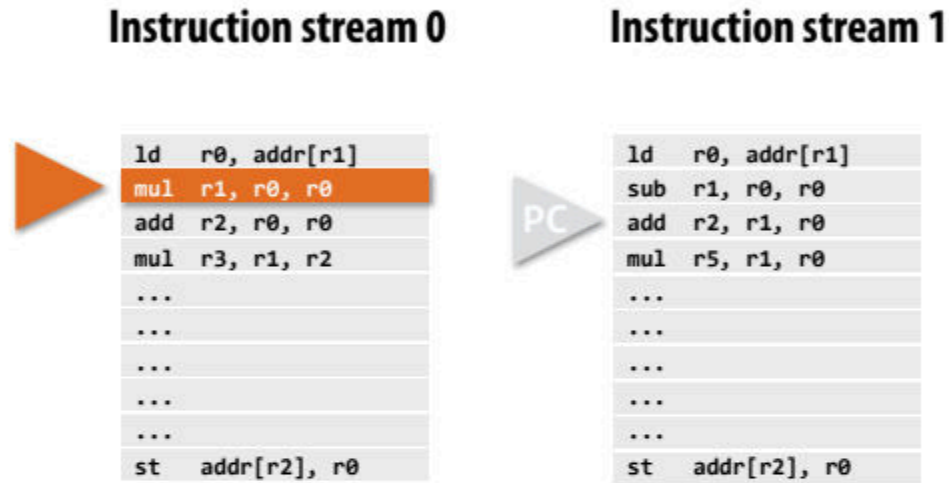


Single core processor, single-threaded core.

Two-way superscalar core:

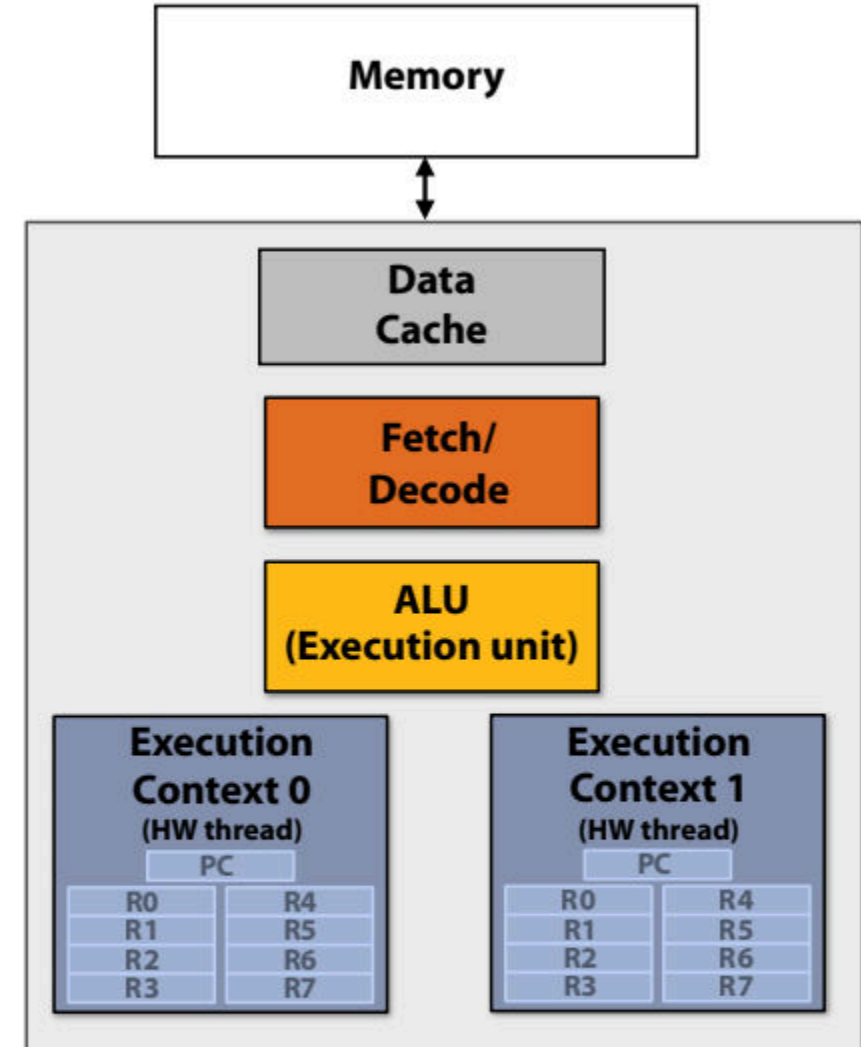
can run up to two independent instructions per clock from one instruction stream, provided one is scalar and the other is vector

Multi-threaded core



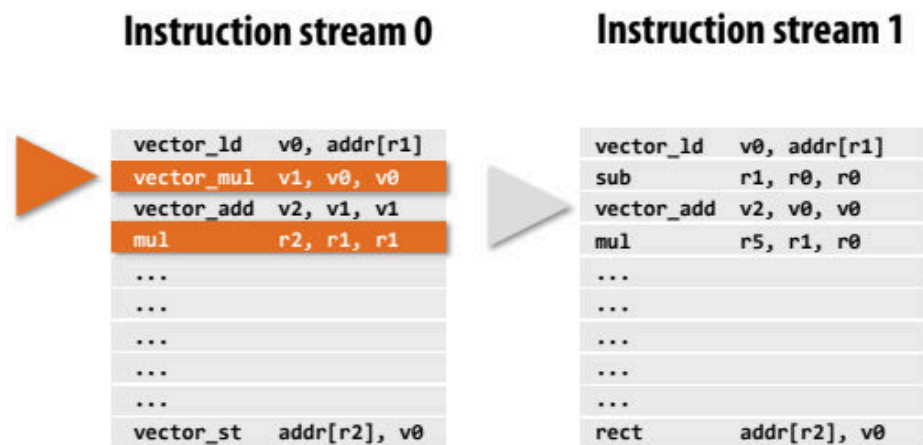
Note: threads can be running completely different instruction streams (and be at different points in these streams)

Execution of hardware threads is interleaved in time.



**Single core processor, multi-threaded core (2 threads).
Can run one scalar instruction per clock from one of the instruction streams (hardware threads)**

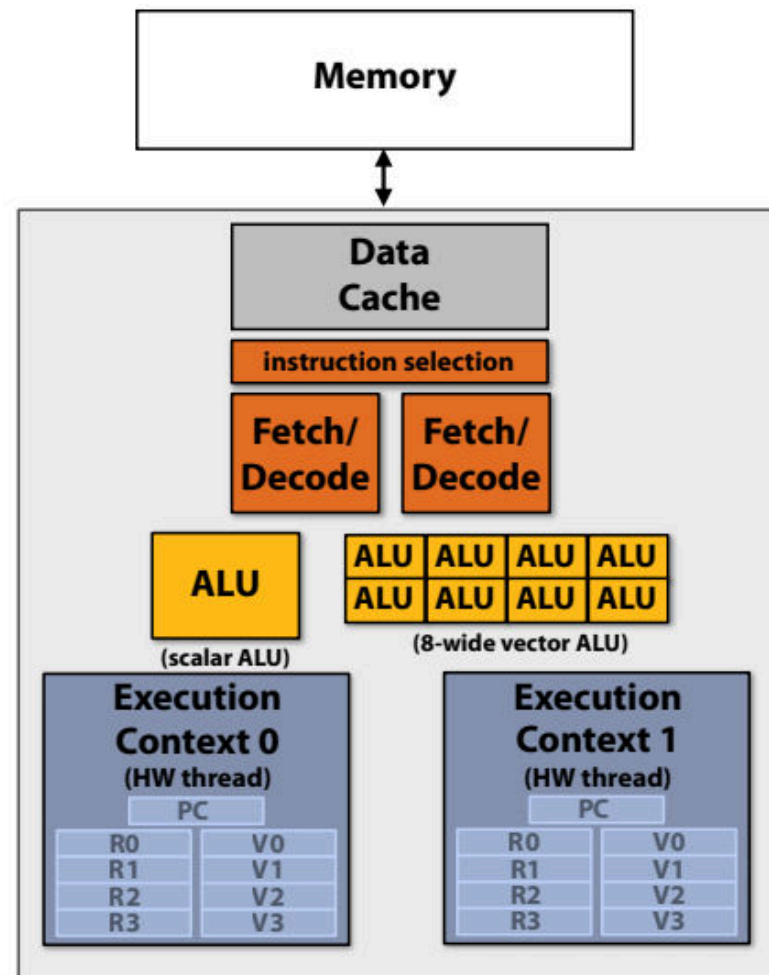
Multi-threaded, superscalar core



Note: threads can be running completely different instruction streams (and be at different points in these streams)

Execution of hardware threads is interleaved in time.

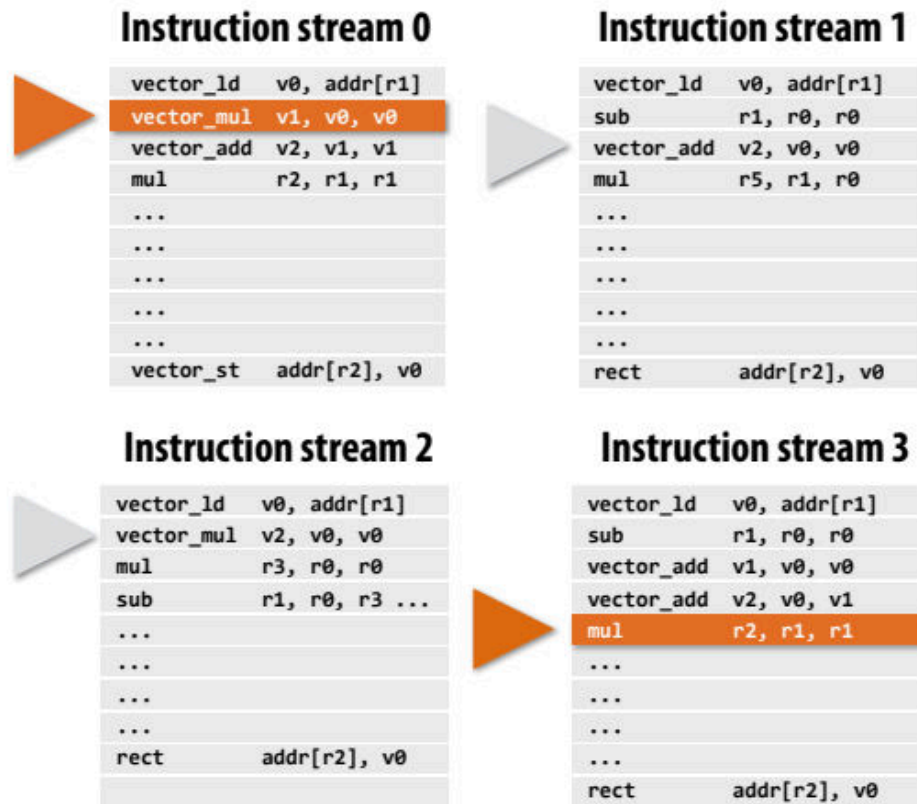
* This detail was an arbitrary decision on this slide:
a different implementation of "instruction selection" might run two instructions where one is drawn from each thread, see next slide.



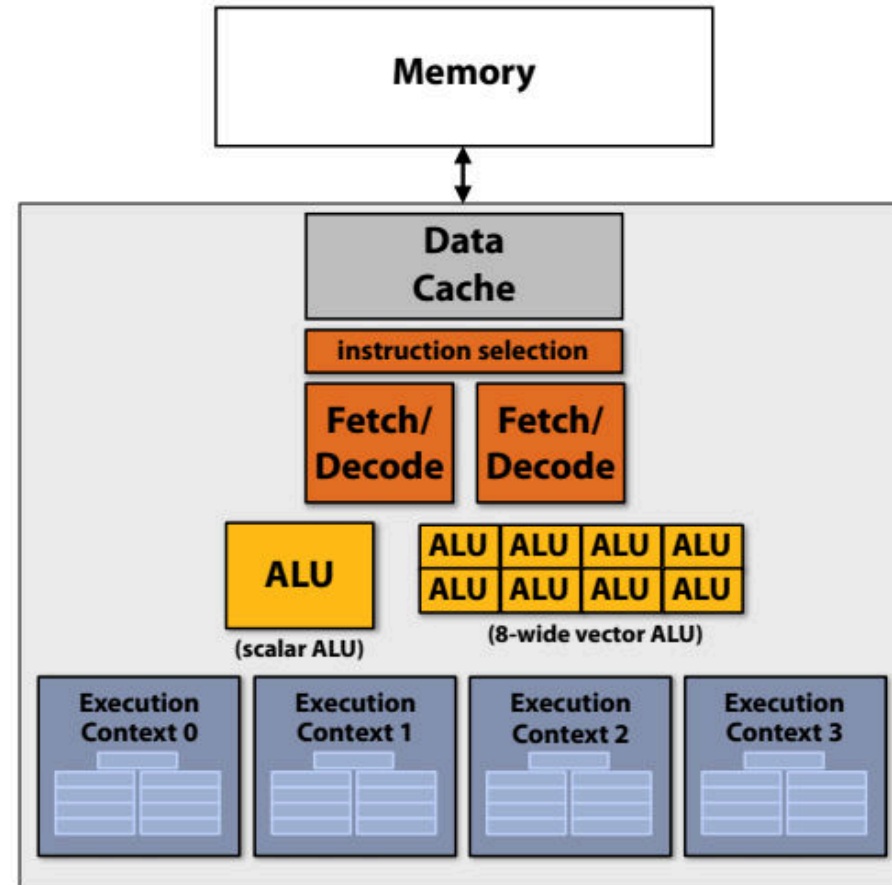
**Single core processor, multi-threaded core (2 threads).
Two-way superscalar core: in this example I defined my core as being capable of running up to two independent instructions per clock from a single instruction stream*, provided one is scalar and the other is vector**

Multi-threaded, superscalar core

(that combines interleaved and simultaneous execution of multiple hardware threads)



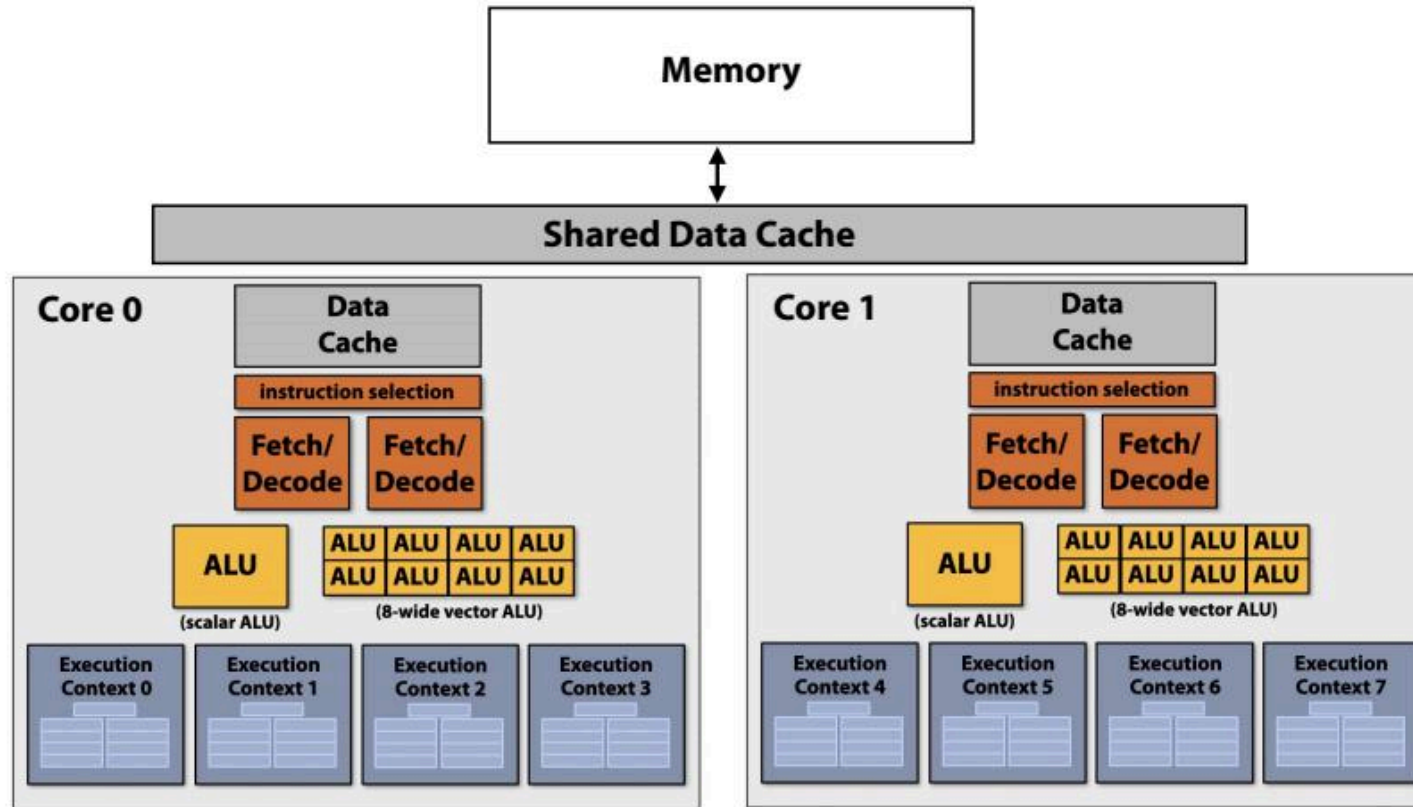
Execution of hardware threads may or may not be interleaved in time
(instructions from different threads may be running simultaneously)



Single core processor, multi-threaded core (4 threads).

Two-way superscalar core:
can run up to two independent instructions
per clock from any of the threads,
provided one is scalar and the other is vector

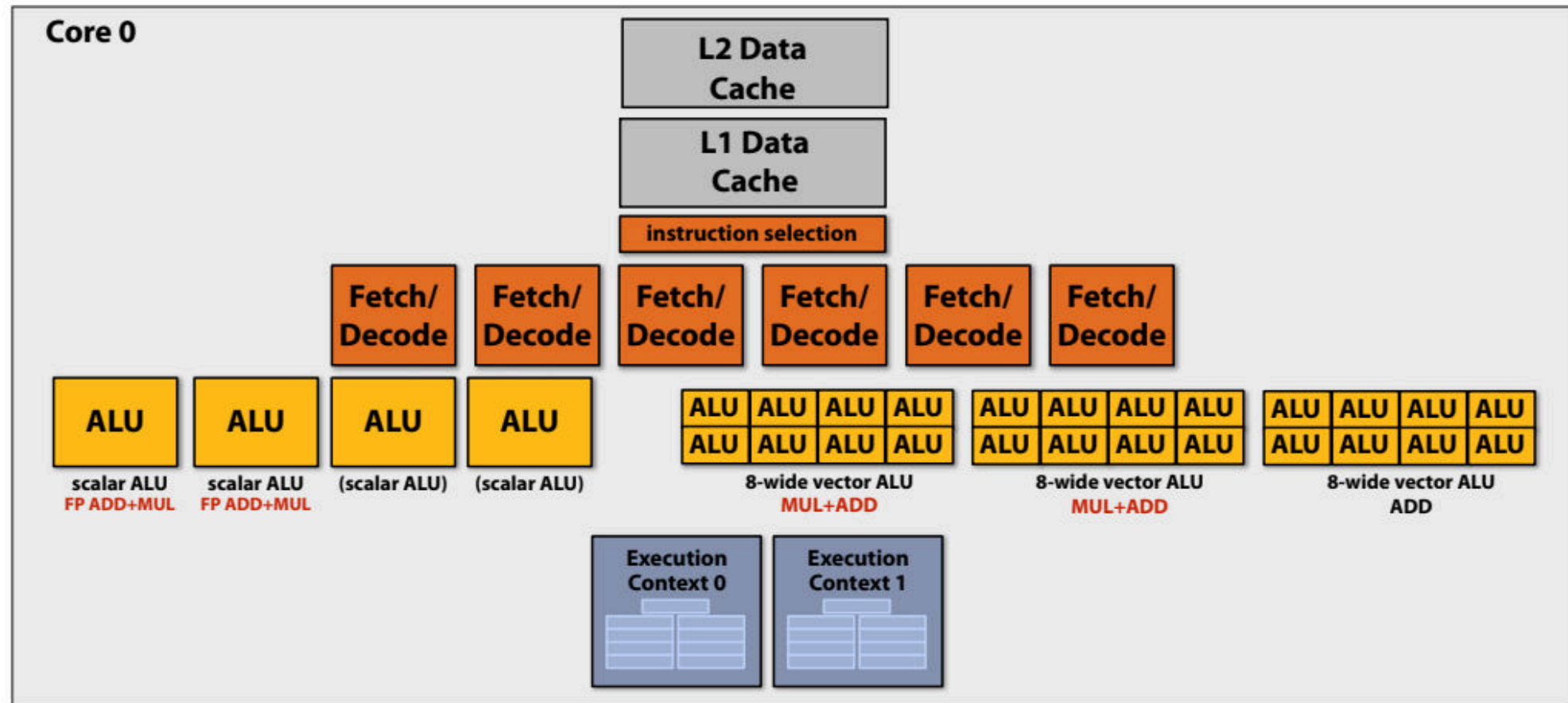
Multi-core, with multi-threaded, superscalar cores



Dual-core processor, multi-threaded cores (4 threads/core).
Two-way superscalar cores: each core can run up to two independent instructions per clock from any of its threads, provided one is scalar and the other is vector

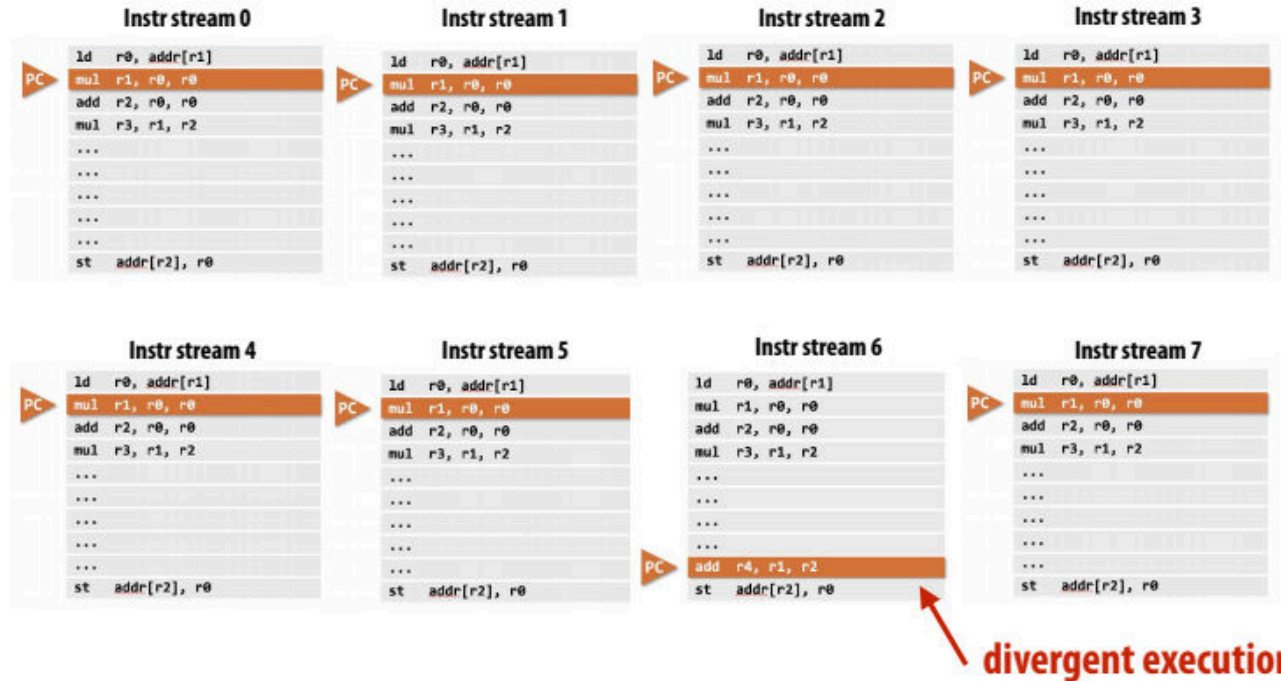


Example: Intel Skylake/Kaby Lake core



Two-way multi-threaded cores (2 threads).
Each core can run up to four independent scalar
instructions and up to three 8-wide vector instructions
(up to 2 vector mul or 3 vector add)

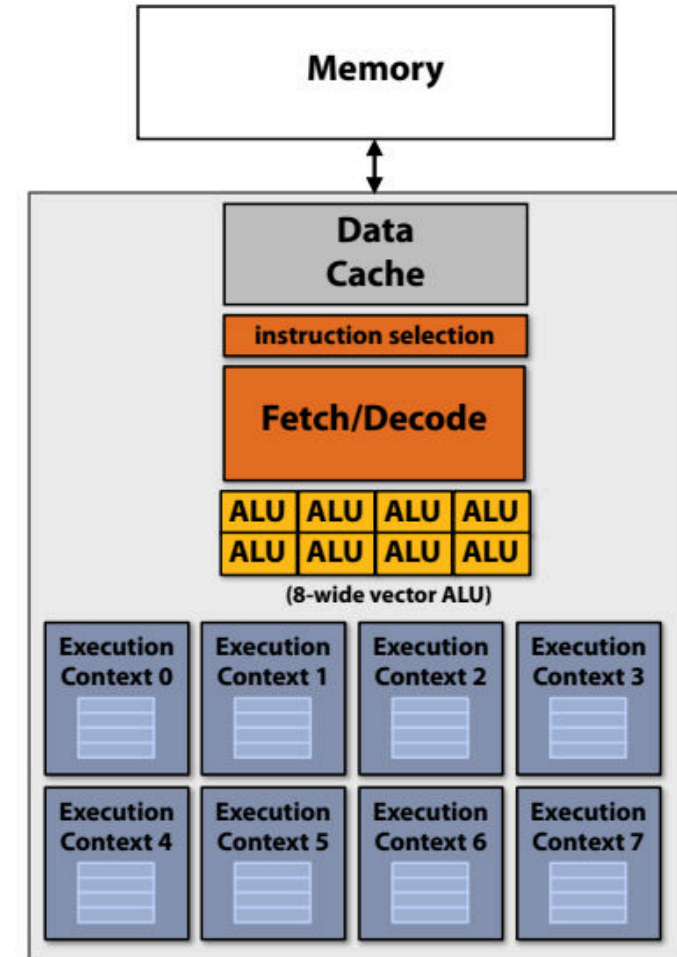
GPU “SIMT” (single instruction multiple thread)



Many modern GPUs execute hardware threads that run instruction streams with only scalar instructions.

GPU cores detect when different hardware threads are executing the same instruction, and implement simultaneous execution of up to SIMD-width threads using SIMD ALUs.

Here ALU 6 would be “masked off” since thread 6 is not executing the same instruction as the other hardware threads.



Thought experiment

- 두 개의 스레드를 생성하는 C 애플리케이션을 작성합니다.
- 이 애플리케이션은 아래 표시된 프로세서에서 실행됩니다.
 - 2개의 코어, 코어당 2개의 실행 컨텍스트, 클럭당 최대 2개의 명령어, 하나의 명령어는 8폭 SIMD 명령어입니다.
- 질문: 애플리케이션의 스레드를 프로세서의 스레드 실행 컨텍스트에 매핑하는 책임은 “누가” 집니까? **Answer: the operating system**
- 질문: OS를 구현하는 경우 두 개의 스레드를 네 개의 실행 컨텍스트에 어떻게 할당할 수 있나요?
- 또 다른 질문입니다: C 프로그램이 5개의 스레드를 생성하는 경우 실행 컨텍스트에 스레드를 어떻게 할당하나요?

